目录

Chapter 1 5
1.1 软件的定义 5
1.2 软件的特性 5
1.3 软件危机 5
1.4 软件工程定义 6
1.5 软件工程的核心要素 6
1.6 过程框架 7
1.7 框架性活动 7
1.8 过程对不同项目的适用性 8
Chapter 2 过程模型 8
2.1 过程分析模型 8
2.2 Process Flow 加工流程图 9
2.3 Identify TaskSet in Action 任务集 9
2.4 里程碑常用过程模型 10
2.4.1 The Waterfall Model 瀑布流 10
2.4.2 The Incremental Model 增量开发 11
2.4.3 The Incremental ModelApply Incremental Model 12
2.4.4 Evolutionary Models: Prototyping 演化模型 12
2.4.5 Prototyping 原型 12
2.4.6 Evolutionary Models: Spiral 螺旋模型 13
2.5 The Unified Process (UP) 14
2.5.1 Depict of UP Phases(1) 15
2.5.2 Depict of UP Phases(2) 15
2.5.3 Depict of UP Phases(3) 16
2.5.4 UP 裁剪 16
Chapter3 华为增量开发 16
Chapter 5 理解需求 22
5.1 需求的重要性 22
5.2 什么是需求 22
5.3 需求类别和来源 23
5.4 需求实例 24
5.5 需求过程 24
5.5.1 Inception 24
5.5.2 获取需求 25
5.5.3 My Experience(1) 26
5.6 构建分析模型 27
5.6.1 用例图 27
5.6.2 类图 28
5.6.3 状态图 28
5.7 需求沟通 29
5.8 规格说明 29
5.9 Validating Requirements - I 30
5.10 Validating Requirements - II 30

Chapter 6 Requirements Modeling: Scenarios, Information, and Analysis Classes 30
6.1 Requirements Analysis 30
6.2 Elements of Requirements Analysis 31
6.3 需求分析方法 31
6.3.1 OOAOOA & UML 32
6.3.2 Object Oriented Analysis 32
6.3.4 OOA 分析第一步：用例建模 34
6.3.5 OOA 分析第三步：领域建模 42
6.3.6 OOA 分析第四步：行为建模 53
Chapter 7 Requirements Modeling: Flow Oriented Modeling 60
7.1 结构化分析方法 60
7.2 Data Flow Diagram 60
7.2.1 Flow Modeling Notation 61
7.2.2 External Entity 61
7.2.3 Process 61
7.2.4 Data Flow 62
Data Stores 62
7.2.5 DFD 建模方法：详细 62
7.3 Example of DFD(1) 63
7.3 Control Flow Modeling 控制流 66
Example : 状态图 66
Chapter 8 软件设计 66
8.1 什么是软件设计 66
8.2 软件设计的价值 66
8.3 好的软件设计 67
8.4 软件设计如何实施 67
8.5 软件设计主要任务 67
8.6 软件设计模型 68
8.6.1 体系结构 68
8.6.2 类/数据设计 68
8.6.3 接口 69
8.7 Fundamental Concepts 69
数据抽象 69
8.7.1 过程抽象 69
8.7.2 体系结构优先 69
模块化 69
信息隐藏 70
8.7.3 逐步求精 70
8.7.4 功能独立 71
重构 71
设计类 71
Chapter 9 体系结构设计 72
9.1 理解体系结构及设计 72
(1) 什么是软件体系结构 72

(2) 体系结构设计中的关键决策 73
9.2 软件体系结构风格 73
9.2.1 数据中心结构 73
9.2.2 管道-过滤器结构 73
9.2.3 调用返回结构 74
9.2.4 面向对象结构 74
9.2.5 消息总线结构 75
9.2.6 层次化结构 75
9.2.7 客户-服务器结构 75
9.2.8 MVC 76
9.2.9 微服务架构 76
9.2.10 混合风格 76
9.3 体系结构描述 76
9.4 体系结构设计方法 77
9.4.1 面向对象设计方法 77
9.4.3 结构化设计方法 78
Chapter 10 构件设计 83
10.1 引入构件设计 83
10.1.1 什么是构件 83
10.1.2 结构化视角下构件设计 83
10.1.3 面向对象视角下构件设计 83
10.1.4 面向对象构件设计任务 84
10.2 构件设计指导原则 84
10.3 面向对象构件设计过程 86
10.4 传统构件设计过程 87
10.5 传统构件设计过程 87
Chapter 11 用户接口设计 88
11.1 UI 设计 88
11.2 常见的 UI 设计错误 88
11.3 黄金规则 89
11.3.1 让用户掌控一切 89
11.3.2 减少用户记忆 89
11.3.3 保持一致性 89
11.3.4 补充规则 89
11.4 UI 设计过程 90
11.4.1 界面分析 90
(2) 任务分析 90
11.4.2 界面设计 91
11.5 设计评审 91
Chapter 14 Quality Concepts 91
14.1 Quality Dimensions-McCall model 92
14.2 Conflicts among Quality Factors 92
14.3 Quality Model- 国标 92
14.4 Cost of Quality 92

14.5 Achieving Software Quality 93
Chapter 15 Review Techniques 93
15.1 Quality Control 93
15.2 What Are Reviews? 93
15.3 评审的价值 94
15.6 Informal Reviews 94
15.7 Formal Technical Reviews 94
15.8 Review Process 95
15.9 Review Reporting 95
Chapter 16 Software Quality Assurance 95
16.1 Software Quality Assurance 95
16.2 软件质量保证活动 96
Chapter 17 软件测试策略 96
17.2 测试的目的 96
17.3 软件测试策略 96
策略 1：测试不能取代评审 97
策略 2：谁来做测试 97
策略 3：测试什么时候开始 ？ 97
策略 4：测试过程 97
17.4 传统软件:单元测试 97
17.5 传统软件：集成测试 98
17.6 面向对象软件:单元与集成测试 99
17.7 确认测试 99
17.8 系统测试 100
17.8.1 恢复测试 100
17.8.2 安全测试 100
17.8.3 压力测试 100
17.8.4 性能测试 101
17.9 软件调试 101
Chapter 18 Testing Conventional Applications 102
18.1 关于测试的 2 个问题 102
18.2 What is a “Good” Test 102
18.3 测试用例设计方法 102
18.3.1 白盒测试 White-Box Testing 102
18.3.2 Basis Path Testing 103
18.3.3 Flow Gragh 104
18.3.4 Basis Path Testing 104
18.3.5 Independent Path 105
18.3.6 Control Structure Testing 105
18.3.7 Condition Testing 106
18.3.8 Data Flow Testing 106
18.3.9 Loop Testing 106
18.3.10 Black-Box Testing 黑盒测试 107

Chapter 1

1.1 软件的定义

软件 = 程序 + 数据 + 文档

程序：计算机可以接受的一系列指令，运行时可 以提供所要求的功能和性能。

数据：使得程序能够适当地操作信息的数据结构。

文档：描述程序的研制过程、方法和使用的图文资料。

1.2 软件的特性

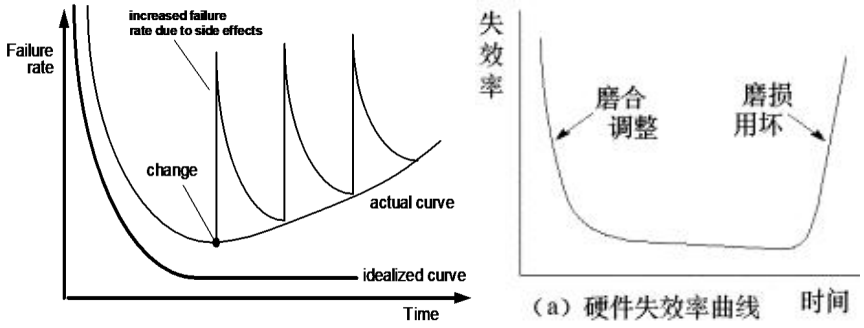
软件的生产和硬件不同，没有明显的制造过程。

软件不会磨损。

软件开发至今尚未完全摆脱手工开发方式。

软件是一种逻辑实体，具有抽象性。

软件的开发和运行受到计算机系统的限制，对计算 机系统有不同程度的依赖性。



（a）硬件失效率曲线

1.3 软件危机

软件危机是指落后的软件生产方式无法满足迅速增长的计算机软件需求，从而导致软件开发与维护过程中出现一系列的 严重的问题的现象。

软件危机的表现:



软件危机产生的原因

一方面是与软件的复杂性和本身的特点有关；另一方面由软件开发、维护、管理方法不正确有关.

1.4 软件工程定义

1968 年，北大西洋公约组织首次提出“软件工程”概念。

软件工程主要研究软件生产的客观规律，建立与系统化软件生产有关的概念、原则、方法、技术和工具，指导和支持软件系统的生产活动，以期达到降低软件生产成本 、改进软件产品质量、提高软件生产率的目标。

1.5 软件工程的核心要素

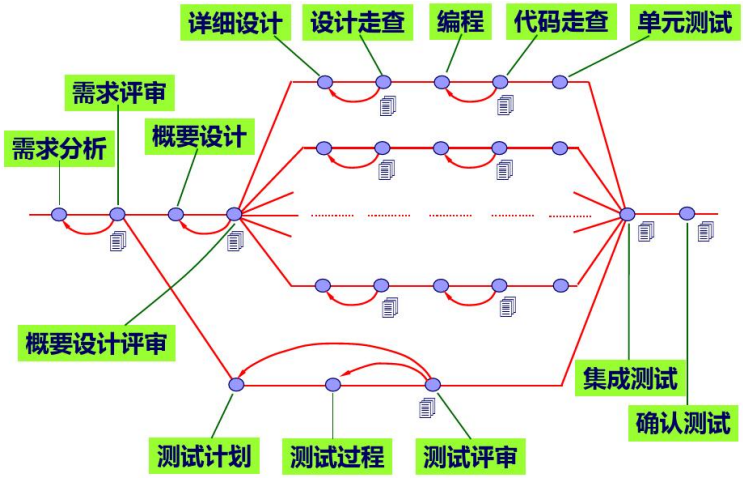
tools
methods
process model
“quality” focus

1) Software engineering must rest on an organizational commitment to quality.
质量是软件产品的生命线，软件工程以质量保证为基础。

2) 过程

过程定义了解决特定问题或目标达成涉及的一系列步骤、活动以及交付的内容等。

软件开发包括若干子过程：开发过程、维护过程、管理过程等；



（1）过程影响产品质量；　（2)不同过程适用于不同项目软件工程的核心要素

3) 方法

定义“**开发过程**中每项活动，任务实现的**技术手段**”。

Methods encompass a broad array of tasks that include **requirements analysis, design, program construction, testing,　and support.**

4) 工具

为软件开发方法提供自动的或半自动的软件支撑工具或环境。如分析工具、设计工具、编译工具、测试工具等。当这些工具被集成起来使得一个工具产生的信息可以被另外一个工具使用时，一个支持软件开发的系统就建立了，称为**计算机辅助软件工程**（CASE）。

如华为 CodeArts，可即开即用，随时随地在云端进行项目管理、代码托管、在线编码、代码检查、编译构建、部署、测试、发布等。

1.6 过程框架

过程框架定义了若干框架活动，较粗粒度的说明了软件过程需要具备的基本活动。

这些活动可普遍应用于所有软件开发项目,无论项目的规模和复杂性如何。

活动可分为两类

框架性活动- 面向开发

普适性活动- 面向管理

1.7 框架性活动

Communication　Planning　Modeling　Analysis of requirements　Design　Construction　Code generation　Testing　Deployment

层次结构: 每个活动可细分至任务普适性活动



普适性活动主要帮助软件团队管理和控制项目进度、质量、变更和风险。

Software project tracking and control
Formal technical reviews
Software quality assurance
Software configuration management
Work product preparation and production
Reusability management
Measurement
Risk management

1.8 过程对不同项目的适用性

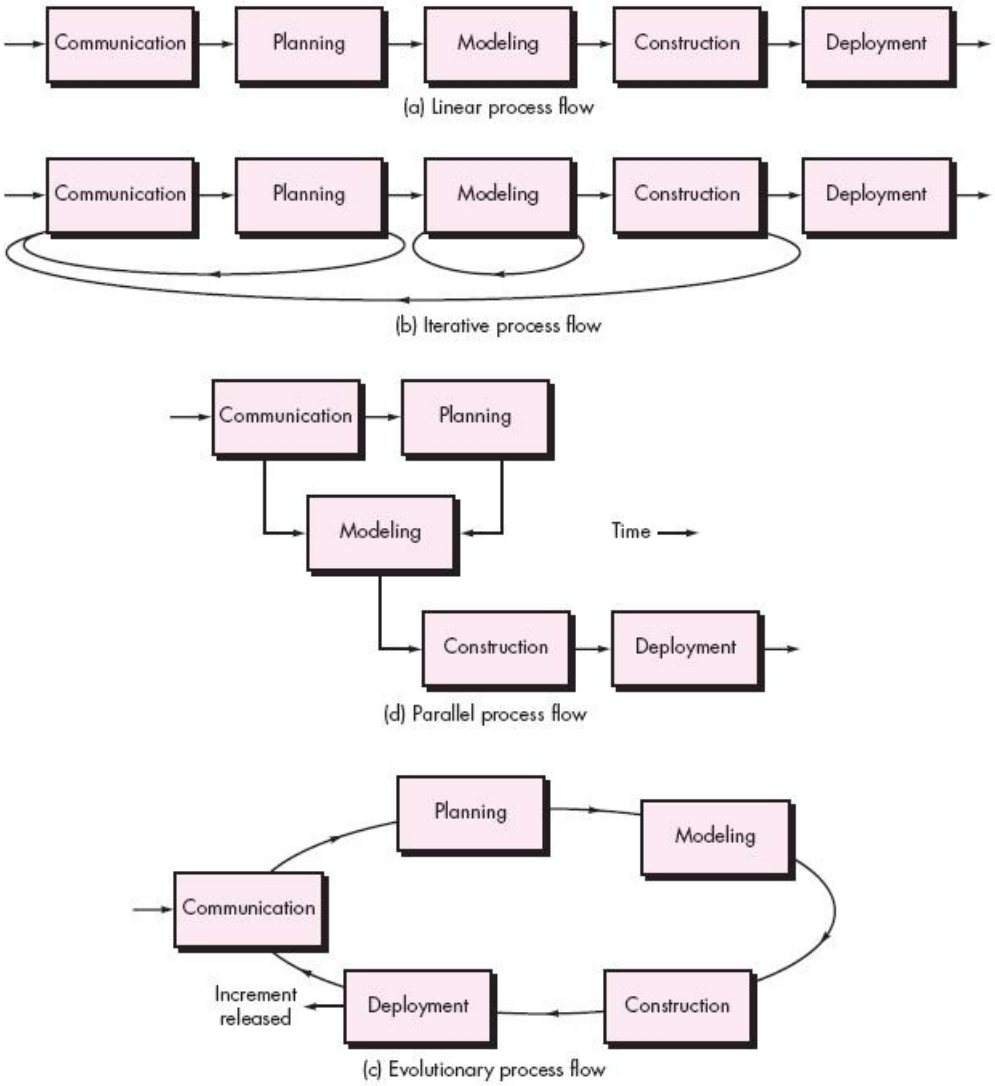
每个企业可以定制自己的过程模型。
不同的软件项目，应该采用相适应的软件过程。

Chapter 2 过程模型

2.1 过程分析模型

软件过程是软件产品构建时所执行的一系列活动、动作和任务的集合。
成熟的软件企业都会定义自己的软件过程模型。
过程模型定义
Define process flow
Define actions in each activity
Define task sets in each action

2.2 Process Flow 加工流程图



2.3 Identify TaskSet in Action　任务集

如果过程模型中的动作(Action)还不具备可操作性,可以继续将其细化为任务集。

任务集主要包括：任务、工作产品、质量保证点、里程碑。

例如：对业务需求分析，可以继续细分为下列任务

业务需求分析	准备调研提纲、问卷	咨询实施顾问	《需求调研提纲》	《一般业务调研提纲》
				《分产品的调研提纲》
				《行业调研提纲》
	制定调研计划	**项目经理	《需求调研计划》	《需求调研计划》
	详细业务需求调研	**项目经理/咨询实施顾问		
	需求和产品匹配分析	**项目经理		
	需求分析报告	**项目经理	《需求分析报告》	《需求分析报告》

Project milestones

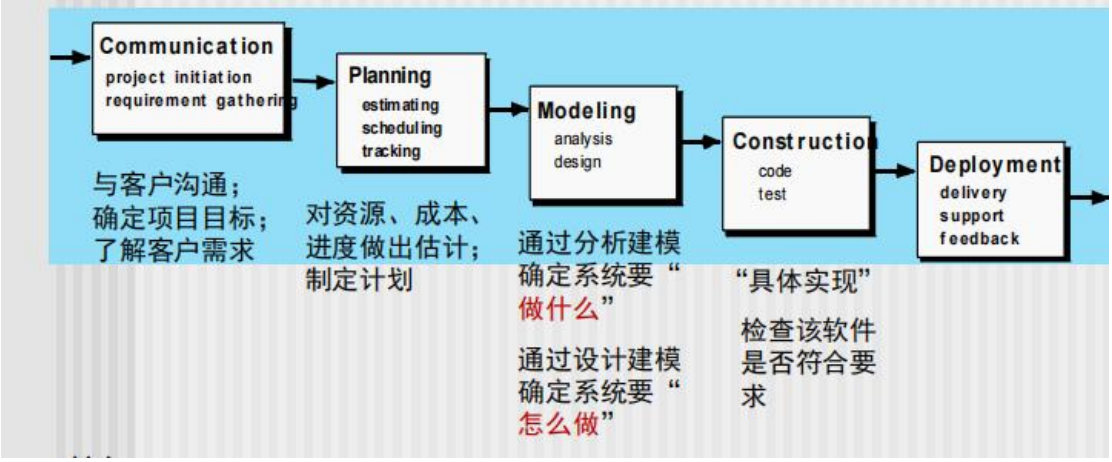
里程碑是开发过程中某个重要的时间节点，识别标志性工作的完成情况.如：需求分析结束，就应该设置一个里程碑，对应标志性工作 PRD 的完成。

2.4 里程碑常用过程模型

业界提供了很多可参考和复用的过程模型。

从高层次上来看，模型中的基本活动没有多大变化，变化主要在于这些基本活动次序的安排以及活动持续时间等方面。

2.4.1 The Waterfall Model 瀑布流



特征：

强调了每一阶段活动的严格顺序。

推迟实现。

质量保证：每个阶段均提供文档（里程碑），且需要通过技术评审，才能进行下一阶段的工作.

主要缺点:

并非完全遵循顺序开发模式。

要求用户在项目一开始就完整、准确的陈述需求是不现实的。

用户必须有足够耐心，在项目接近完成前，产品不能投入使用，这增加了项目失败的风险。

早期的错误可能要等到开发后期的测试阶段才能发现，进而带来严重后果。

适合于：

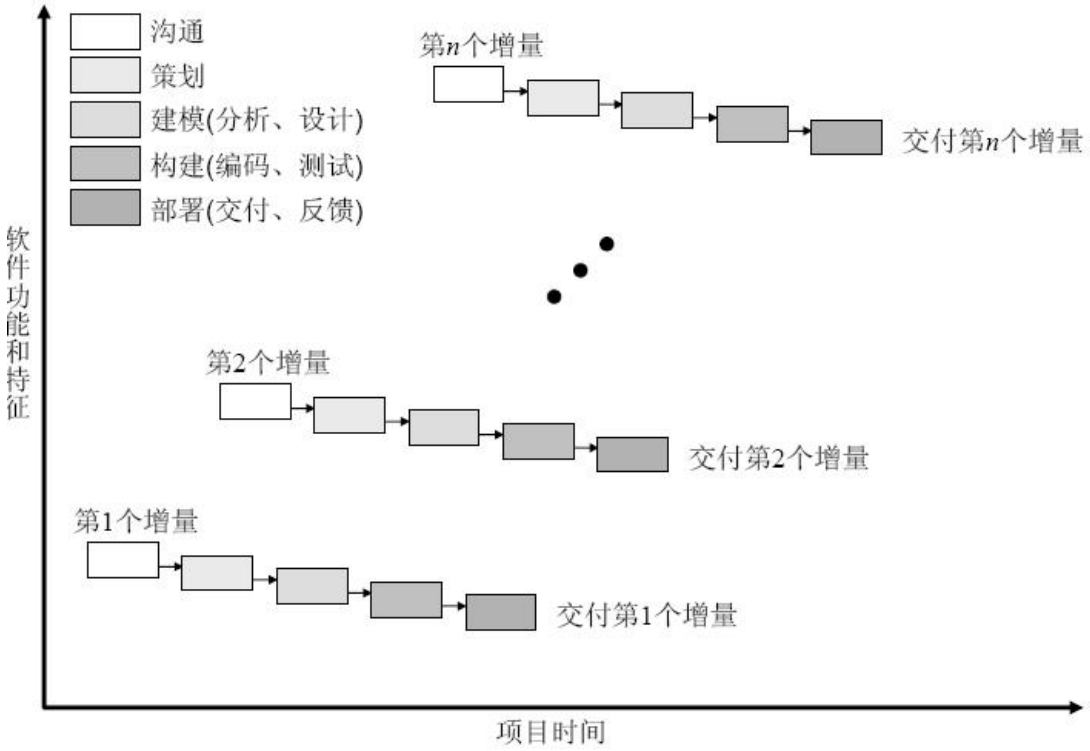
需求稳定的项目

开发团队对该应用领域非常熟悉

2.4.2 The Incremental Model 增量开发

The incremental model combines elements of linear and parallel process flows.

Each linear sequence produces a deliverable "increment" of the software.



特征：

开发人员可以自己定义每个增量的内容和优先级，并优先完成重要的、需求稳定的核心增量产品。

由于一些模块必须在另一个模块之前完成，所以注意增量间的顺序和良好的接口。

在连续增量中，需要将用户反馈融入到细化的产品中。

优势：

可提高对用户需求或市场需求的响应。

在团队成员不足以支持项目完整开发时尤其适用。

通过将需求不确定、不完全掌握相关技术的功能放在后续增量开发，可降低风险。

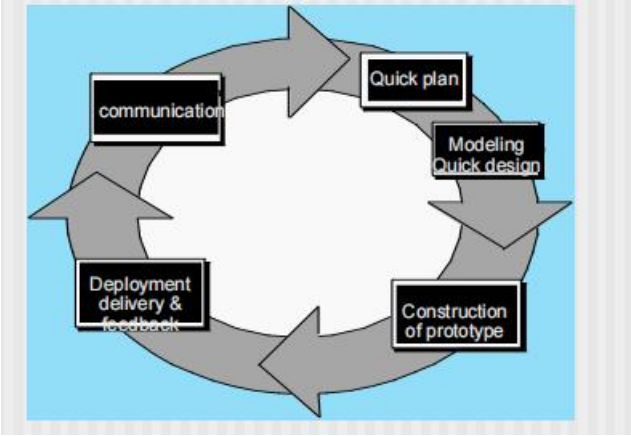
缺点：需要大量回归测试，增加测试工作量。

2.4.3 The Incremental ModelApply Incremental Model

例如：通过增量模型，开发一文档编辑系统

增量 1：提供基本的文档生成、编辑、存储功能；
增量 2：提供更高级的编辑功能，如图表编辑等；
增量 3：提供语法检查功能；

2.4.4 Evolutionary Models: Prototyping 演化模型



演化模型的本质：通过迭代中一系列活动的重复应用，使软件持续改进。

基本思想是: 通过构建可视化的原型帮助开发团队和用户更好地理解和沟通需求，减少开发过程中的风险和不确定性。

2.4.5 Prototyping 原型

原型可以是静态的，如页面布局图或线框图，也可以是动态的，如交互式模型。

原型的应用方式

(1) 抛弃型：主要用于需求阶段，搞清楚用户需求，之后就不再使用原型。
(2) 演化型：从原型开始，直至演化得到最终的产品。

Apply Prototyping(1)

例：应用抛弃型原型法，开发某库存系统

Apply Prototyping(2)

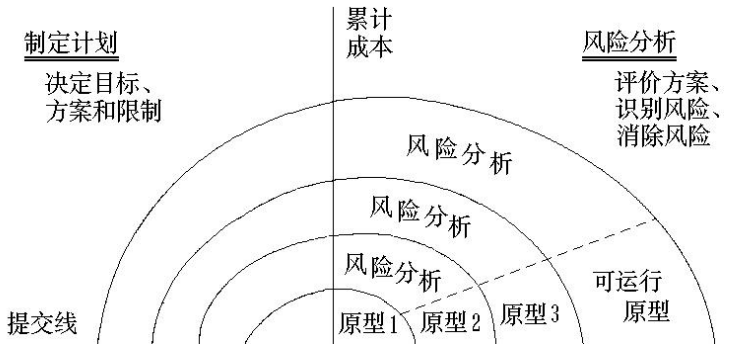
例：应用演化型原型法，开发“教务管理系统”
第一次原型迭代：构建交互式原型，不可实际执行。
第二次原型迭代：根据交互式原型得到实际系统，完成基本的学籍管理、课程管理、成绩管理功能。
客户反馈：基本满意，但数据量变大时执行效率下降。
第三次原型迭代：修改数据库读取策略，提高成绩统计和报表执行效率。

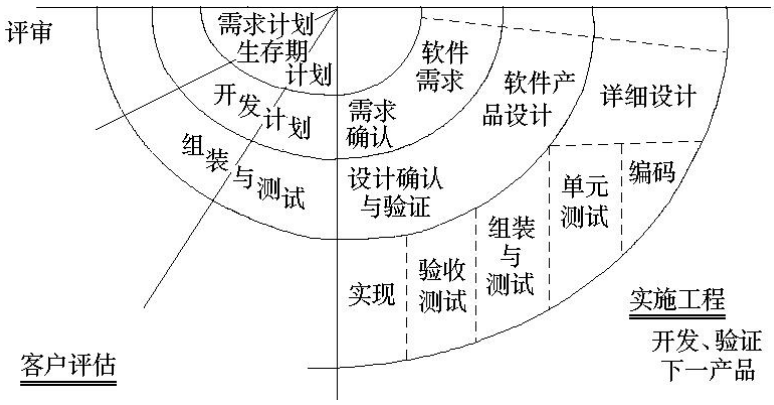
优点
有效避免开发人员和用户在需求理解上的不一致。
可以及时暴露问题，及时反馈优化；

缺点
没有严格的开发文档，后期维护困难。
开发人员为快速构建原型，往往在质量上作出妥协（例如：只关注局部而非整体结构，选择简单的算法）
需要快速原型开发工具和客户的深度参与，否则原型开发容易失败。

适用情况
适用于规模不大的系统 或 需求不确定的系统。

2.4.6 Evolutionary Models: Spiral 螺旋模型

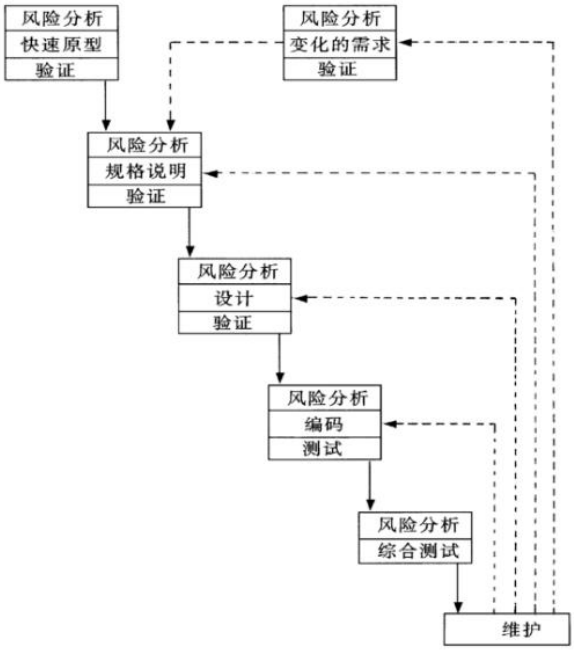




结合了原型模型和瀑布模型的特征，是一种风险驱动的过程模型。

要求不断迭代，同时象螺旋一样不断前进，即对整个开发过程进行迭代，而不仅仅对编码、测试进行迭代。

可以把螺旋模型看作在每个阶段之前都增加了风险分析过程的原型开发模型



特点： 螺旋模型适用原型作为风险分析和控制的机制，在迭代中逐步加深系统定义和实现的深度。

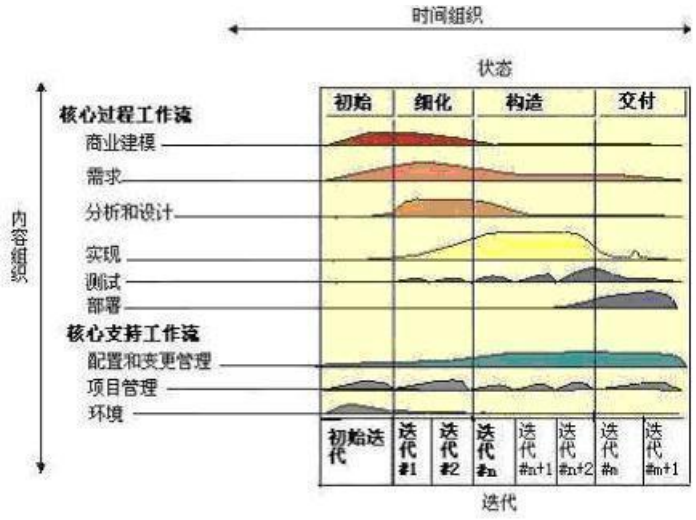
缺点： 过多的迭代次数可能增加开发成本。

适合于: 大规模软件项目、高风险项目

2.5 The Unified Process (UP)

UP 将项目生命周期分为四个阶段：

初始 细化 构造 交付



每一个阶段由若干次迭代组成，只不过每一次迭代开发的侧重点不同（每个阶段集中力量做正确的事）。

一个迭代构成一个完整的开发循环，产生一个可执行的产品版本，是最终产品的一个子集。

2.5.1 Depict of UP Phases(1)

在 UP 初始阶段的迭代中，项目组必须：

确定开发目标与范围

评估潜在的技术、商业风险

制定迭代实施计划

识别系统的关键用例(10%左右)

选择合适的软件架构

- 例如编译软件适合的架构为管道-过滤器风格

如果需要构造原型，可以进行设计和实现

2.5.2 Depict of UP Phases(2)

到了细化阶段的迭代，需要:

识别大部分用例？（至少 80%）

通过多次迭代，每次选择当前最关注的用例进行体系结构设计，细化为对应的组件模型、实现模型、部署模型等。

核心是需求、分析和设计。

实现重要的部分用例，验证体系结构的稳定性。

2.5.3 Depict of UP Phases(3)

进入构建阶段后，核心是所有功能的实现、测试，为应用部署做好准备
最后到了交付阶段，核心是部署、系统上线、运行和维护。

2.5.4 UP 裁剪

UP 是一个通用的过程模板，非常庞大。对不同的开发机构和项目，使用 UP 时需要裁剪，如：

确定本项目需要哪些工作流。RUP 的 9 个核心工作流并不总是需要的，可以取舍。

确定每个工作流需要哪些制品。

确定每个阶段内的迭代计划。规划 RUP 的 4 个阶段中每次迭代开发的内容。

2.6 课堂练习

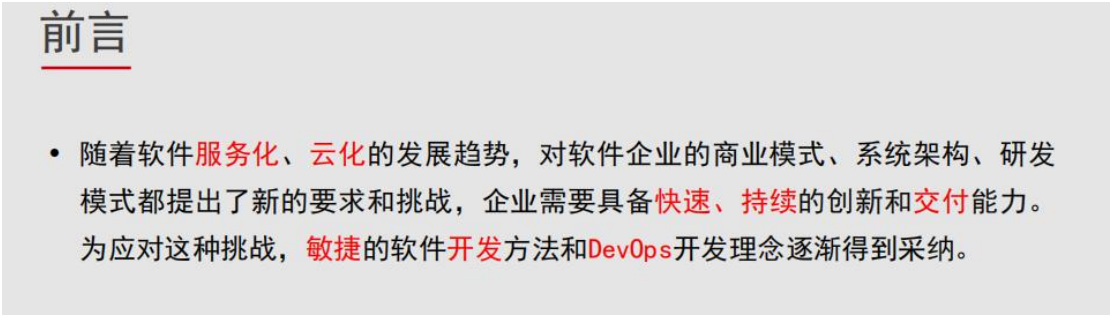
某公司开发一个通用 CAD 软件产品,产品需求是逐步完善的,某些需求在一定范围内是明确的,某些需求需要进一步细化,但是迫于市场竞争的压力产品需要尽快上市，

问：应该采用什么过程模型合适？

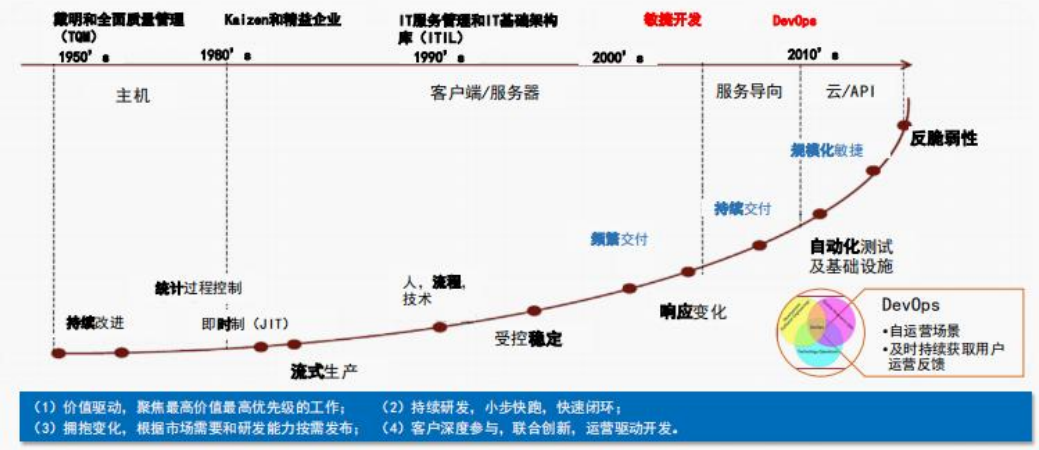
项目举例:某公司需要给火车站开发一个交互式火车车次查询系统,这是车站首次使用该系统，因此需求不是很明确。

问：应该采用什么过程模型合适？

Chapter3 华为增量开发



软件研发模式不断创新



5 HUAWEI

传统重型软件开发方法的不足

- 遵循严格的过程和计划
  - 倾向于预先制定详细的计划，并期望软件开发过程与计划之间的偏差越少越好
- 以文档为中心
  - 通过文档记录各个阶段的成果，将文档作为交互的媒介
  - 难以有效应对软件需求的变化
- 等到开发后期才能得到可运行软件

什么是敏捷（Agile）方法

一种轻量级软件开发方法

强化可运行的软件，弱化文档

以适应变化为目的来推进开发

持续、及时交付可运行的软件系统

敏捷价值观 – 敏捷软件开发宣言 (1)

敏捷软件开发宣言

我们一直在实践中探寻更好的软件开发方法，身体力行的同时也帮助他人。
由此我们建立了如下**价值观**

个体和互动 高于 流程和工具
工作的软件 高于 详尽的文档
客户合作 高于 合同谈判
响应变化 高于 遵循计划

也就是说，尽管右项有其价值，
我们更重视左项的价值。

9 HUAWEI

敏捷价值观 – 敏捷软件开发宣言（2）

敏捷宣言遵循的原则

1，**我们最重要的目标，是通过持续不断地及早交付有价值的软件使客户满意。**【持续交付】

2，**欣然面对需求变化，即使在开发后期也一样。【拥抱变化】**

3，**经常地交付可工作的软件，相隔几星期或一两个月，倾向于采取较短的交付周期。【小步前进】**

4，业务人员和开发人员必须相互合作，项目中的每一天都不例外。【紧密合作】

5，激发个体的斗志，以他们为核心搭建项目。提供所需的环境和支援，辅以信任，从而达成目标。【以人为本】

6，**不论团队内外，传递信息效果最好效率也最高的方式是面对面的交谈。【面对面沟通】**

7，**可工作的软件是进度的首要度量标准。**【尽早交付】

8，敏捷过程倡导可持续开发。责任人、开发人员和用户要能够共同维持其步调稳定延续。【稳步可持续】

9，坚持不懈地追求技术卓越和良好设计，敏捷能力由此增强。【追求卓越】

10，**以简洁为本，它是极力减少不必要工作量的艺术。【大道至简】**

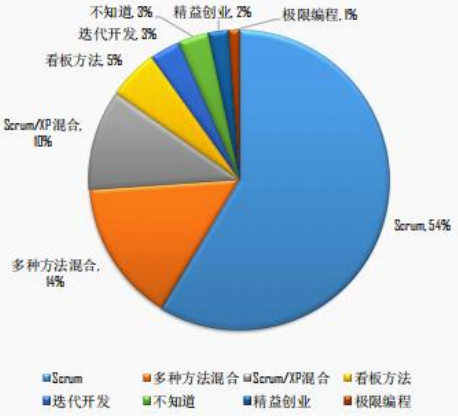
11，最好的架构、需求和设计出自自组织团队。【自组织团队】

12，团队定期地反思如何能提高成效，并依此调整自身的举止表现。【定期反思】

10

HUAWEI

敏捷常用的工程方法



根据2019年的第13届VersionOne版年度敏捷行业状态报告（如左图），以Scrum为基础的方法论（包括**Scrum、Scrum/XP混合**等）体系仍然居于主流地位，使用率最高。其他还有：**看板方法、精益创业、极限编程**等。

全面视角的Scrum框架

- Scrum是一个轻量级的软件开发框架，它的核心在于迭代。



首先要有产品待办列表，在计划会议上，团队从产品待办列表当中，选择合适的条目进入到迭代的待办列表，然后团队开始2~4周的迭代开发，在迭代开发的每一天团队都会进行每日站会，迭代结束后会提交一个潜在的可交付的增量，给用户评审。

13

HUAWEI







Scrum 三种工件

产品 Backlog

PO 负责整理 user story，形成 product backlog。

用户故事（user story）是一个用来确认用户需求的简短描述，作为什么用户，希望如何，这样做的目的或者价值何在。

例如：

1、作为管理者，我希望可以在时间视图设置任务的时间，这样我就可以方便的安排计划了；

2、作为用户，我希望可以方便的在子任务和任务之间切换，这样就能避免层级过深带来的困
惑；

3、作为用户，我希望在时间视图也可以创建任务，这样我就可以更方便的做计划了；

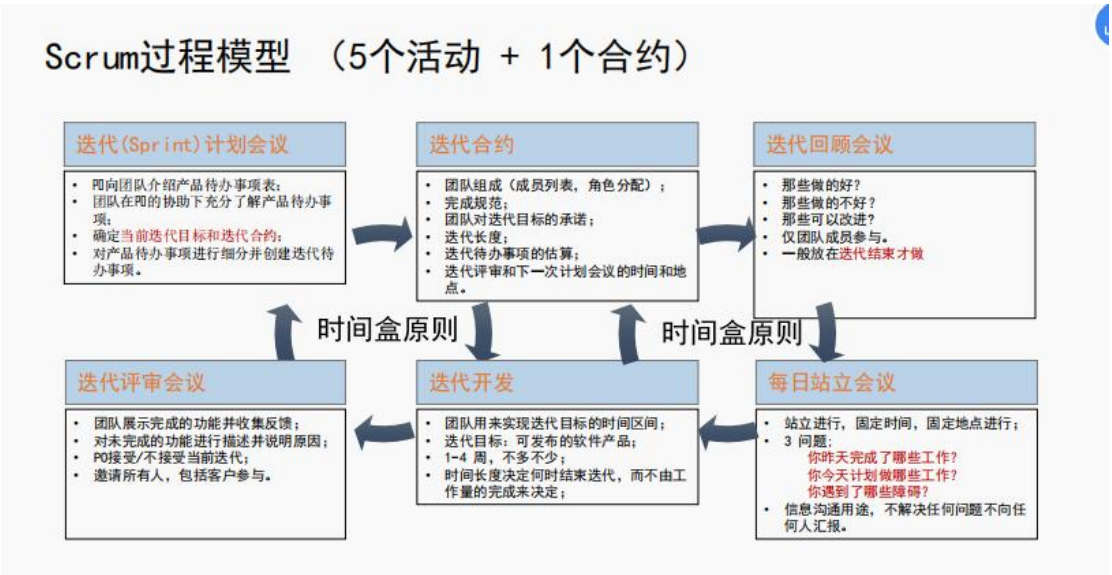
4、作为一个病人，我可以预约一个医生，让他给我看病。

Scrum 三种工件

Sprint Backlog

Sprint Backlog 确定这一期迭代要完成的 story 列表；

对每一个 story 进行任务分解，分解的标准是完成该 story 的所有任务，每个任务都有明确的
负责人，并完成工时的初估计。

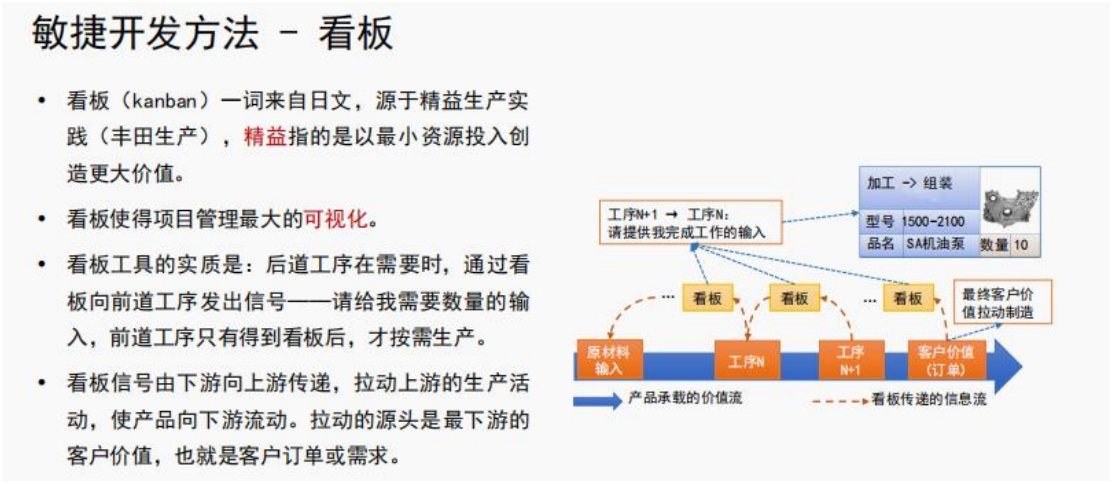


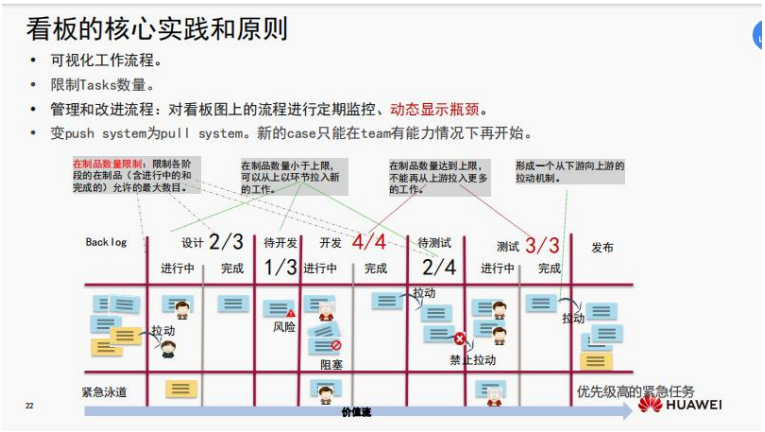
Scrum 对比增量开发模型　　共同点：　增量

最大区别：

Scrum 的每个迭代周期更短，交付更频繁；增量模型的每个增量开发周期更长；

Scrum 中每个迭代均允许提出新的需求；增量迭代中不允许有变化的需求，需要项目最初就做好估算，不能轻易更改。





敏捷开发带来的改变

• 一个软件从零开始到最终交付，大概包括以下几个阶段：规划、编码、构建、测试、发布、部署和维护。

• 敏捷开发对上述过程的冲击

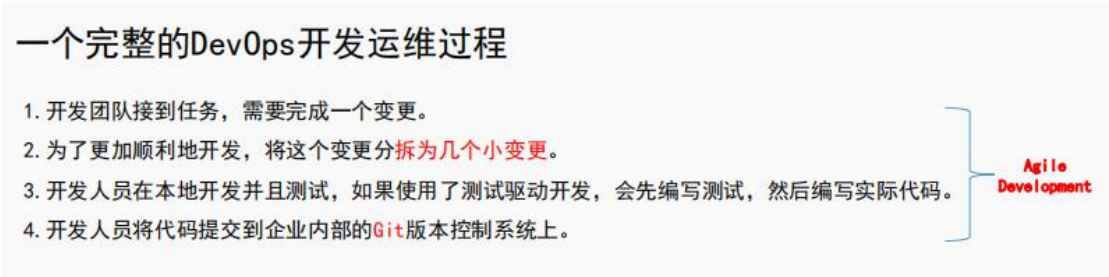
敏捷开发提升了软件开发的效率和版本更新的速度，但效果仅限于开发环节，运维成为了新的瓶颈。

运维团队的座右铭，是“稳定压倒一切”，因此排斥改变。

DevOps 覆盖端到端交付周期

• **持续集成**强调开发人员提交了新代码之后，自动化编译程序立刻进行构建、单元测试。根据测试结果，我们可以确定新代码和原有代码能否正确地集成在一起。

• **持续交付**是持续集成的延伸，将集成后的代码部署到类生产环境，确保可以以可持续的方式快速向客户发布新的更改。如果代码没有问题，可以继续手动部署到生产环境中。



Chapter 5 理解需求

5.1 需求的重要性

开发计算机软件时，如果软件解决的问题不对，那么再精巧的软件也满足不了任何人的要求。

理解问题的需求是软件工程师所面对的最困难的任务之一。

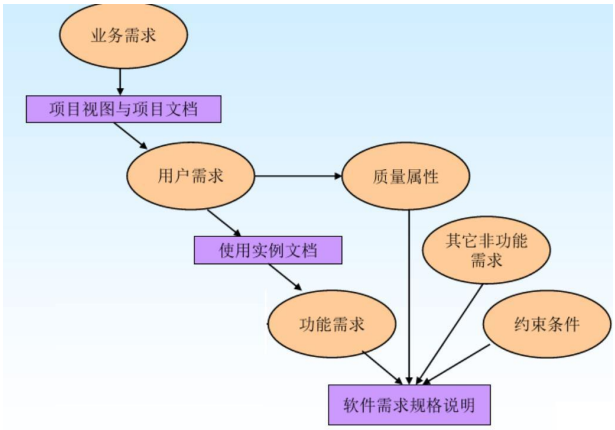
困难的原因有二：

客户不知道或无法全面的阐述需求； 客户需求可能在项目实施过程中改变；

导致项目失败的原因包括：

(1) 13.1% 不完整的产品要求； (2) 12.4% 缺乏用户的参与；
(3) 10.6% 缺少资源（人、财） (4) 8.7% 产品要求与指标的改变；
(5) 8.1% 没有订计划； (6) 9.9% 不现实的期望 (7) 9.3% 高层领导支持不足
(8) 7.5% 不再需要该系统

5.2 什么是需求



软件需求是从软件外部可见的，软件所具有的、满足用户特点的功能及其它属性的集合。

软件需求通常表达“做什么”，而不是“如何做”。

软件需求是多层次的。

业务需求：表示组织或客户高层次的目标。

用户需求：站在用户角度，描述用户的目标，或用户要求系统完成的任务。

功能需求：站在产品角度，描述产品中必须实现的功能，用户利用这些功能来达成自己的目标。

举例：家庭安保系统中

用户需求：我想安保系统帮我自动监测“可能的非法入侵”。

功能需求:

(1) 利用传感器探测非法入侵；

(2) 利用摄像头探测非法入侵；

（3）针对非法入侵自动报警；

非功能性需求：

(1）系统误报率<5%；

(2) 可灵活增加摄像头；

5.3 需求类别和来源

主要的需求来源：项目干系人(也称利益相关者)**除功能需求外，软件需求还包括各种非功能性需求：**

性能要求

运行环境要求

可靠性要求

安全保密要求

用户界面要求

可移植性需求

可复用性需求

外部接口要求

资源使用要求

其它约束和要求，如质量标准、可维护性要求等。



5.4 需求实例

需求描述应该满足：清晰、准确、一致且无二义性

性能需求

要求系统能满足 100 个人同时使用，页面响应时间不超过 6 秒；计费话单成功率达到 99.999%以上.定位精度误差不超过 80 米。

安全保密需求

提供运行日志管理及安全审计功能，可追踪系统的历史使用情况。
业务数据需要在存储时进行加密。

可靠性需求

可靠性需求反映了系统在一定条件下无故障地运行的能力。

- 缺陷率每 1,000 小时最多发生 1 次故障。
- 全年持续运行故障停运时间累计不能超过 10 小时。

兼容性需求

系统应支持 IOS，Android , Windows 操作系统；最多只有 5%的系统实现需要具体到特定的操作系统;

5.5 需求过程

整个需求过程包括下列七项任务

Inception(起始)—ask a set of questions that establish basic understanding of the problem

Elicitation(抽取)—elicit requirements from all stakeholders

Elaboration(分析建模)—create an analysis model

Negotiation(协商)—agree on a deliverable system

Specification(规格说明)

Validation(需求确认)—a review mechanism

Requirements management（需求管理）

5.5.1 Inception

Establish a basic understanding of the problem
收集客户的资料，了解系统的背景，进行初步的可行性分析，确定项目范围。

Identify stakeholders 识别干系人

识别可能对项目及其可交付成果施加影响的人。

360 儿童手表最初只把家长定位为用户，只有定位功能，因此销售不好。

后来，意识到孩子才是真正的用户，因此加入打电话，添加好友等功能。

Recognize multiple points of view

Work toward collaboration

identify areas of commonality and areas of conflict

5.5.2 获取需求

需求获取的主要方式

需求讨论会　头脑风暴法　调查问卷　场景分析法

角色扮演考察　原型法

Eliciting Requirements(1)

(1) Meetings(需求会议)

软件工程师和项目干系人共同参与

拟定会议议程 ： 确定会议计划　　准备调研提纲

指定“调解人”控制会议

- 可以是客户或开发人员
- 可由客户方的情况介绍开始，分析人员逐渐成为主导，引导和把握调研的重点和方向。
- 从业务流程入手，先关注用户需求；再由用户需求导出功能需求（核心是 IPO）。

(1) Meetings(需求会议)

如果满足用户需求有多种备选方案，可以协商不同的方法，提出一套解决方案的要素。

例如：高速不停车收费系统、车库不停车收费系统

(2) 头脑风暴

通过群组效应相互启发，激发对新产品或系统新的想法。

该方法对需求不确定的情况非常适用。

(3) 调查问卷

由分析人员拟定问卷（以选择、判断为主）

适合于需求调研的后期，用于对前期发现的一些不确定或不一致的地方进行确认。

例如：前期，用户提出了高安全性要求，但是基于硬件（usb key)还是软件实现方式，需要用户明确。

(4) 场景分析法

场景分析可以还原需求产生的过程，由此判断其是不是一个真实的需求。

场景分析的核心是 3 个维度的梳理：角色、场景、方案。

需求场景分析表		
角色	场景（时间、地点、动机）	方案（途径+流程）
A类用户	情况1，要做事情1（而产生了该需求）	方案1：方式&流程1 方案2：方式&流程2
	情况2，要做事情2（而产生了该需求）	方案1：方式&流程1 方案2：方式&流程2

角色	场景	方案	用户占比
订票用户	购买火车票（目的地车票充足）	正常订票-支付	70%
	购买火车票（没有直达车票，但有中转到达方案）	推荐中转方案-按中转方案生成多订单-多订单支付	20%
	购买火车票（没有任何车票）	推荐旅店-生成第二天订单-支付	10%

(5) 角色扮演

如果不能实际约谈客户，怎么获取需求？可以考虑角色扮演，把自己想象成各类用户。

如：以蚂蜂窝为例，用户角色可能包括

分享类用户，爱旅游爱分享，喜欢分享各种旅行攻略；

浏览类用户，看旅行攻略和他人游记为主；

旅行赚钱类，如背包客；

软文推广类，旅行社/公司职员，旅行编辑，写旅行文案推广；

组队约伴型，组队旅行，顺便预定一个酒店。

(6) 原型法

没看到实际系统前，用户往往自己都不知道需要什么。

界面原型能将系统功能、数据、UI 等多种需求可视化。

高保真方式

低保真方式

5.5.3 My Experience(1)

(1) 调研前做好必要的相关背景知识准备

(2) 通过用户获取重要的原始资料（如表单，文件)

(3) 渐进、反复的进行

每次调研后，及时对需求整理、分类，以便由用户确认需求；

对需求整理时新发现的问题，列入下次调研计划。

(4) 根据需求的重要性划优先等级

5.6 构建分析模型

需求建模的必要性

更加准确、规范地描述需求

更加方便与用户以及开发人员的交流

利用不同模型建模软件需求的不同侧面

Elements of the analysis model

Scenario-based elements 关注功能

Class-based elements 关注业务对象

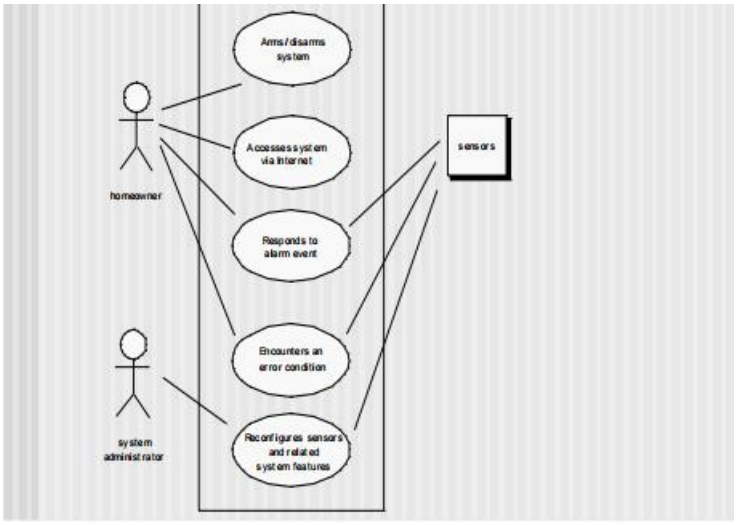
Behavioral elements 关注人机交互

Flow-oriented elements 关注业务流程

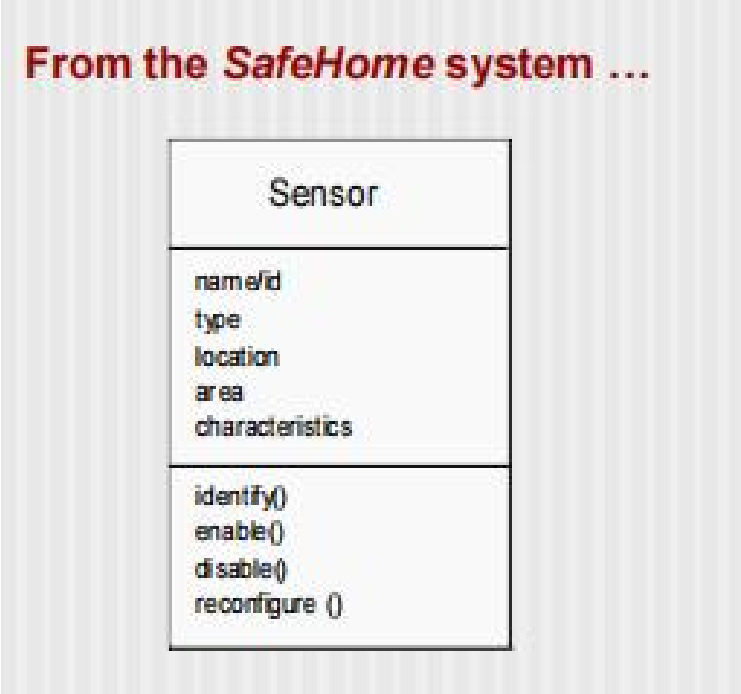
E-R model 关注数据

需求建模方法：面向对象方法和结构化方法

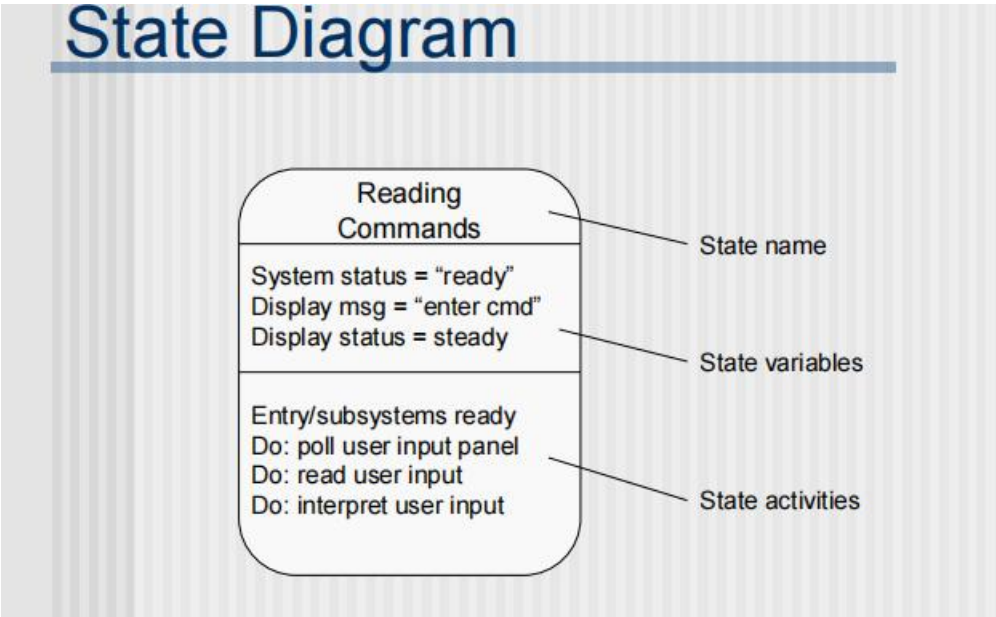
5.6.1 用例图



5.6.2 类图



5.6.3 状态图



5.7 需求沟通

Identify the key stakeholders

These are the people who will be involved in the negotiation

Determine each of the stakeholders “win conditions”

Negotiate

Work toward a set of requirements that lead to “win-win”

5.8 规格说明

规格说明可以是一份写好的文档、一套图形化的模型、一组使用场景、一个原型或上述各项的任意组合。

对大型系统而言，规范、标准的文档和图形化模型是最佳选择;

对于技术明确的小型系统而言，使用场景可能就足够了。

SRS(Software Requirement Specification)的作用

1）开发者与用户间事实上的技术合同；

2）下一步设计和编码的基础；

3）测试验收目标系统的依据；

SRS 主要内容

1 引言

编写目的、项目背景、定义、专门术语、缩写词、参考资料等。

2 任务概述

目标、运行环境、产品功能、用户特点、一般约束、假设和依据。

3 数据描述/数据要求

数据词典、静态数据、动态数据（包括输入数据和输出数据）、内部生成数据、数据约定、数据采集。

4 功能需求

功能划分、功能描述、输入、加工/处理、输出。

5 性能需求

数据精确度、时间特性（如响应时间、更新处理时间、数据转换与传输时间等）等。

6 运行需求/外部接口需求

用户界面（如屏幕格式、报表格式、菜单格式、输入输出的相对时间、功能键的可用性等）、硬件接口、软件接口、通信接口、故障处理。

7 设计约束

其他标准的约束、硬件的限制等。

8 其他需求

如数据库、适应性、可使用性、安全保密、可维护性、可移植性等。

5.9 Validating Requirements - I

需求文档质量如何？

参与者

需求分析员、开发人员、测试人员、客户代表、项目经理、产品经理……

需求评审要点

Have all requirements been specified at the proper level of abstraction?

Is the requirement really necessary to the objective of the system?

Is each requirement bounded and unambiguous?

-系统能提供适当的文档浏览器供用户在线阅读各类文档。

5.10 Validating Requirements - II

Does each requirement have attribution(归属)?

Do any requirements conflict with other requirements?

-如用户隐私保护需求与产品精准推荐需求。

Is each requirement achievable(可实现的) in the technical environment?

Is each requirement testable, once implemented?

-如果需求是“抗十二级台风”，如何验证？造个十二级台风来试验？计算机模拟台风？

Chapter 6 Requirements Modeling: Scenarios, Information, and Analysis Classes

6.1 Requirements Analysis

需求分析的本质是：通过建立概念化的分析模型，对收集的需求进行提炼和审查。

需求分析模型包括:

Scenario-based models of requirements from the point of view of various system “actors”

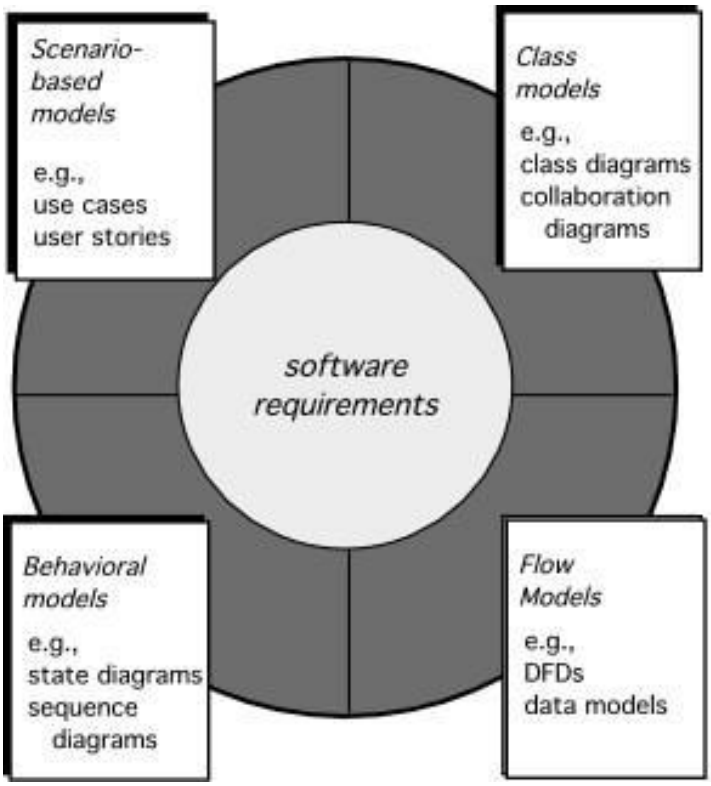
Flow-oriented models that represent the functional elements of the system and how they transform data as it moves through the system

Data models that depict the information domain for the problem

Class-oriented models that represent object-oriented classes and the manner in which classes collaborate to achieve system requirements

Behavioral models that depict how the software behaves as a consequence of external “events”

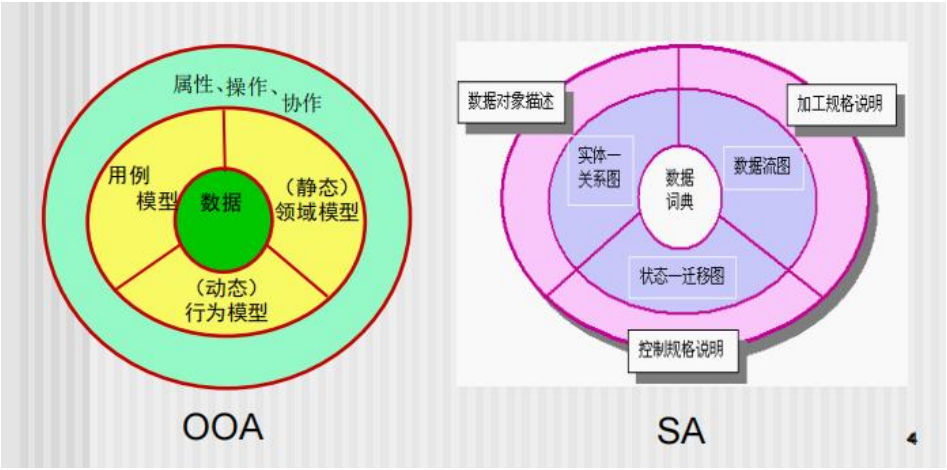
6.2 Elements of Requirements Analysis



6.3 需求分析方法

Structured analysis (SA) 结构化分析方法

Object –oriented analysis (OOA) 面向对象分析方法



6.3.1 OOAOOA & UML

现实世界由实体及其关系构成，可将其理解为问题空间（现实世界）中的对象。

OOA 方法通过将其映射为解空间（软件系统）中的对象及其关系来寻求问题的求解。

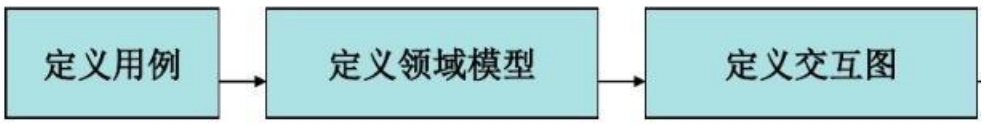
一般使用 UML（Unified Modeling Language）完成需求分析模型的定义。

UML 是一种标准图形表示法。

6.3.2 Object Oriented Analysis

OOA 分析模型包括三类：用例模型、（静态）领域模型和（动态）行为模型。

OOA 分析过程



第一步：建立用例模型

从不同用户的角度获取并描述功能需求，核心是用例。

第二步：建立领域模型

发现和描述用例中涉及的对象和概念，核心是类图。

第三步：建立动态(交互)模型

分析用例实现中，对象之间的交互行为，核心是时序图、状态图等

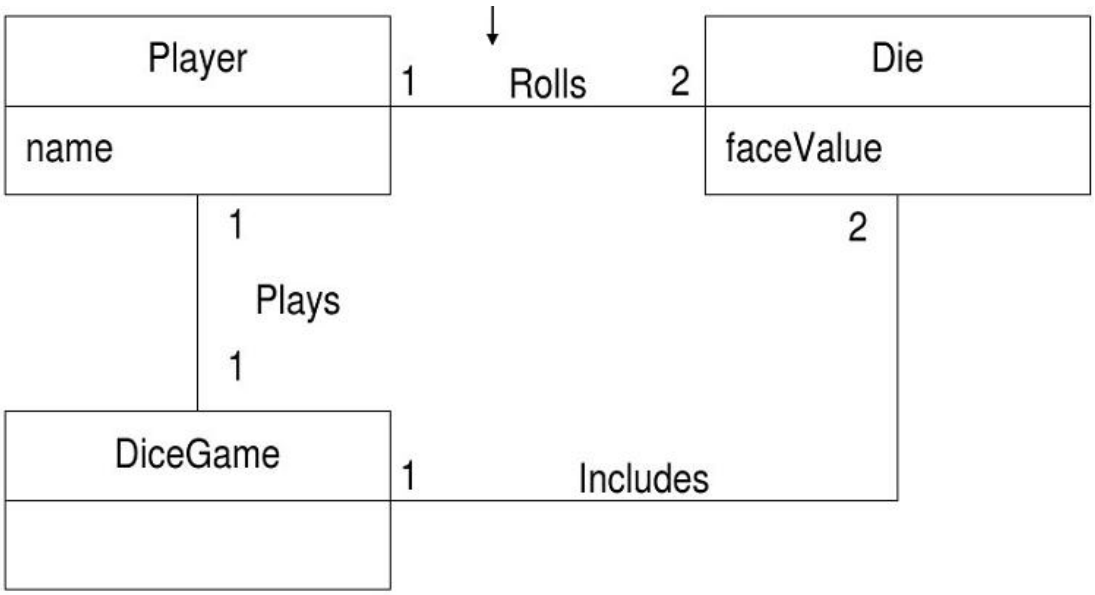
OOA 分析实例(1)

如开发一小游戏：游戏者掷两个骰子，如果总点数为 7，则游戏玩家赢得比赛，否则输掉比赛。

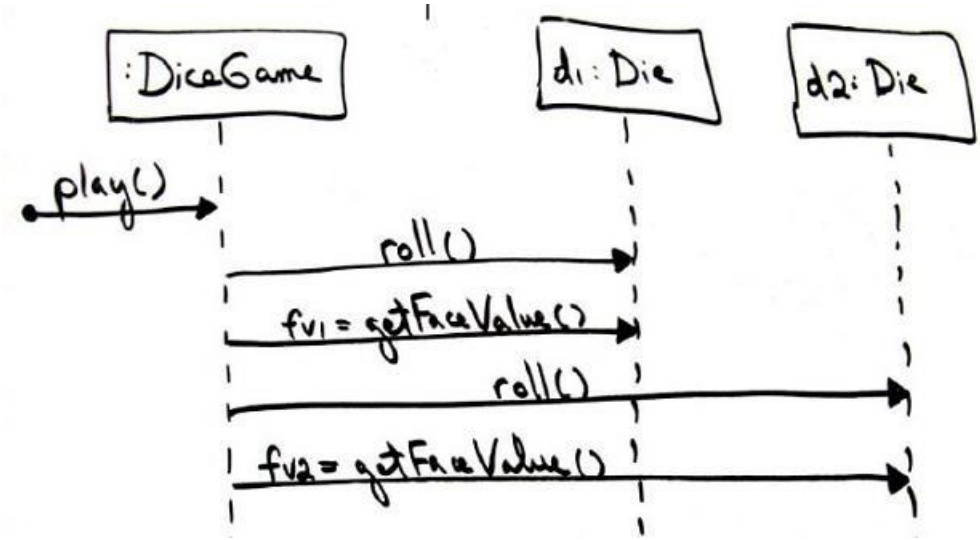
步骤 1：用例及用例描述

用例: Play Game（进行游戏）
参与者：Player （游戏者）
描述：
游戏者请求掷骰子；
系统展示骰子结果；
判断胜负：如果骰子总点数是7，则游戏者赢，否则游戏者输。

步骤 2：根据用例描述建立领域模型
-有几个类？分别是什么？



步骤 3：根据用例描述建立交互模型
-根据交互过程，找出参与对象在该用例中的职责或行为



6.3.4 OOA 分析第一步：用例建模

用例表示从执行者的角度观察到系统的功能和外部行为。

用例图主要描述用户需求，强调谁在使用系统，系统可以实现哪些功能目标。

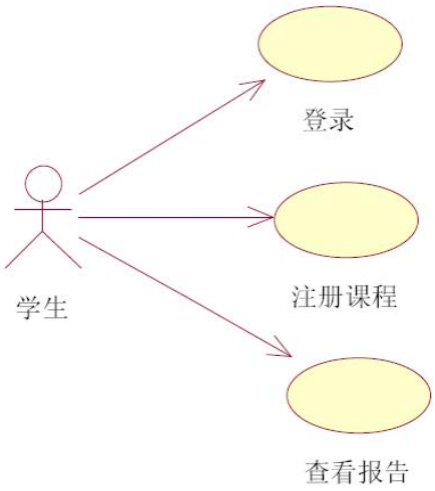
思考：ATM 存钱是用例，用户插卡，点钞是用例吗？

用例特征

用例都是动宾结构

用例由执行者启动

有可观测的执行结果



识别用例方法

识别用例最好的方法就是从分析系统的执行者（可以是角色、外部系统、设备甚至时钟信号等）开始，考虑每一个执行者是如何使用系统的。

用例分析练习

需求陈述：

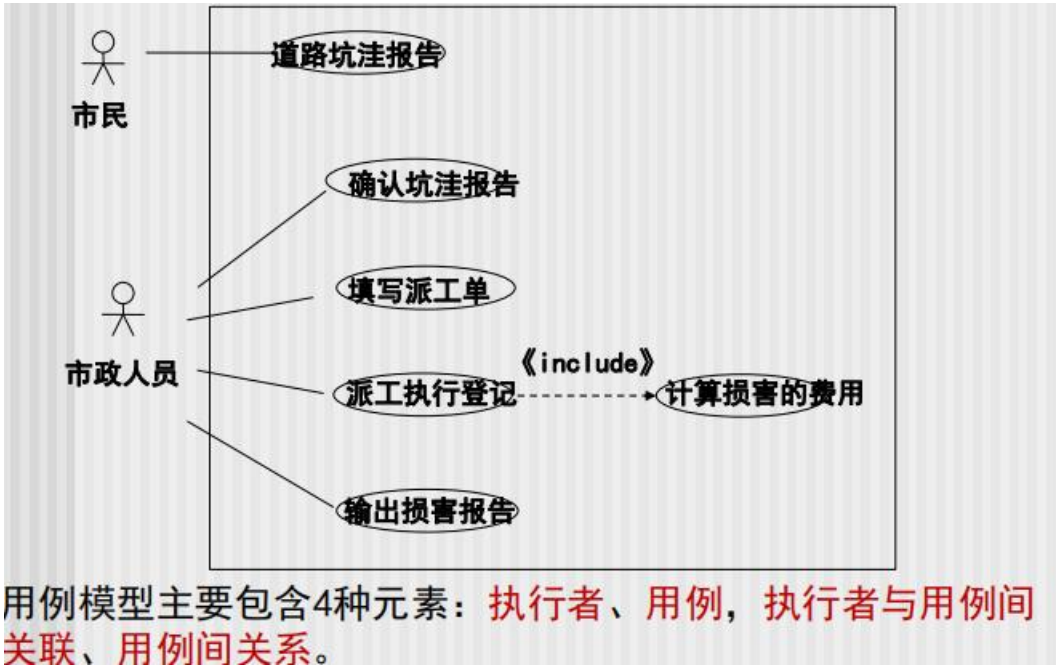
市民可以登录网站报告坑洼的位置和严重程度。每个被报告的坑洼，将登记到“市政部门的修复系统”中，并被赋予一个标识号，按街道地址、大小、位置、城区和修复的优先级（坑洼严重程度）存储。

市政人员确认坑洼情况属实后，填写派工单并对派工执行情况进行登记。内容包括：位置和大小、修理队的人数、使用的修理装备、修复所用的时间、坑洼状况（正在工作、已被修理、临时修理、未修理）、使用填料的数量和修理的开销（由使用的时间、人数、使用的材料的装备计算得到）。

坑洼经修复后，可输出坑洼的损害报告，包括市民的姓名、地址、电话号码、损害类型和修复总费用。

此问题中，系统外部执行者包括：

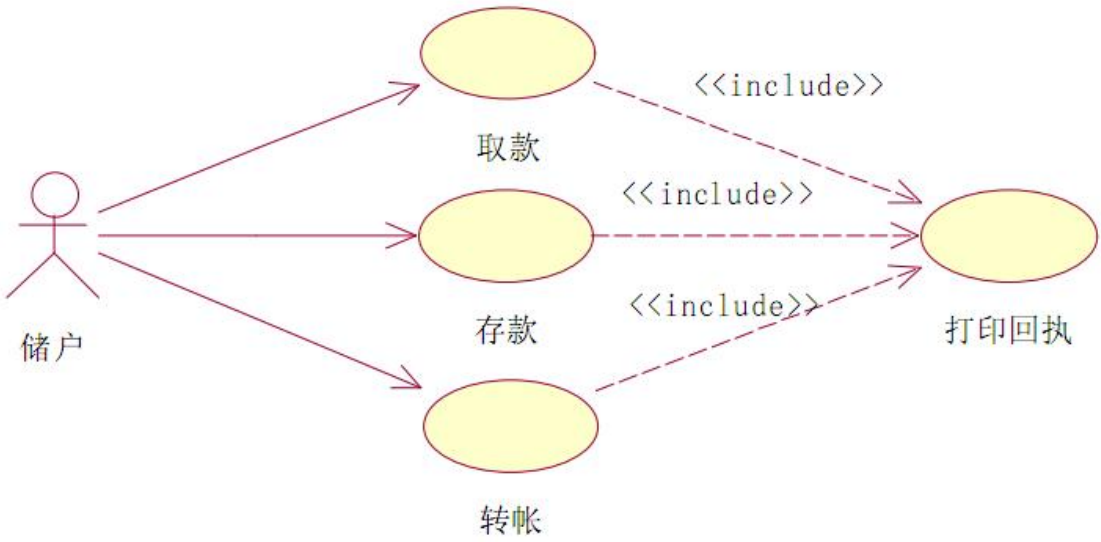
市民---------- 登记报告坑洼
市政工作人员------确认坑洼报告
填写派工单
派工执行登记
输出损害报告



用例模型主要包含4种元素：执行者、用例，执行者与用例间关联、用例间关系。用

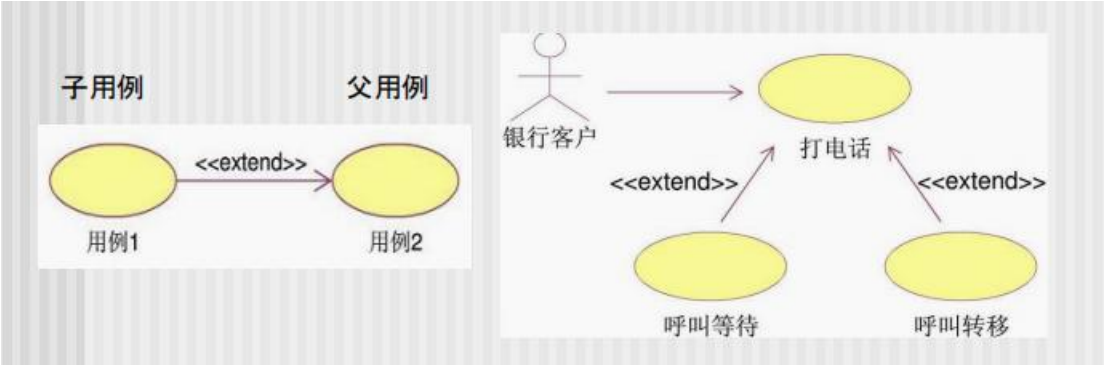
用例图(1)《Include》关系

父用例执行过程中会用到子用例(无条件执行)
子用例一般表示为公共功能。



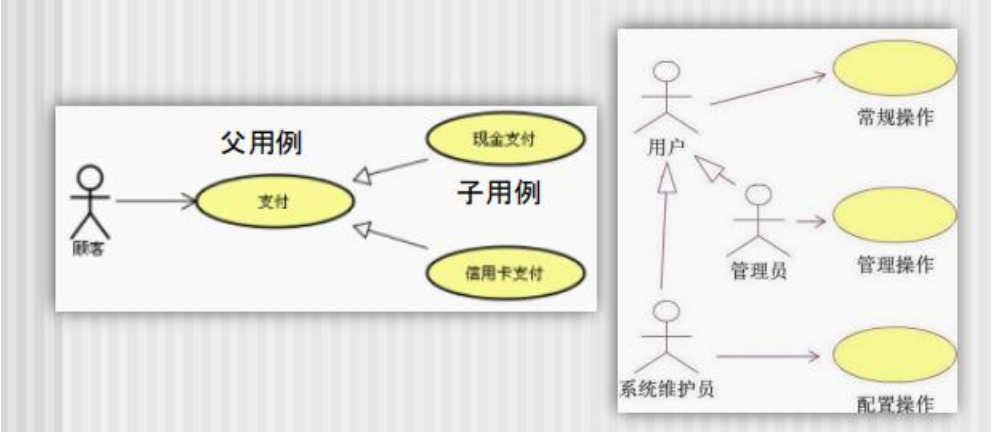
用例图(2) 《extend》关系

父用例在特定情况下（称为扩展点）会用到子用例(有条件执行)
子用例一般表示为异常功能。



用例图(3) 泛化(generalization)关系

当多个用例共同拥有一种类似的行为时，可以将它们的共性抽象成为父用例，其他用例作为子用例。
该关系也可存在于参与者之间。



用例描述

单纯的用例图并不能描述完整的信息，需要用文字描述不能反映在图形上的信息。

用例描述是将用例发生的各种场景描述出来，表示参与者与系统交互时双方的行为，即参与者做什么，系统做什么反应。

场景能真实反映参与者在用例执行中可能遇到的各种情形。

可以使用用例模版或活动图描述用例执行中的各种场景。

用例描述—基于模版

用例描述模版的内容大致包括：用例名、参与者，前置条件，后置条件，事件流等。

用例的事件流

说明参与者与系统之间的交互过程；

说明用例在不同条件下可以选择执行的多种方案；

分为基本流和备选流两类

基本流：描述该用例正常执行的一种场景，系统执行一系列活动步骤来响应参与者提出的服务请求；

备选流：描述用例执行过程中异常的或偶尔发生的一些情况。

用例名称：	ATM存款
描述：	客户通过ATM向银行存入现金
actors：	客户和银行主机
前置条件：	无
基本流：	1. 客户插入银行卡。 2. ATM从银行卡读入卡号，验证卡的有效性。 3. 客户输入密码。 4. ATM验证帐号和密码。 5. ATM显示包括存款在内的服务功能，客户选择“存款”。 6. 客户放入钞票。 7. ATM点钞并显示存入金额。 8. 客户确认金额无误后，ATM向银行主机发送账户余额更新请求，银行主机处理结束向ATM机返回存款成功信息。 9. ATM打印并吐出凭条。
备选流：	4a. 累计3次密码错误： ATM吞卡，[用例失败] 4b. 无此帐号： ATM吞卡，[用例失败] 7a. 钞票无法识别： ATM吐出钞票，客户可取回或重新放入。
扩展点：	无
非功能需求：	ATM响应客户时间不超过15秒

用例描述—基于模版

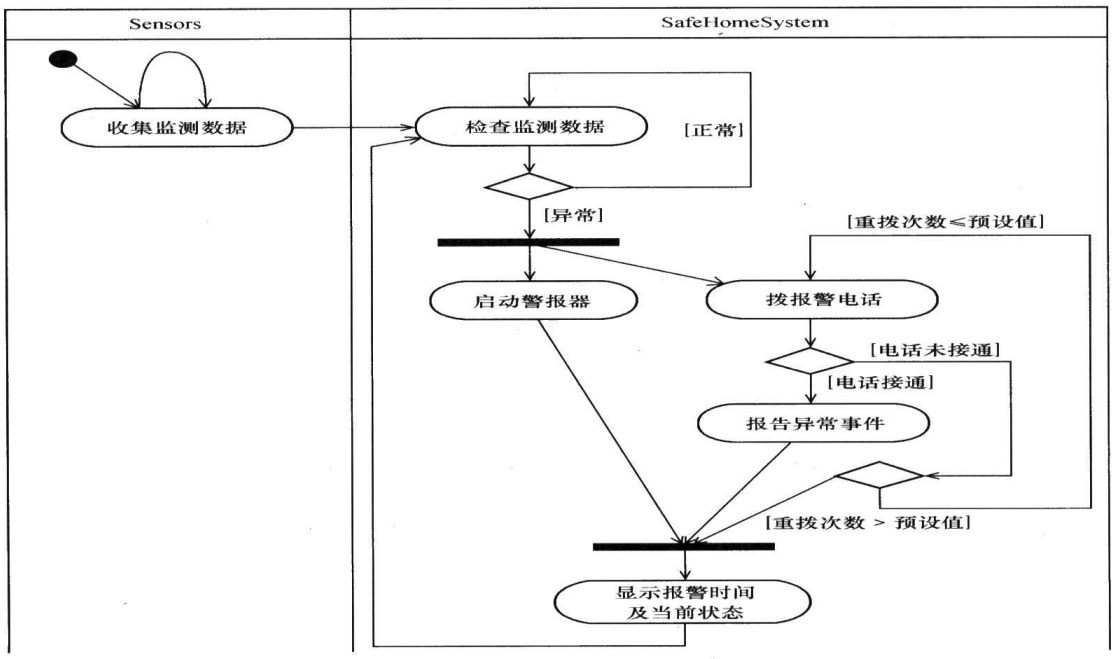


图 6.9 “传感器监测”用例的活动图表示

OOA 分析第二步：数据建模

关注系统中的数据对象及其关系。

Focuses attention on the data domain

Creates a model at the customer's level of abstraction

Indicates how data objects relate to one another

·数据对象

一种几乎所有必须被软件理解的复合信息的表示。

可以是外部实体、事物、事件、角色、组织单元、地点或结构。

数据对象的描述包含了数据对象及其所有属性。

数据对象和属性

数据对象包含一组起作用的属性作为一个方面、质量、特征、或对象的描述符

对象：汽车

属性　制造　模型　体型　　价格

关系

数据对象以不同的方式相互连接。

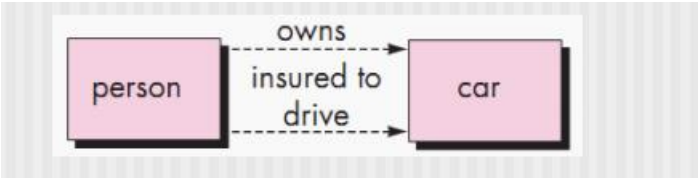
在人和车之间建立了一个连接，因为这两个对象是相关的。

一个人拥有一辆车

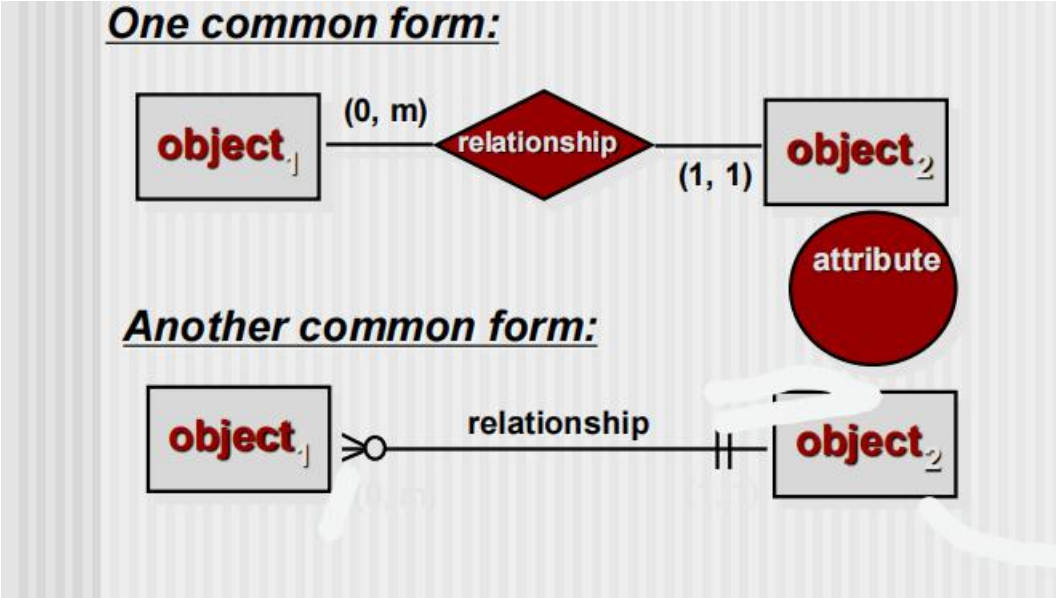
一个人要投保才能开车

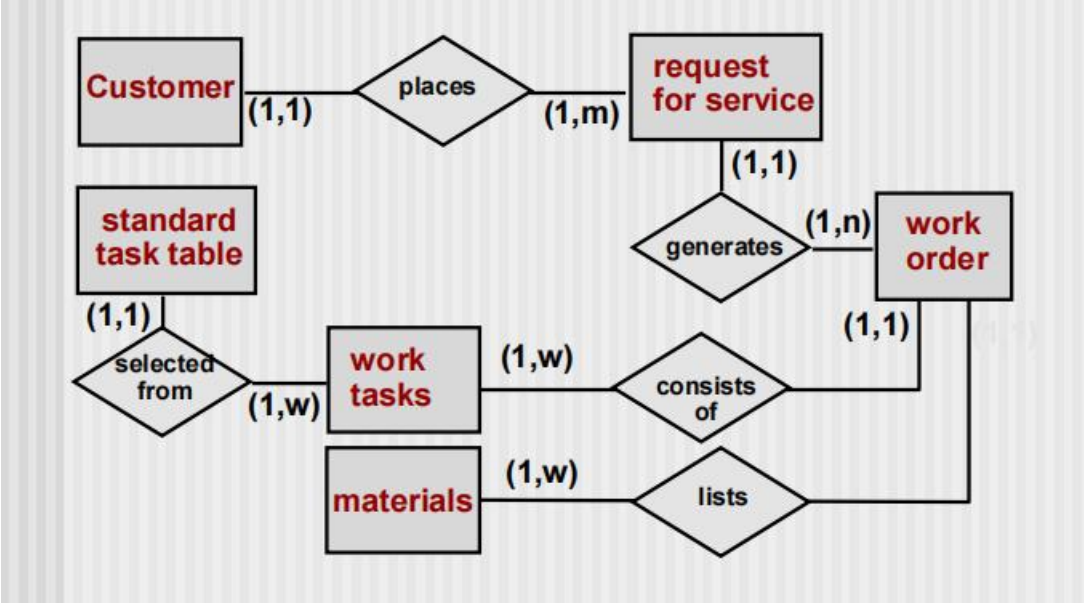
拥有和被保险人驾驶的关系定义了人与汽车之间的相关联系。

对象可以通过多种不同的方式进行关联



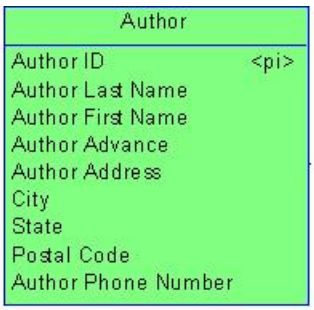
ERD Notation

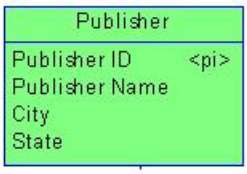


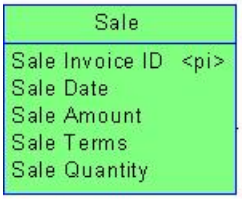


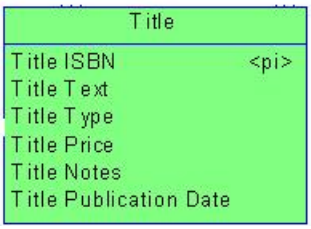
(

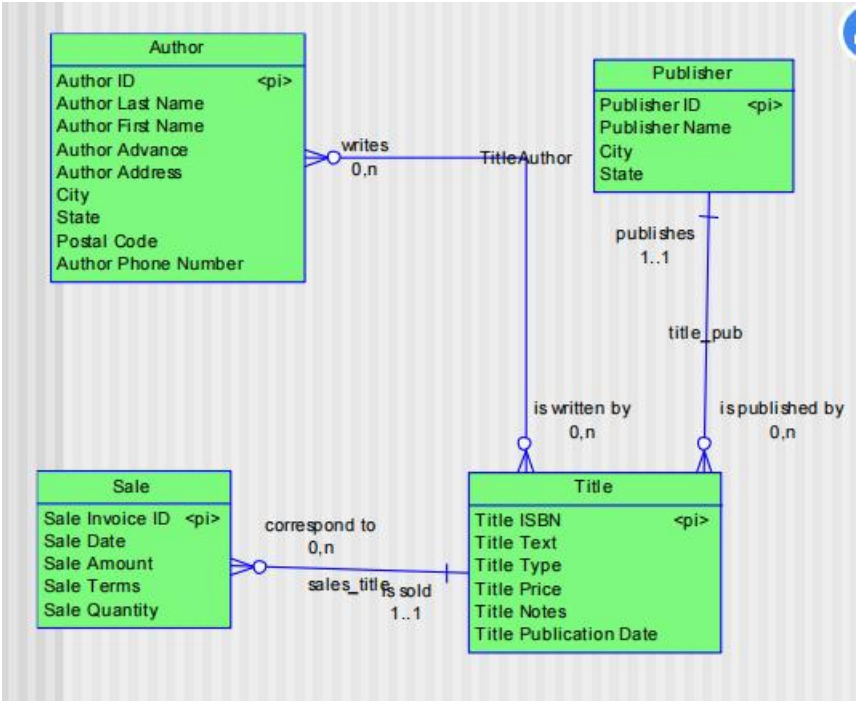
数据建模练习

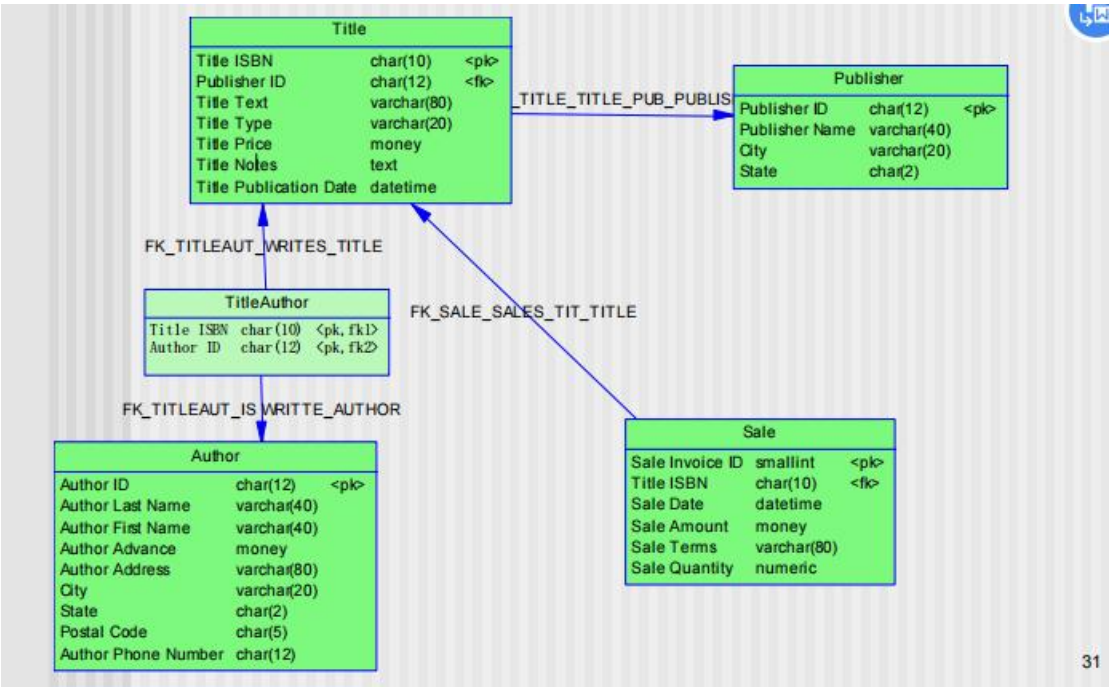

Author
Author ID <pi>
Author Last Name
Author First Name
Author Advance
Author Address
City
State
Postal Code
Author Phone Number

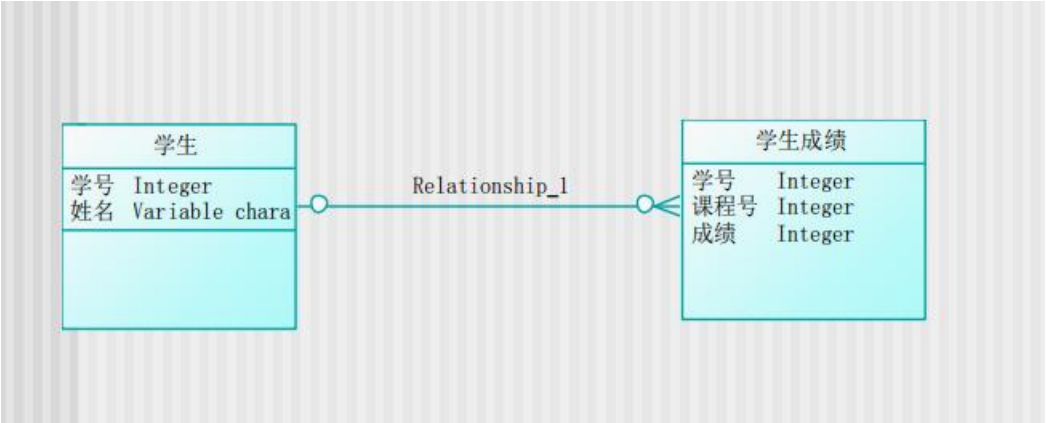

Publisher
Publisher ID <pi>
Publisher Name
City
State


Sale
Sale Invoice ID <pi>
Sale Date
Sale Amount
Sale Terms
Sale Quantity


Title
Title ISBN <pi>
Title Text
Title Type
Title Price
Title Notes
Title Publication Date


Author
Author ID <pi>
Author Last Name
Author First Name
Author Advance
Author Address
City
State
Postal Code
Author Phone Number
writes
0,n
TitleAuthor
Publisher
Publisher ID <pi>
Publisher Name
City
State
publishes
1..1
title_pub
is written by
0,n
is published by
0,n
Sale
Sale Invoice ID <pi>
Sale Date
Sale Amount
Sale Terms
Sale Quantity
correspond to
0,n
sales_title
is sold
1..1
Title
Title ISBN <pi>
Title Text
Title Type
Title Price
Title Notes
Title Publication Date





6.3.5 OOA 分析第三步：领域建模

领域建模以用例模型作为输入，将系统分解为相关联的一组概念类(也称分析类)，以描述系统的静态结构。

分析类具有突出业务领域、突出概念性的特征，主要针对问题和职责，不涉及细节。



例：回顾之前的“ATM 存款”用例，你认为 ATM 机可建模为哪些分析类？

ATM 界面类 读卡器类 点钞器类 打印机类

分析不是终点，到设计阶段，分析类可进一步演化为设计类，提供更多设计表达。

1）领域模型

包括

类图 (Classes diagram)
CRC 模型(CRC models)
包图(Packages)

2）类图建模

目标是对领域内的概念类进行分析，并抽象出对应的类及其关系等。
具体包括三个步骤：
1.从用例场景中寻找概念类。
2.细化概念类，识别其类型（如边界类、实体类等）。
3.为概念类添加关联和属性。



Identify Analysis Classes

Examining the usage scenarios developed as part of the requirements model and perform a “grammatical parse” 分析用例场景

Classes are determined by underlining each noun or noun phrase and entering it into a simple table. 找出其中的名词或名词短语作为分析类备选

But what should we look for once all of the nouns have been isolated?
如何判断分离出的词是否是分析类？
借助以下原则识别可能的分析类

- 保留信息：只有当相关信息必须被记录才能保证系统正常工作时，潜在类在分析过程中才是有用的。
- 所需服务：潜在类必须具有一种可确认的、能用某种方式改变类的属性值的操作。
- 多个属性：在需求分析过程中，焦点应在于“主”信息；事实上，只有一个属性的类可能在设计中有用，但是在分析活动阶段，把它作为另一个类的某个属性可能更好。
- 公共属性：可以为潜在类定义一组属性，这些属性适用于类的所有实例。
- 公共操作：可以为潜在类定义一组操作，这些操作适用于类的所有实例。
- 必要需求：在问题空间中出现的外部实体，或者任何系统解决方案的运行所必需的信息，几乎都被定义为需求模型中的类。

用例名称	学生选课(Select Course)
执行者	学生
用例简述	学生通过该用例选择感兴趣的课程。
前置条件	学生已通过身份验证。
后置条件	系统正确记录学生选课信息。
基本流程	1. 学生进入选课界面，用例开始。 2. 学生查询课程。 3. 系统显示可选课程。 4. 学生选中感兴趣的课程。 5. 系统验证课程是否可选，如果可选，则保存学生选课信息。 6. 系统显示“选课成功”。
备选流程	5a. 课程如不可选 1. 系统提示不可选的原因（如人数已满）。 2. 学生重新选课。 3. 系统再次验证直至成功。
字段列表	学生信息包括以下字段：学号、姓名、密码、专业、班级。 课程信息包括以下字段：课程号、课程名、开课时间、课程学时、选修人数。
非功能需求	系统响应时间应该在30秒以内。

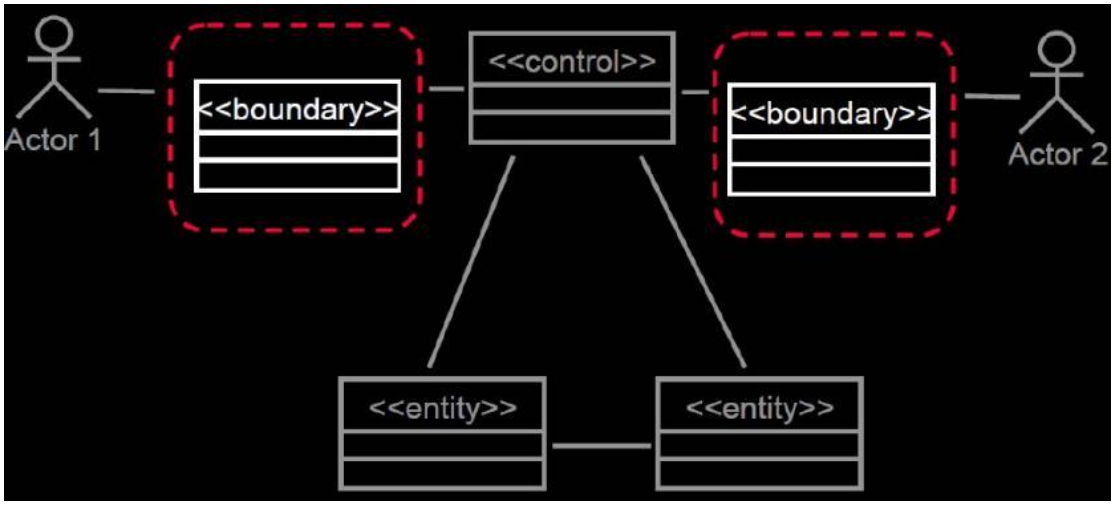
通过分析，可以确定以下分析类

类名	类名	类型
Student	学生	实体类
SelectCourseForm	选课界面	边界类
Course	课程	实体类
SelectSuccessForm	提示选课成功（选课成功界面）	边界类
SelectedCourse	选课信息	实体类

分析类是概念层次的东西，源于问题空间，与实现方式无关。

分析类类型

《Entity》 classes 表示系统中的持久信息或贯穿应用的数据封装
《Boundary》 classes 实现用户与系统的交互，如用户界面或外部接口
《Control》 classes
实现对用例行为的封装，管理、调度其它类



Define Attributes

Attributes describe a class that has been selected for inclusion in the analysis model.

Build two different classes for professional baseball players

For Playing Statistics software: name, position, batting average, fielding percentage（守备率）, years played, and games played might be relevant

For Pension Fund software（养老系统）: average salary, credit toward full vesting, pension plan options chosen, mailing address, and the like.

属性描述了一个已被选择以包含在分析模型中的类。

为职业棒球运动员建立了两个不同的职业

玩统计软件：名称，位置，打击平均，守野率（守备率），打球年份，和游戏可能是相关的

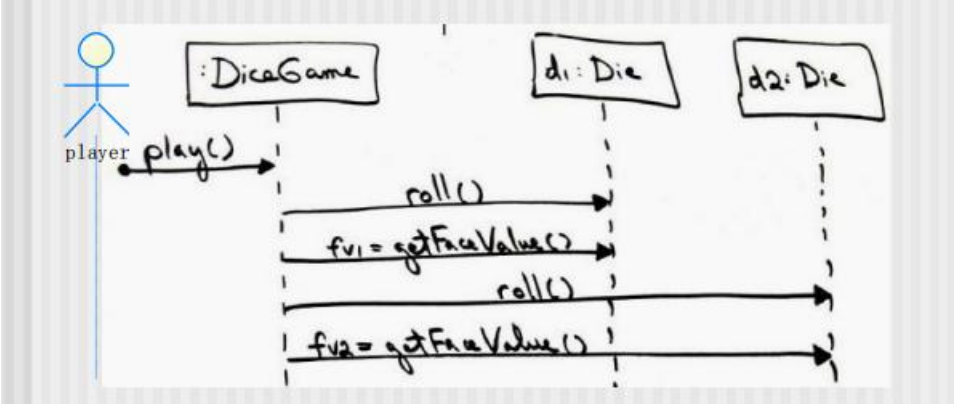
为养老基金软件（养老系统）：平均工资，全额发放的信贷，选择的养老金计划选项，邮寄地址等。

Define Operations

识别类所属行为

方法 1：Do a grammatical parse of a processing narrative and look at the verbs 遍历用例描述，寻找动词或动词短语。

方法 2：结合动态(交互)建模，识别分析类的行为。



Example-学生选课用例

Example-学生选课用例

基本流程	1. 学生进入选课界面，用例开始。 2. 学生查询课程。 3. 系统显示课程。 4. 学生选择课程。 5. 系统验证课程是否可选，如果可选，保存学生选课信息。 6. 系统显示选课成功。

类名	属性	操作
Student	- SID: String - SName: String - SPassword: String - SSpecialty: String	+ viewCourse(): void + selectCourse(): void
Course	- CID: String - CName: String - CStartDate: Date - CPeriod: int - CStudentAmount: int	+ validateCourse(Course c): boolean + displayCourse(Course[] c) :void
SelectedCourse		+ saveSelectedCourse(Course c): boolean

Define Relationships

4. Define Relationships

1. 关联关系

关联关系可以使一个类可以引用另一个类的属性和方法。
一般使用成员变量来实现。



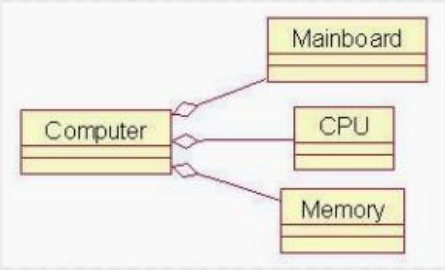

```
class  Department{
  private Employee e;
  //private Employee[] es;
  public getCost(){
     return e.getcost();
  }
}
```

```
class Employee{
   private name;
   private cost;
   public getCost(){
      return cost;
   }
}
```

2. 特殊关联之一聚合关系

表示整体和部分的关系，整体与部分可以分开，使用带空心菱形的实线来表示

如： 电脑包括主板、CPU等




```
public class Computer{
  private CPU cpu;
  public CPU getCPU(){
     return cpu;
  }
  public void setCPU(CPU cpu){
     this.cpu=cpu;
  }
  //开启电脑
  public void start(){
     //cpu运作
     cpu.run();
  }
```

46

3. 特殊关联之一组合关系

也是整体与部分的关系，但是整体与部分不可以分开，使用实心带菱形的实线来表示。



组合关系中整体类和部分类必须有相同的生命周期。




```
public class A {
  private B b;
  public A() {
    this.b = new B();
  }
}
```

47

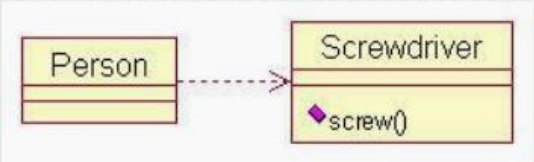
4. 依赖关系

即一个类的实现需要另一个类的协助。

多数情况下，依赖关系表现在某个类的方法使用另一个类的对象作为参数。

比如你要拧螺丝，要借助(也就是依赖)螺丝刀(Screwdriver)来完成

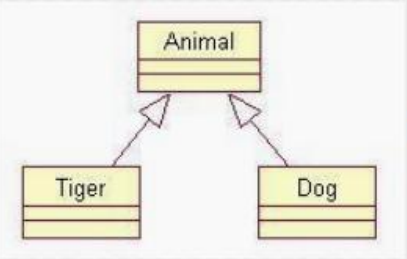
```
public class Person{
   public void screw(Screwdriver screwdriver){    /** 拧螺丝 */
     screwdriver.screw();
   }
}
```



48

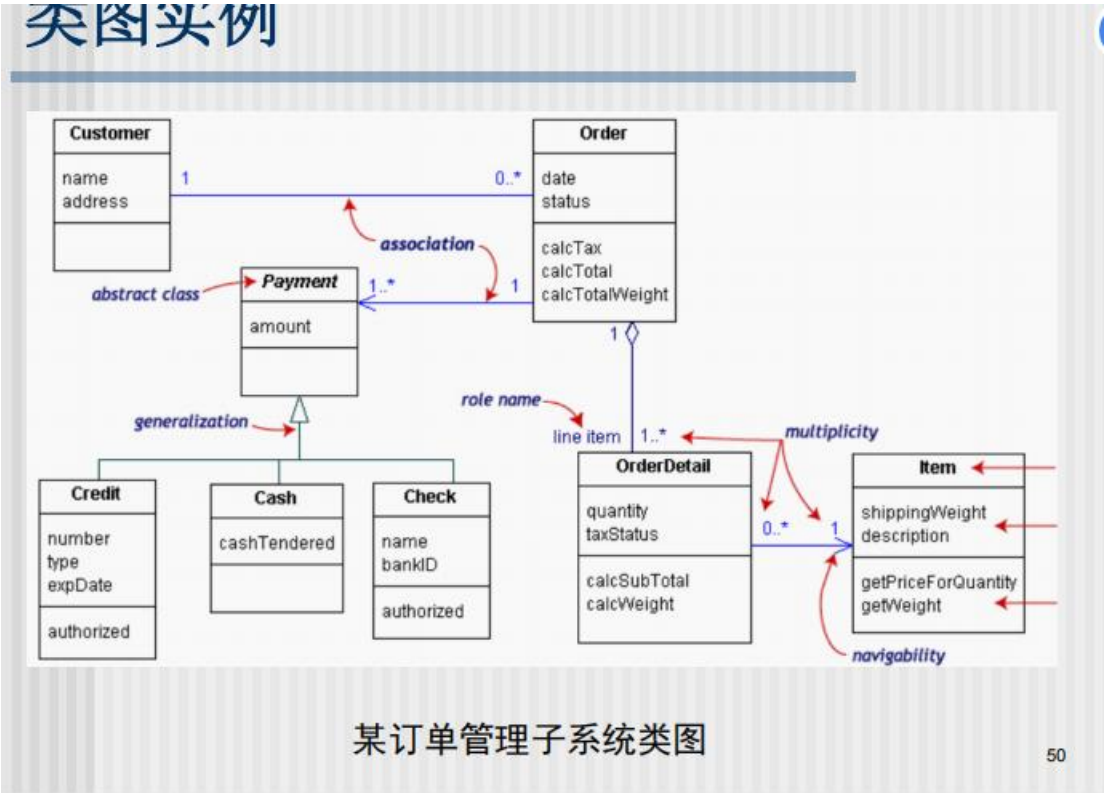
5. 泛化关系

表示类与类、接口与接口之间的继承关系




```
class Animal{}
class Tiger extends Animal{}
public class Test
{
   public void test()
   {
      Animal a=new Tiger();
   }
}
```

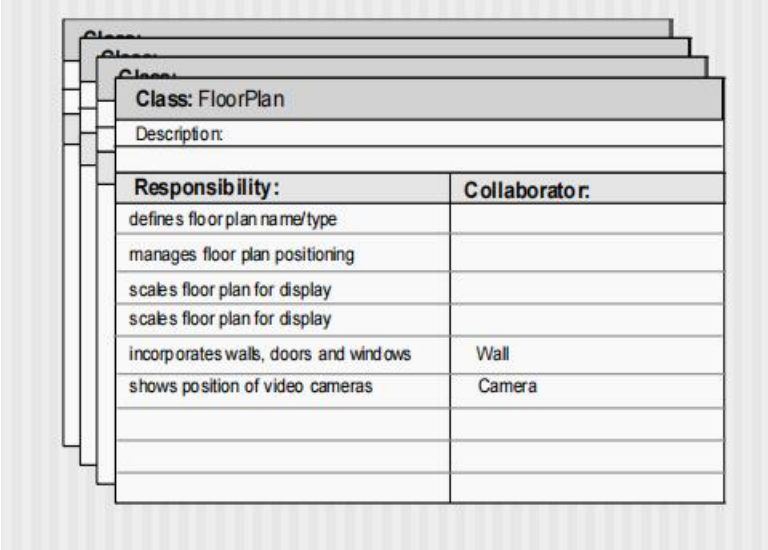
49



3）CRC Models

Class-responsibility-collaborator (CRC) modeling

作用：帮助补充、完善、优化分析类（属性、行为）



Responsibilities

职责是与类相关的属性和操作。

简单的说，职责就是“类所知道的或能做的任何事情”,可理解为类的能力。

例如：顾客有名字、地址和电话号码，这是顾客知道的东西。顾客要借书和还书，这是顾客要完成的事情。

分配职责的指导原则：属性和与之相关的行为应该在同一个类中。

Collaborations

类主要通过两种方式实现其职责：

通过自己内部的行为，实现特定的职责。

请求其它类的协作。

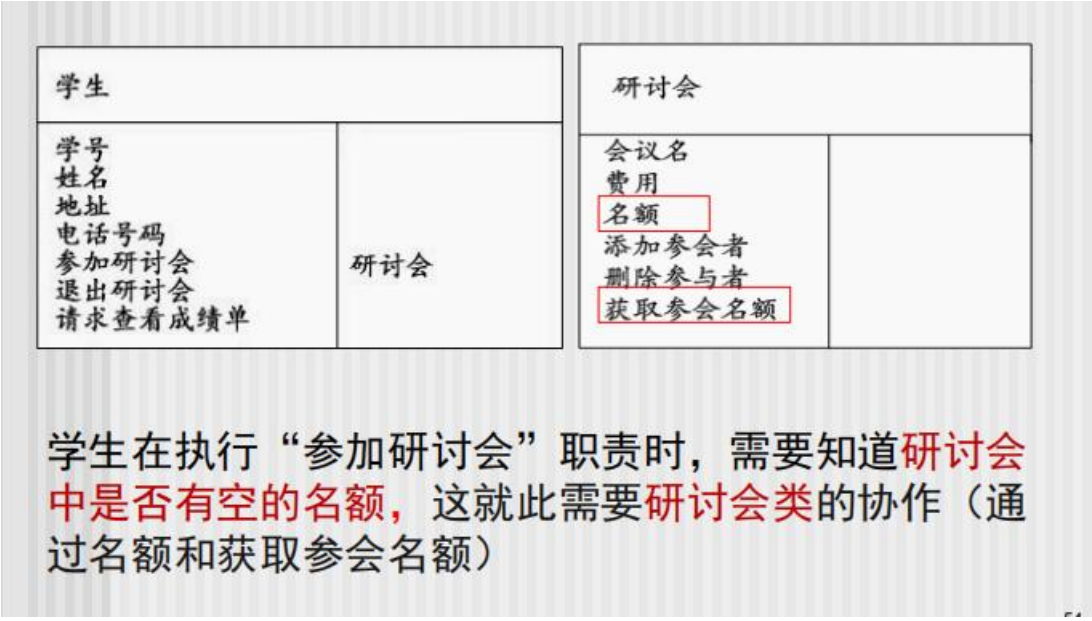
如果某个对象请求协作，就会向其它对象发送消息。

消息接收者作为服务方，回应消息发送者的协作请求

职责是一种较高层次的抽象，其在设计阶段可以被细 化为一到多个具体的方法。

如：登录控制类的职责之一是“身份验证” 。该职责在设计阶段，需要被细化为多个方法：getUser()，getRole()， getGroup()，validate()

使用 CRC 模型



Packages

对于较大的项目，由于有很多的类和用例，常常会使系统结构看起非常复杂。

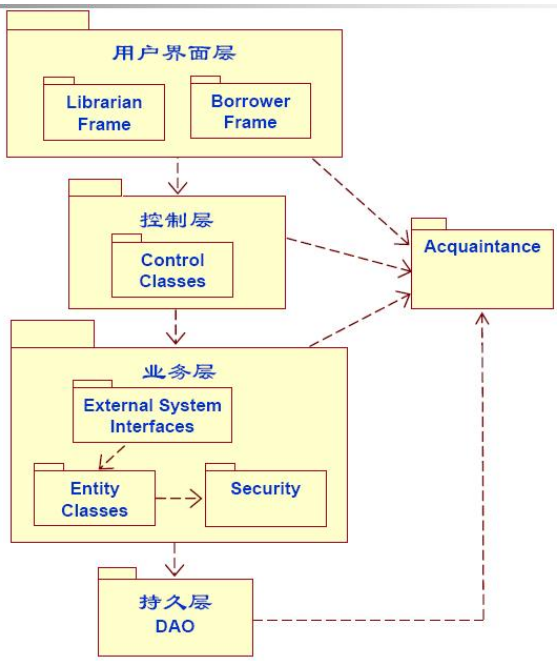
通过包图，庞大系统之间的关系变得更简单，整个系统的架构一目了然。

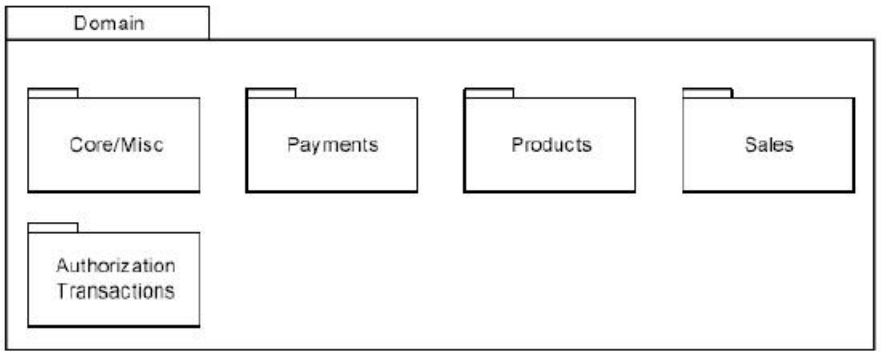
原则：将概念上或语义上相近的类纳入一个包

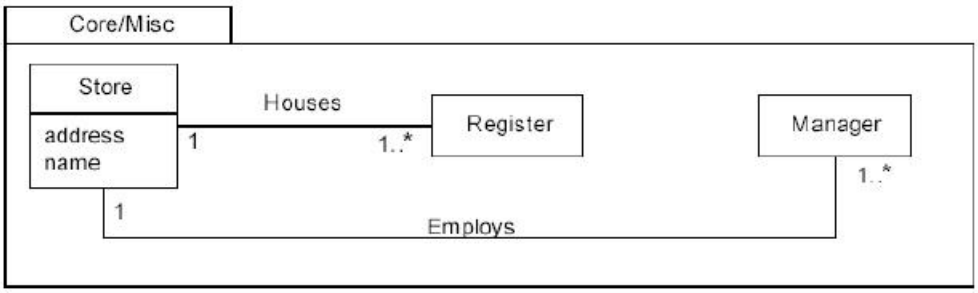
Example:Package

包之间的主要关系是依赖关系。

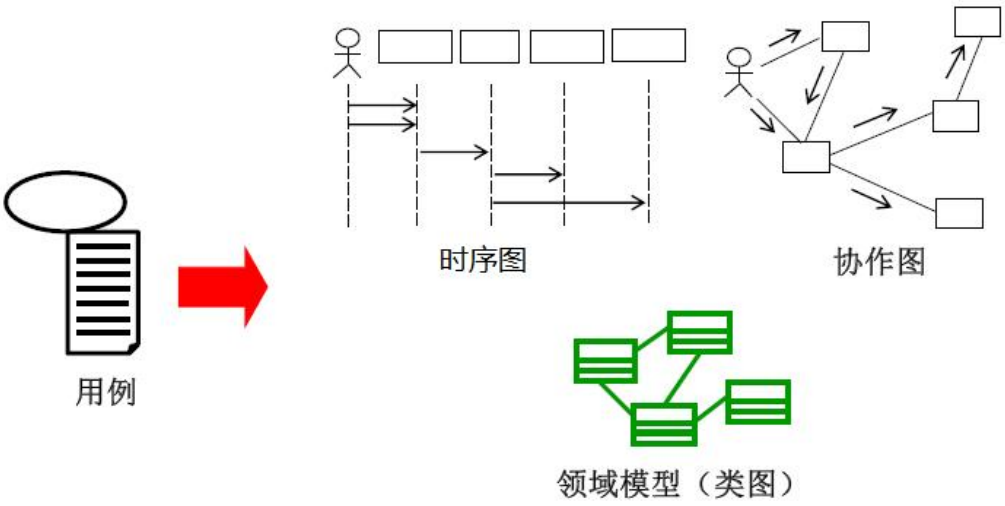
意味着两个 Package 内的元素之间存在着一个或多个依赖







OOA 分析回顾



通过领域建模（类图），我们已经确定了系统中主要的分析类。

但要确定分析类的行为，还要进一步对行为(动态)建模。

例如：之前 ATM 存款用例中，对确定的读卡器类，点钞器类，其行为分别有哪些？

6.3.6 OOA 分析第四步：行为建模

行为模型显示了软件如何对外部事件或激励做出响应。

模型描述：一个用例中，系统接收到特定事件后，涉及的对象以及这些对象之间的交互行为。

行为模型主要包括：

Sequence diagram 时序图

Collaboration diagram 协作图

State diagram 状态图

1）时序图

时序图建模步骤

(1)对每个用例，建立系统时序图（可选）

(2)建立对象时序图

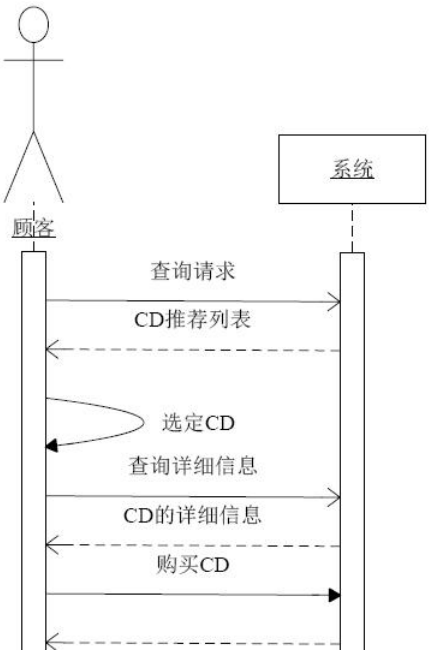
(3)从图中确定分析类的行为

系统时序图

系统时序图中将系统作为一个黑盒子

用例描述:

1. 顾客向系统提起查询请求
2. 系统根据请求为顾客提供一个CD的推荐列表
3. 顾客在推荐列表中选定一个CD，然后要求查看更详细的信息
4. 系统为顾客提供选定CD的详细信息
5. 顾客购买选定CD.
6. 顾客离开.

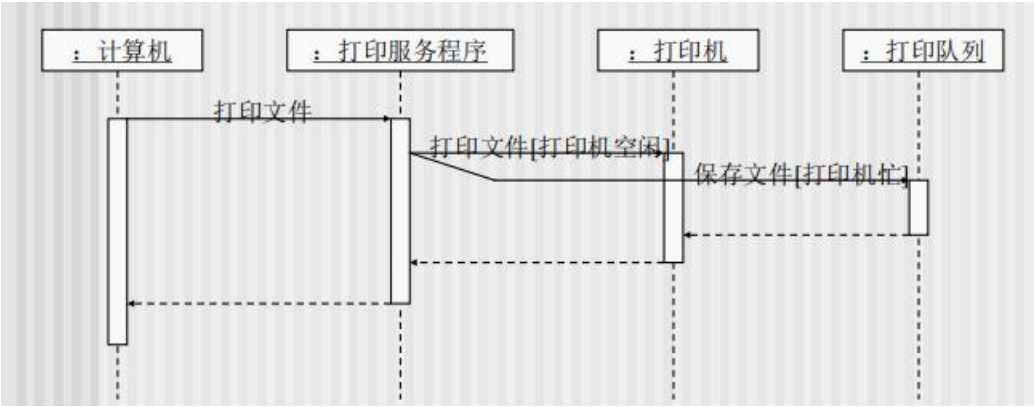


对象时序图

依据时间顺序，将系统的行为逐一分配到所识别的分析类中。

时序图中，纵轴是时间轴，横轴代表在用例中各对象

虚线表示对象存在的生命线，消息用从一个对象的生命线到另一个对象的生命线的箭头表示



主要步骤

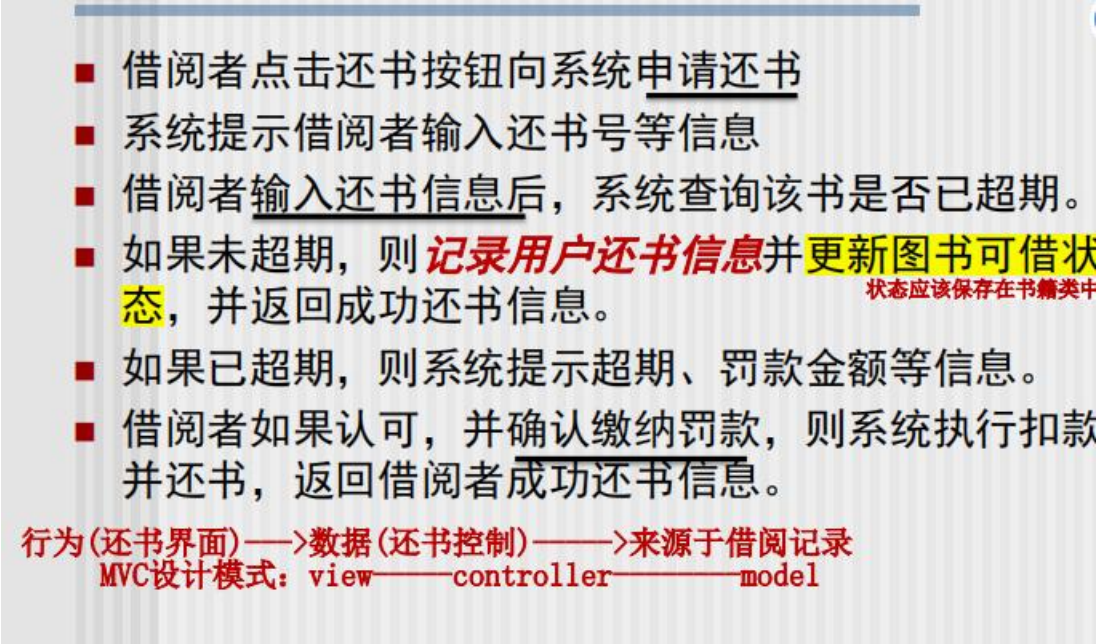
按从左到右的顺序

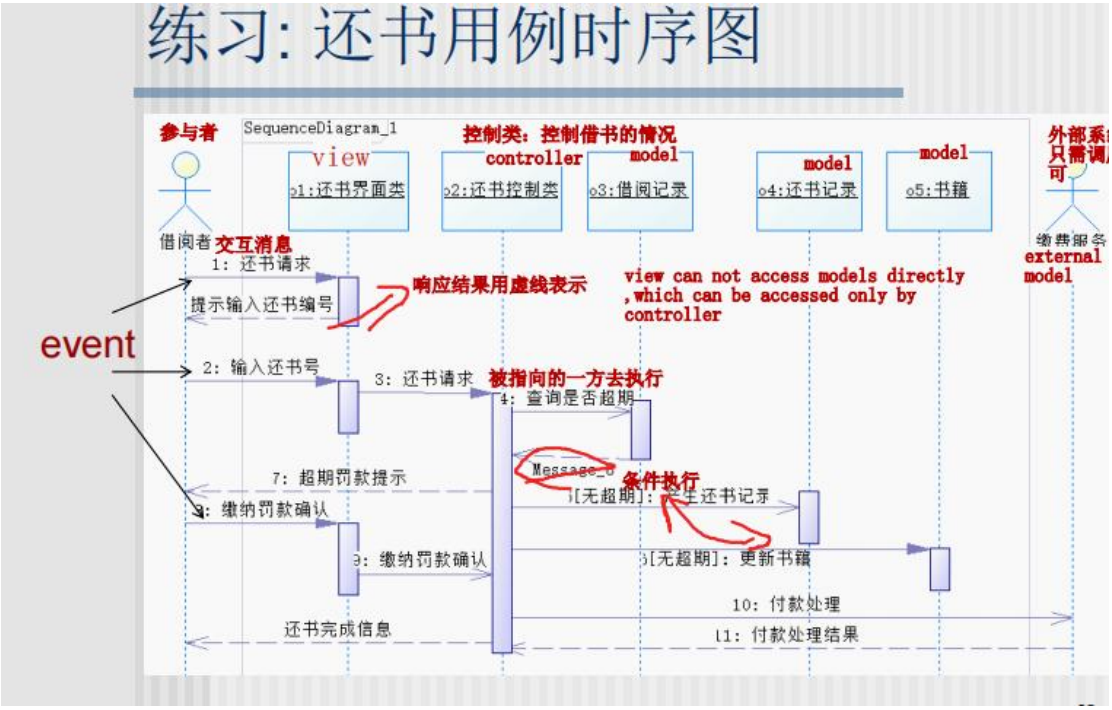
1、列出启动该用例的参与者；

2、列出参与者使用的边界类；

3、列出管理该用例的控制类；

4、根据用例的执行流程，按时间顺序列出分析类 之间的消息序列。





2）消息与职责的关系

对象接收的消息与其承担的职责之间存在一一对应关系，即如果一个对象能够接收某项消息，它就应当承担与该消息相对应的职责

注：部分消息可能不对应任何职责，如返回消息。

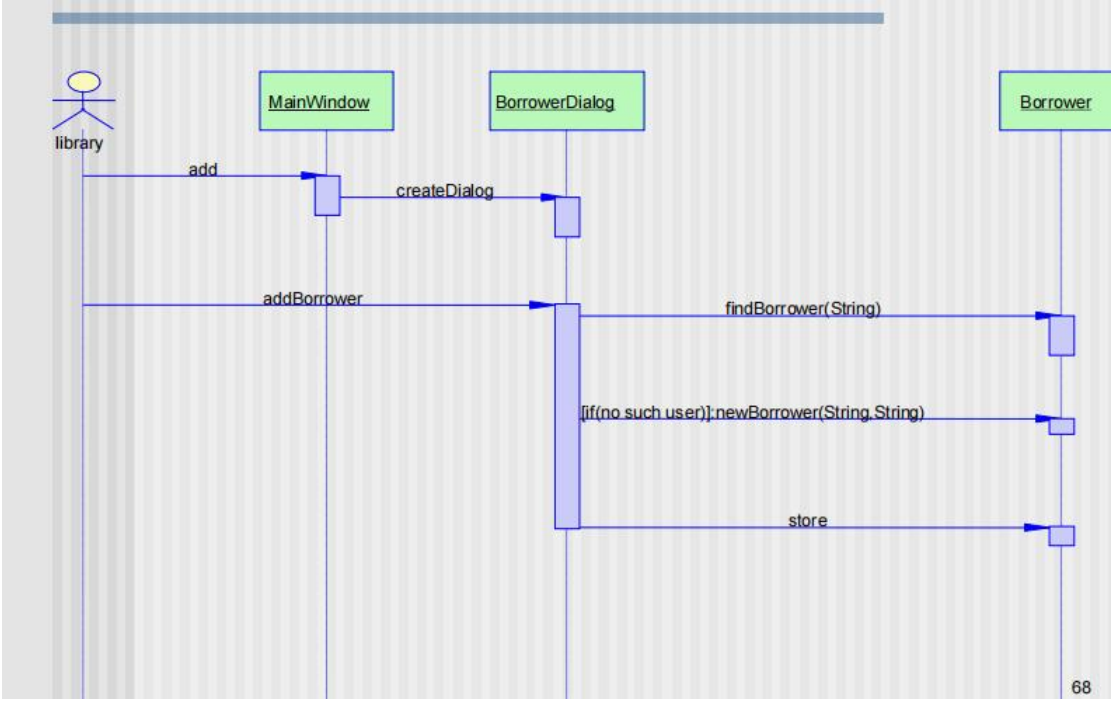
练习：添加借阅者

（1）管理员选择菜单项“添加借阅者”。

（2）系统弹出借阅者对话框，管理员输入借阅者信息并提交。

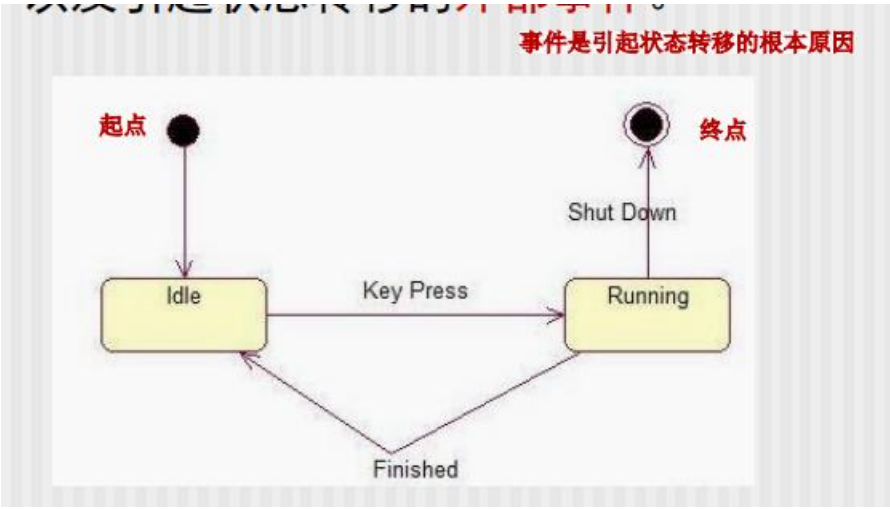
（3）系统根据借阅者 ID 查询数据库，看是否存在该借阅者，如果不存在，创建该借阅者帐户。

画出该用例的时序图。



3）状态图

状态图描述了一个对象生命周期内所经历的状态序列，以及引起状态转移的外部事件。



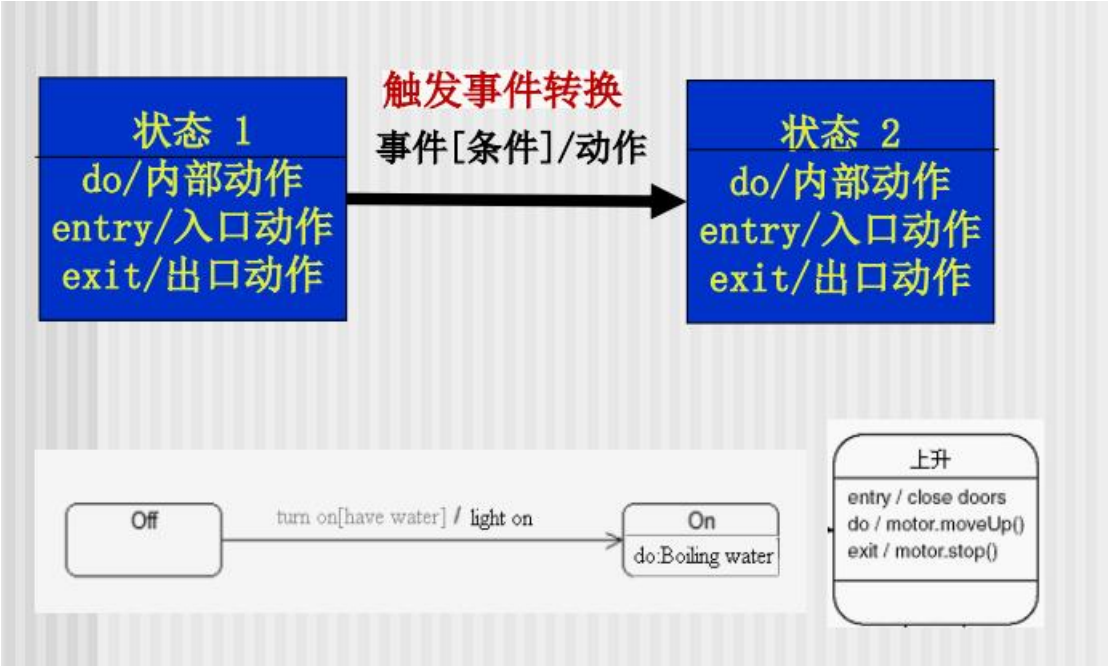
状态图的主要作用

(1)通过描述状态之间的转换顺序，分析相关的事件执行顺序，避免非法的事件序列。

如: 必须先付款进入“已付款”状态，才能通过“发货”事件进入“已发货”状态。

(2)状态图清晰的描述了状态转移时所必须的事件、 护卫条件和动作等因素，有利于程序员控制程序中的非法进入。

状态图基本元素

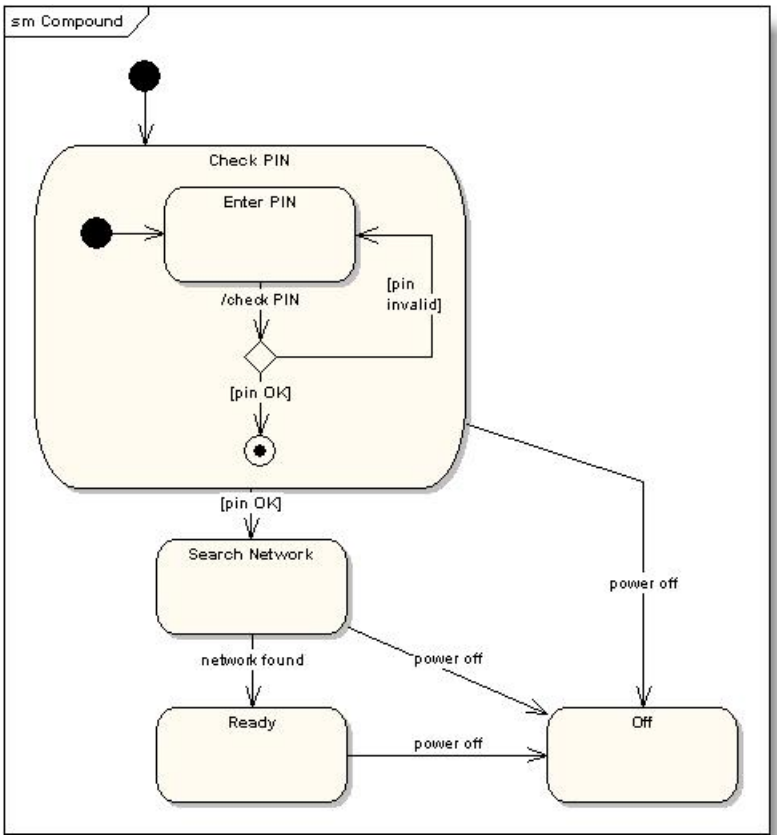


几种特殊状态

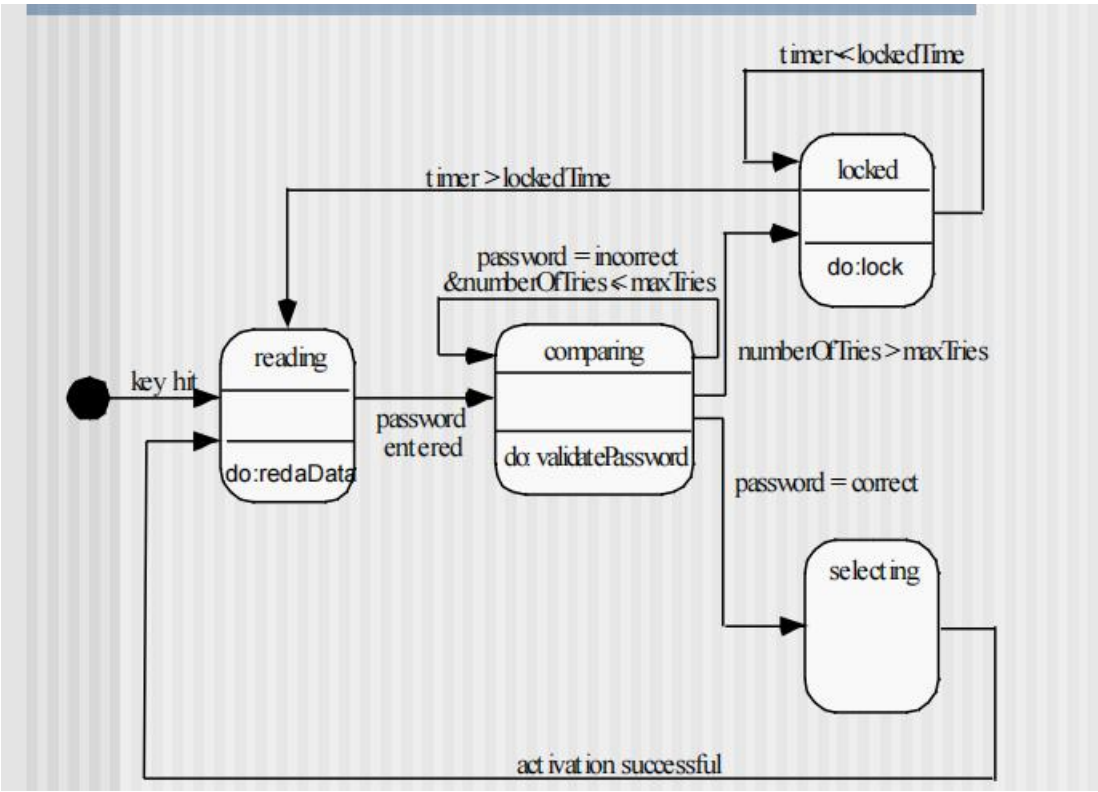
一个起始状态

多个终止状态

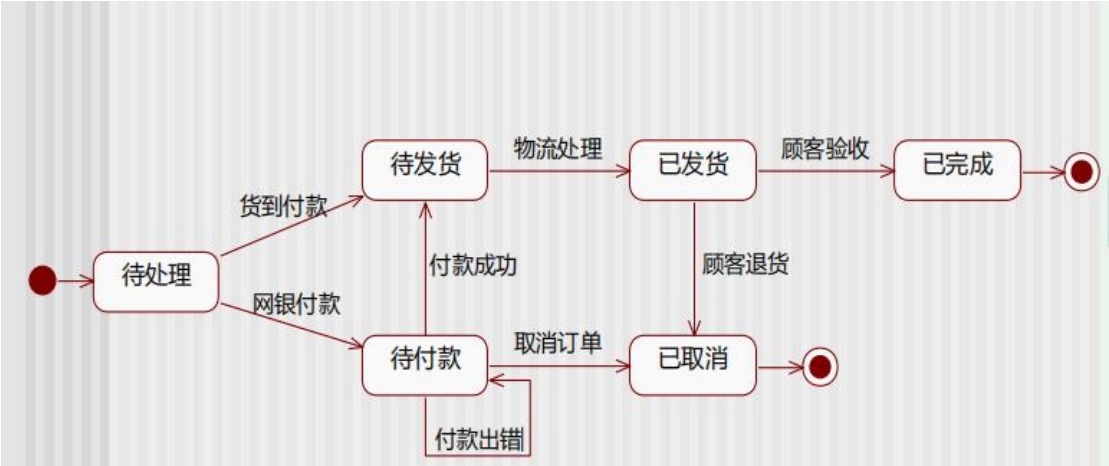
组合状态



控制面板类的状态图



订单状态图



4）行为模型比较

状态图 vs 时序图

状态图针对单个类，跨多个用例。

时序图针对的是单个用例

并不是所有的类都需要画状态图，*只有那些有明确状态*，且不同状态下行为不同的类才需要画状态图。

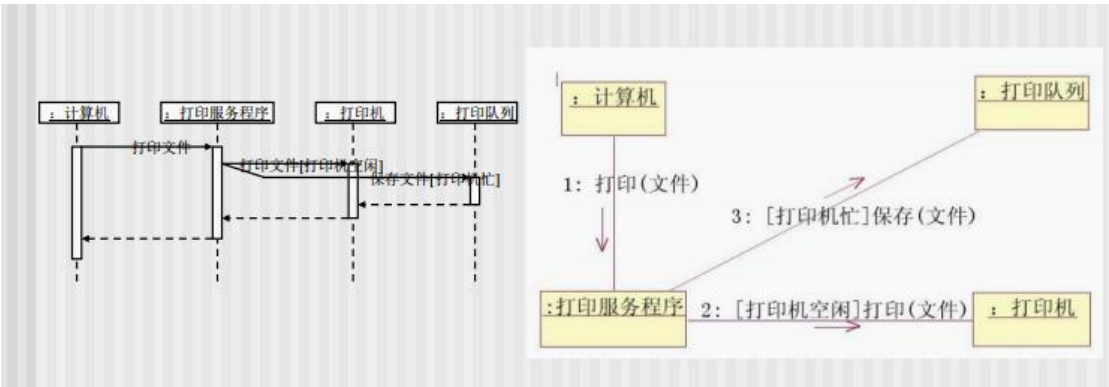
如：电梯类中，同一事件“向下按键”，在不同前置状态、护卫条件下状态变化是不同的，一定要***通过状态图明确状态变化关系，*** 否则就会出现系统行为紊乱。

并不是每个类都需要画状态图行为模型比较

时序图 vs 协作图

时序图可等价的转化为协作图。

区别：协作图更侧重表达对象间的关联关系，而非消息传递的时间先后。



5）面向对象分析总结

1.用例建模。寻找 actor 和用例，对用例场景进行描述。

2.静态建模。从用例场景中寻找分析类，建立初步的分析类图。
3.动态建模。用时序图描述每个用例的交互过程，以此发现每个类对应的职责。
4.建立包图，对分析类、用例等进行分包管理。（可选）
5.如果系统需要用到数据库，建立数据模型（可选）
6.如果类的生命周期比较复杂，则考虑建立状态图（可选）

Chapter 7 Requirements Modeling: Flow Oriented Modeling

7.1 结构化分析方法

结构化分析方法（SA）又称面向数据流的分析方法。

主要是创建基于数据流的模型，通过分析数据流发现系统功能和行为，并对其进行划分。

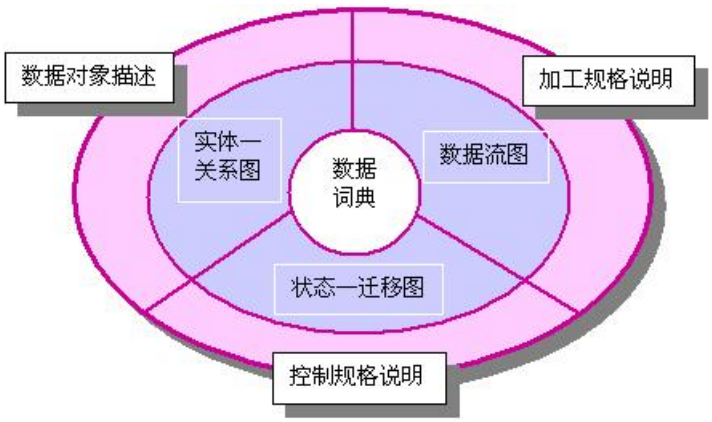
SA 四种分析模型

数据流图——功能视角

数据字典

实体关系图——数据视角

状态迁移图——行为视角

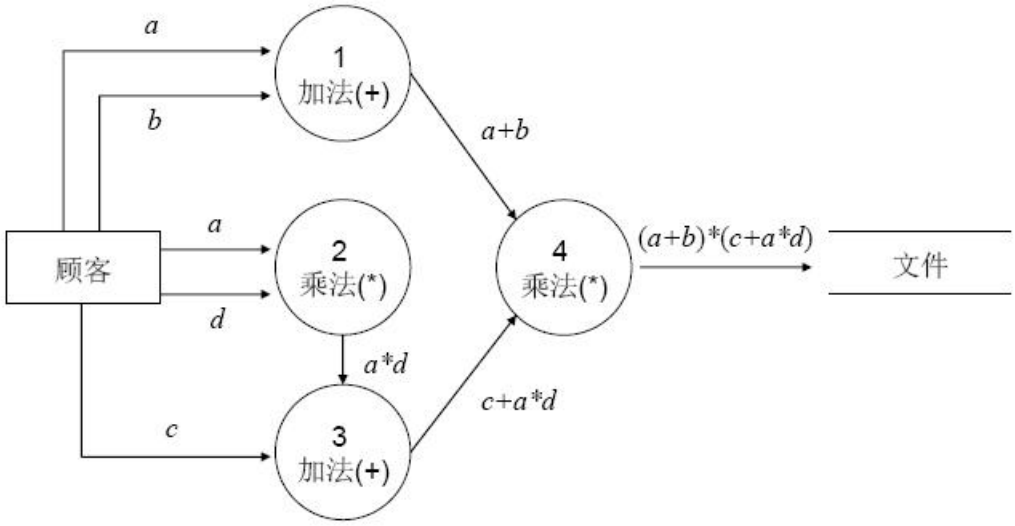


7.2 Data Flow Diagram

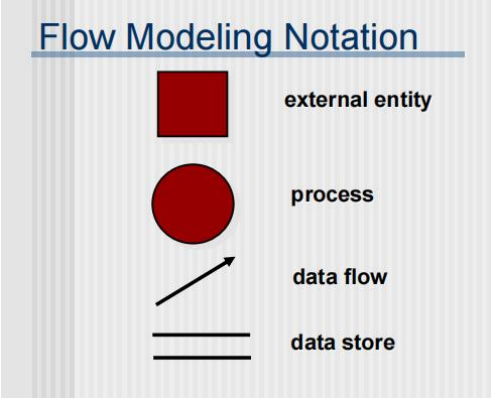
数据流图将系统看作由数据流联系起来的各种功能的组合。

通过 DFD 建模，分析问题域中数据如何流动，以及在各个流动过程中的加工、变化，从而发现数据流和功能。

例如：针对需要分析实现的业务 e=(a+b)*(c+a*d)，可以利用 DFD 发现若干功能：加法，乘法。



7.2.1 Flow Modeling Notation



7.2.2 External Entity

A producer or consumer of data

Examples: a person, a device, a sensor

存在于系统之外的人员、组织或其它系统

表示系统中输入数据流的源点或输出数据流的终点

7.2.3 Process

A data transformer (changes input to output)

Examples: *compute taxes, determine area, format report, display graph*

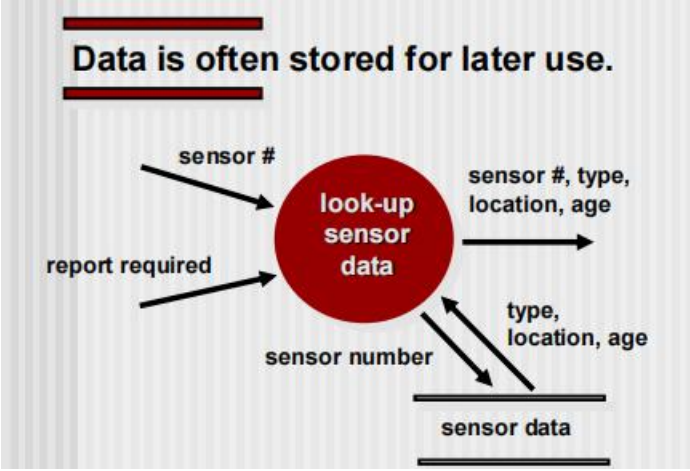
Data must always be processed in some way to achieve system function

7.2.4 Data Flow

Data flows through a system, beginning as input and transformed into output.

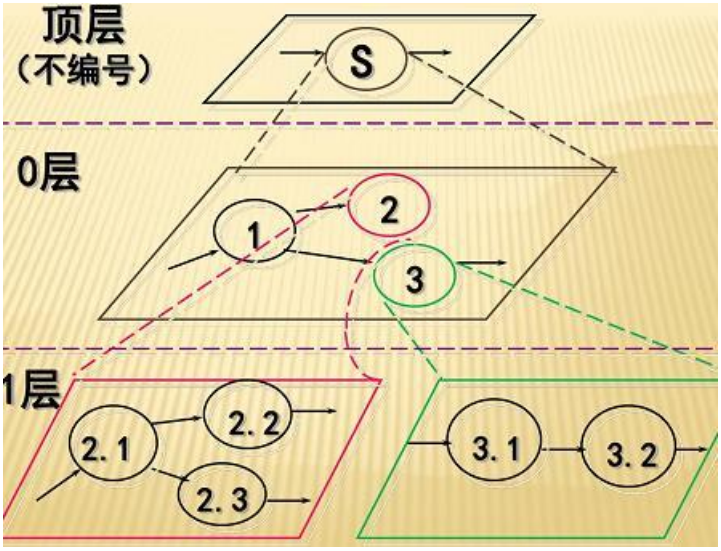
数据流可能存在于
外部实体与加工之间；
加工与加工之间；
加工与数据存储之间；

Data Stores



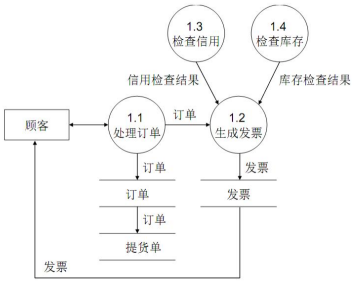
7.2.5 DFD 建模方法：详细

DFD 建模采用自顶向下逐层分解的方法



每个加工需编号
每个加工必须既有输入数据流，又有输出数据流

数据不能直接从一个数据存储流入另一数据存储，也不能直接从数据存储到外部实体



顶层数据流图

顶层数据流图只含有一个加工，表示整个待开发系统。

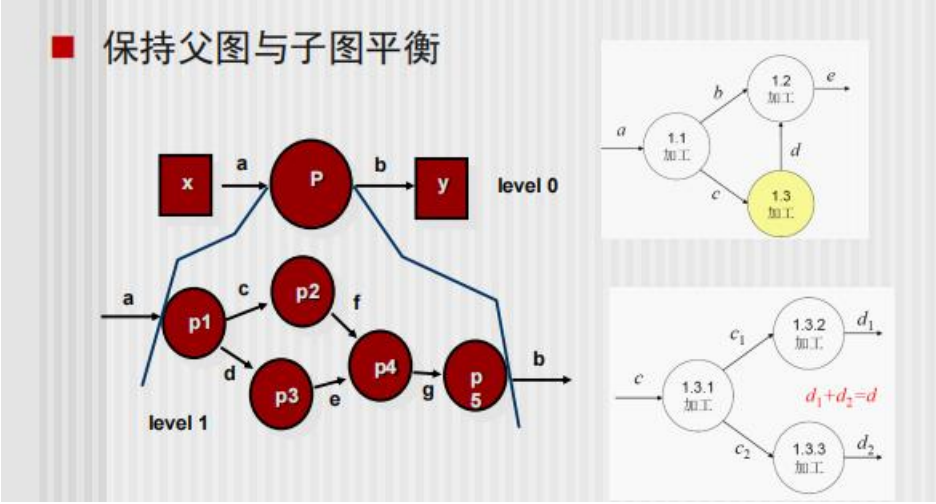
作用：确定与系统相关的外部实体，以及外部实体与与系统之间的数据流。



DFD 子图分解

先将顶层 DFD 中的系统分解为若干个子系统，每个子系统表示为一个子加工。

继续在下层 DFD 中对子系统进行分解，直至每个加工足够简单。



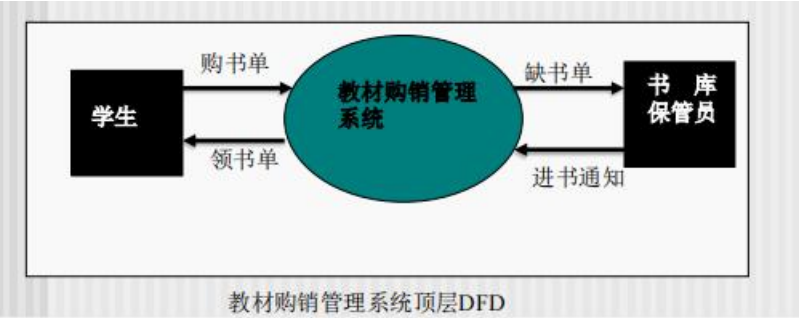
7.3 Example of DFD(1)

教材购销系统可分为两个子系统：销售子系统和采购子系统。

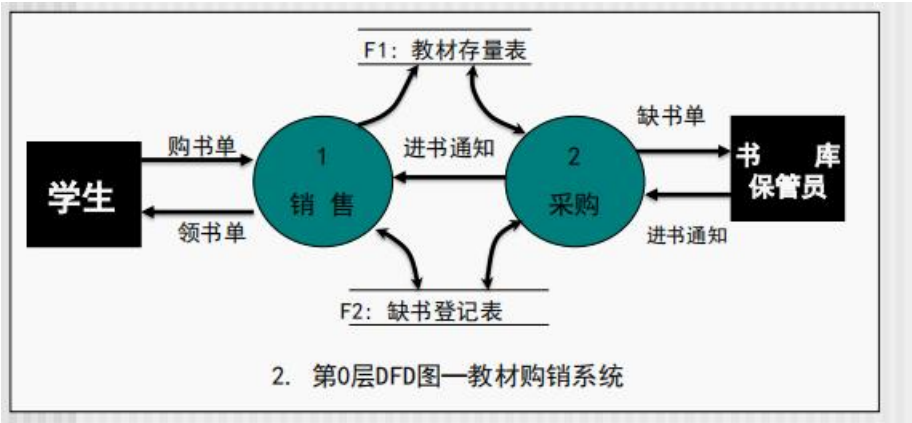
销售子系统：首先由学生提交购书单，经教材科发行人员审核是否有效后，缴费并开发票、登记购书情况并返给学生领书单，学生凭领书单即可领书。

采购子系统：若教材脱销，则登记缺书信息，并发缺书单给书库保管

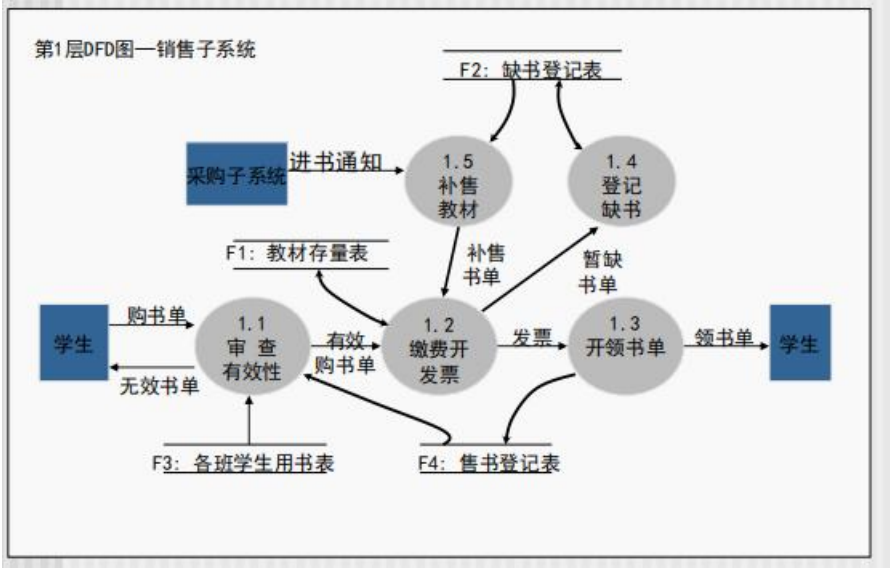
员，书库保管员线下完成采购；新书采购到即入库登记，并发进书通知告知系统。

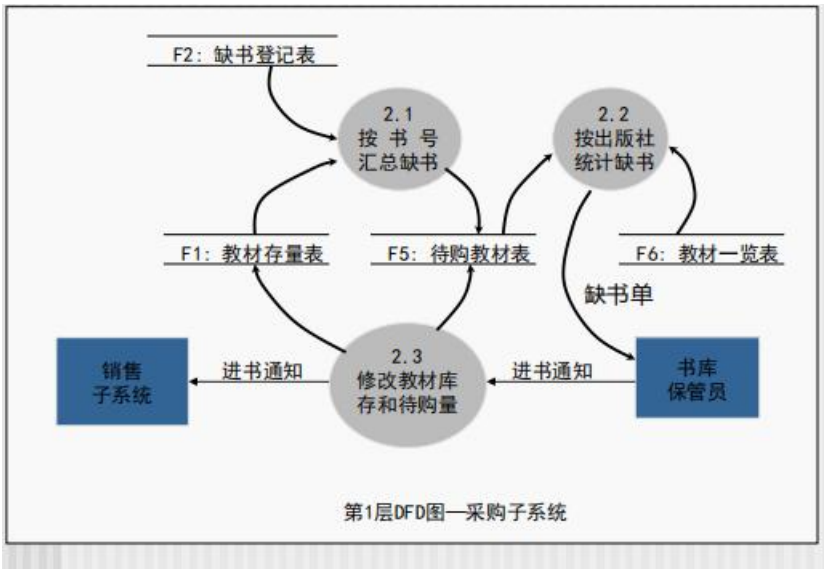


教材购销管理系统顶层DFD



2. 第0层DFD图—教材购销系统





第1层DFD图—采购子系统

Data Dictionary

数据字典主要由四类元素的定义组成： 数据流 数据项 数据存储 加工处理

1. 数据流

领书单 = 学院+专业+班级+学号+姓名+{书号+[书名]+数量}+日期
发票= 学号+姓名+{书号+[书名]+单价+数量+总价}+书费合计

2. 数据存储

教材存量表 = {书号+单价+数量}
暂缺书单 = 学号+姓名+ {书号+数量}
补售书单 = 学号+姓名+ {书号+数量}

教材科领书单

学院：	专业：	班级：	学号：		姓名：
书 号	书 名	数 量	书 号	书 名	数 量

20 年 月 日

发 票

学号：		姓名：			
书号	书名	单价	数量	金额	备注

3. 加工处理

由输入数据、加工逻辑和输出数据等组成。加工逻辑阐明
把输入数据转换为输出数据的策略。

输入： 购书单

输出： 有效购书单 或 无效购书单

加工逻辑：

1. 如果购书单中任一书号不在允许购书书目中，则为无效购书单；
2. 如果购书单中任一书号已经存在对应的班级购买记录，则为无效购书单；
3. 通过上述审核规则的购书单为有效购书单。

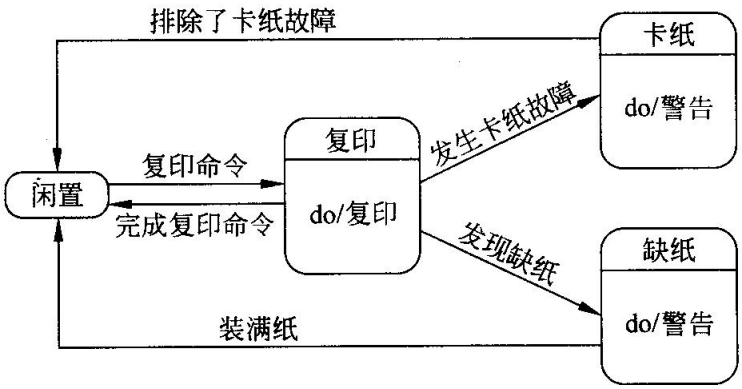
7.3 Control Flow Modeling 控制流

有些应用程序是由事件“驱动”的比数据，产生控制信息而不是报告或显示，并处理信息与非常关注时间和性能。

适合对实时系统、控制类系统等建模

以 State diagram 为主，描述系统对外部事件响应的行为模型

Example： 状态图



Chapter 8 软件设计

8.1 什么是软件设计

软件设计即根据软件需求，产生一个软件内部逻辑表示的过程。

这种更接近于软件的逻辑表达或模型，提供了软件系统关心的体系结构、数据结构、接口和构件等细节。

对比：

需求分析偏重于问题域，提出软件要“做什么”

软件设计则偏重于解决方案，描述软件要“如何做”。

需求1：教学秘书需要将学生的综合成绩按高到低进行排序

设计1：void OrderScores (struct * scores[ ]) { 冒泡排序算法, step1; step2;... }

8.2 软件设计的价值

设计是软件质量形成的关键，软件质量依赖于设计的优劣。

例：对含有上百亿信息的数据库，如何满足用户的高并发访问需求？

推荐设计方案：分库分表、读写分离。

设计是需求与实现之间的“桥梁”

- 设计本质是满足需求，并对实现方案选优的过程。
- 设计模型比代码更易评估、更易修改。

“编写一段能工作的代码是一回事，设计能支持某个长久业务的东西则完全是另一回事”

8.3 好的软件设计

设计必须实现分析模型中的明确需求，包括客户期望的隐性需求.（承上）

对后续的编码、测试以及维护人员，设计必须是可读、可理解的指南。

设计必须提供未来软件的全貌，从实现的角度说明其数据域、功能域和行为域。

例如，自动排课是系统的功能需求之一，那么设计要解决数据存储问题（排课信息保存）、界面显示问题（排课及结果显示）、算法逻辑问题（排课算法）等。

8.4 软件设计如何实施

软件设计是一个迭代的过程

初始时，设计者应关注系统的整体架构（如主要构件、连接关系）等，对应于体系结构设计阶段。

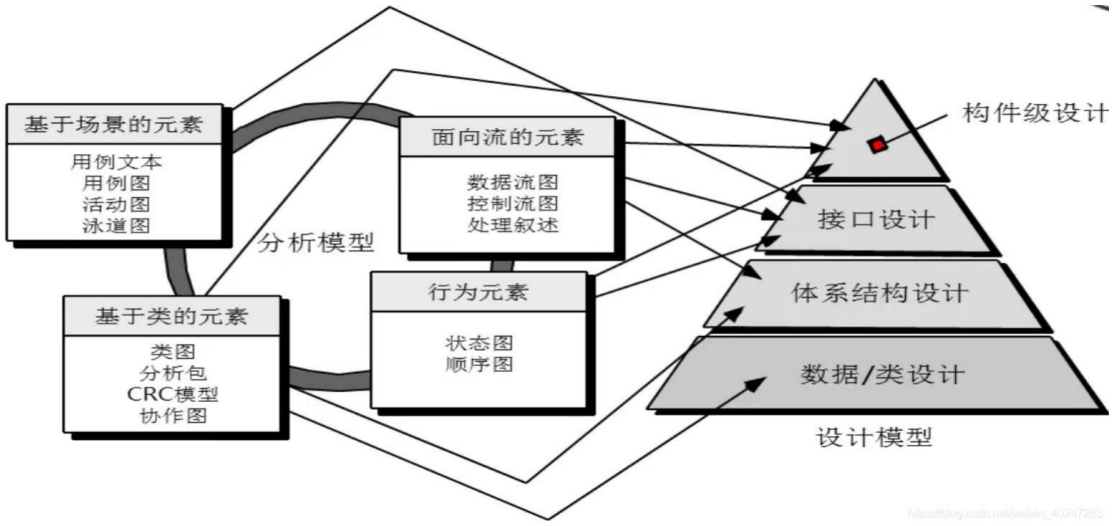
软件设计是一个迭代的过程

随着设计迭代，更多的细节被关注（如数据、构件）等，对应于构件设计或详细设计阶段。

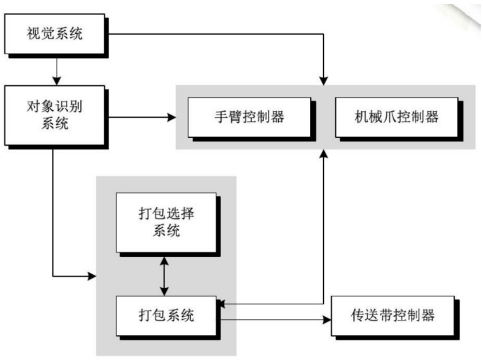
8.5 软件设计主要任务

(1)体系结构设计：定义软件的主要元素（构件）以及元素之间的联系
(2)数据/类设计：将分析类模型转化为设计类以及软件所需要的数据结构
(3)接口设计：定义软件与协作系统之间、软件与用户之间的通信
(4)构件设计或详细设计：定义软件元素（构件）的内部细节，如内部数据结构、算法等
(5)部署设计：定义软件元素在物理拓扑结构中的分布

8.6 软件设计模型

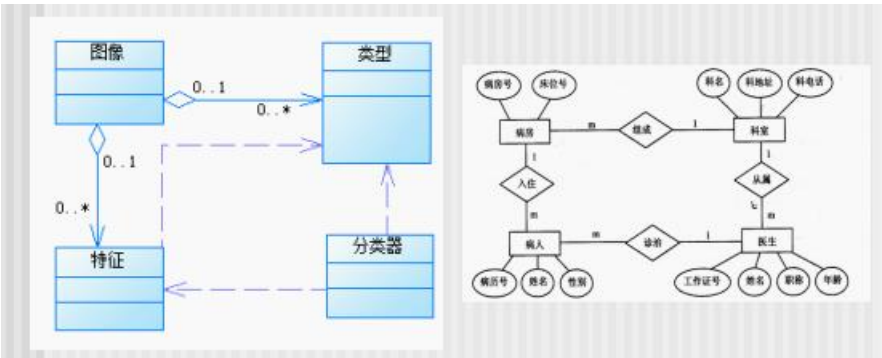


8.6.1 体系结构



8.6.2 类/数据设计

设计类图描述了“对象识别”子系统的静态类结构



8.6.3 接口

8.7 Fundamental Concepts

以下概念奠定了软件设计成功的基础

Abstraction—data, procedure, control

Architecture—the overall structure of the software

Separation of concerns—any complex problem can be more easily handled if it is subdivided into pieces

Modularity—compartmentalization of data and function

Hiding—controlled interfaces

Functional independence—single-minded function and low coupling

Refinement—elaboration of detail for all abstractions

Aspects—a mechanism for understanding how global requirements affect design

Refactoring—a reorganization technique that simplifies the design

Design Classes—provide design detail that will enable analysis classes to be implemented

数据抽象

抽象的本质：强调关键特征，忽略无关细节

如：对打车软件中的“车辆”，你会如何抽象？

8.7.1 过程抽象

8.7.2 体系结构优先

体系结构确定了系统的主体结构(类似于建筑结构)，结构优劣对软件质量具有重要影响。

后期改算法容易，改体系结构很难。因此设计应该先勾勒全貌，不能一开始就跳入细节。

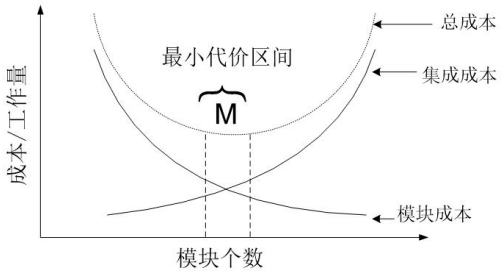
例如，在“垃圾自动分类”应用中，有两种可选体系结构：胖客户端和瘦客户端，如果考虑到分类精度、分类模型易更新等问题，你会选择哪种结构？

模块化

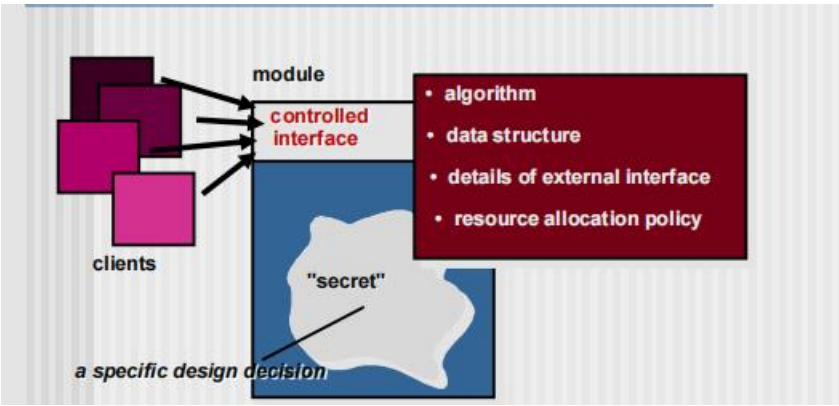
庞大的系统不便于并行开发，软件工程师也难以理解和把握。

采用分治策略，将大系统分解为若干更小、更简单的模块。

这样，问题理解变得更加容易，开发成本也会减小。

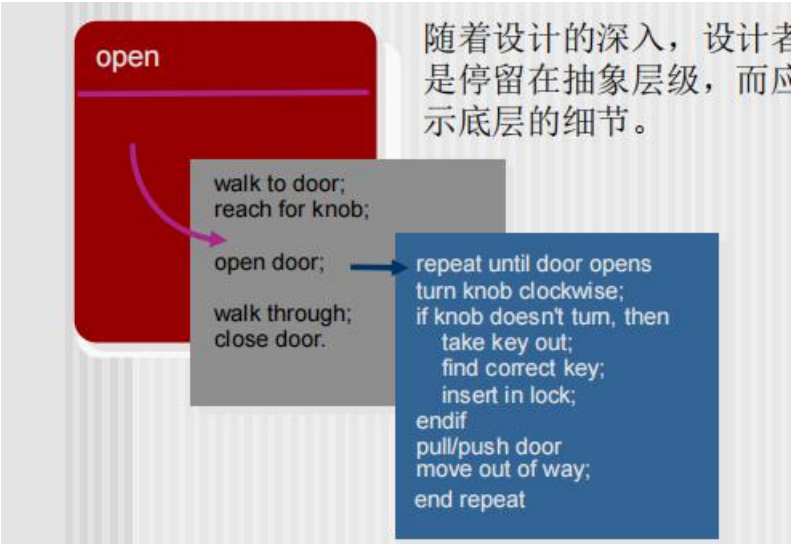


信息隐藏



每个模块都对其它模块隐藏自己的设计细节（就像黑盒子），这样如果模块内部发生变化，外部受到的影响最小。

8.7.3 逐步求精



随着设计的深入，设计者不能老是停留在抽象层级，而应逐步揭示底层的细节。

8.7.4 功能独立

评价模块化时重要的参考：功能独立性。

功能独立是指：希望每个模块尽可能功能专一，同时避免与其它模块过多交互。

Cohesion(内聚)： 表示模块内部相关元素的依赖强度

如果模块内多个元素彼此不相关，则内聚性低。

内聚性强的模块通常只完成一件事情。

Coupling (耦合) ：表示模块之间的相互依赖性

紧密耦合意味着一个模块的变化会对其它模块产生较大影响

模块的耦合性取决于模块之间接口的复杂性、引用方式、传递数据的复杂性等。

功能独立意味着：高内聚、低耦合功能独立

功能独立的优势

更容易开发。因为每个功能均有效分隔，且接口简单。

更容易维护。因为修改代码引起的副作用被限制，减少错误扩散。

复用性更好。

重构

在不改变代码的外部行为的前提下，优化内部结构的过程。

例如：早期设计中发现某构件的内聚性较差，在保持接口不变的情况下，可通过重构将其分解为两个独立的构件，每个新的构件内聚性得以增强。

设计类

分析类只与业务领域有关，和具体实现技术无关。

设计类则面向实现，通过对分析类的细化设计，达到服务于后期编码实现的目的。

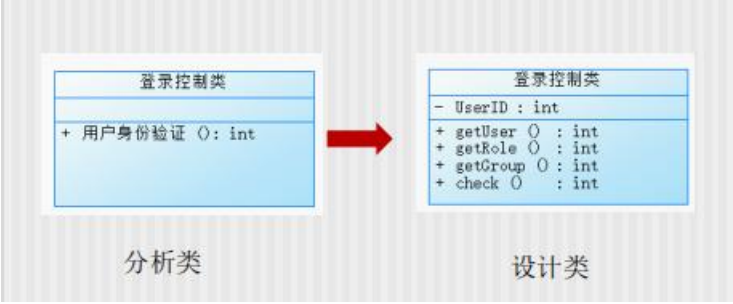
构造设计类的两种方式

(1) 通过对分析类提供细节设计。

(2) 创建新的设计类，该设计类提供软件的基础设施以支持业务解决方案。

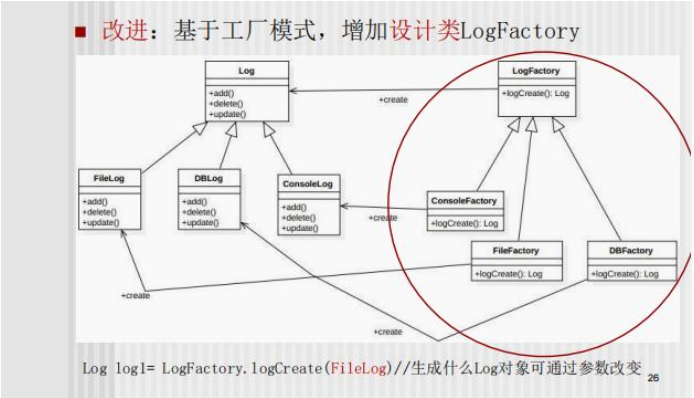
(1)通过对分析类提供细节设计

主要包括对类的成员属性、方法、关系等的细节设计。如：



(2)创建新的设计类

例如：某日志系统类图如下，包括多种日志类。 日志对象主要通过类似：FileLog f=new FileLog()生成。 问题：一旦系统中更换了日志类，每个调用的地方都要修改代 码。



Chapter 9 体系结构设计

9.1 理解体系结构及设计

(1) 什么是软件体系结构

软件体系结构(也称架构)是软件的一种划分形式，从较高抽象层次定义系统的构件、构件之间的连接，以及由构件与构件连接形成的拓扑结构。

构件是一个抽象的概念，在程序中可以指函数、模块、对象、类等。

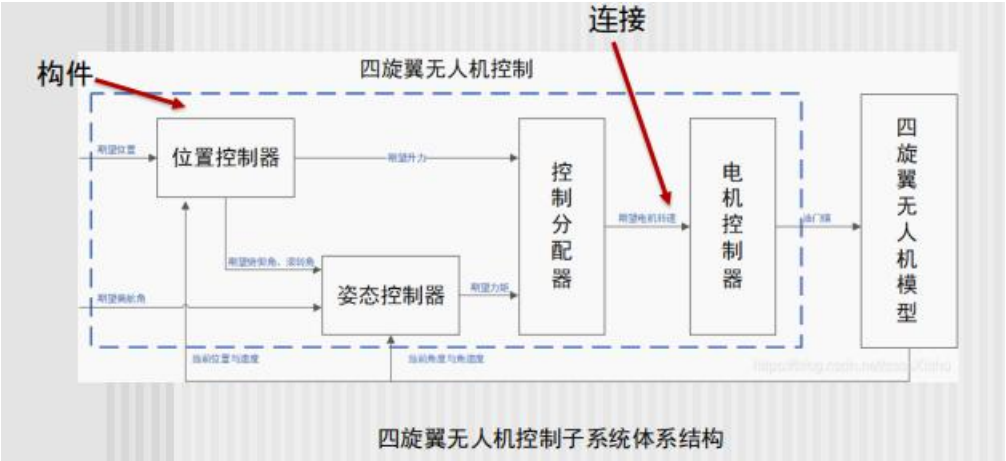
连接方式包括过程调用、数据流、消息机制、共享数据等。

从计算机看“体系结构”

主要硬件模块：控制器、运算器、内存储器、外存储器、输入设备.....

硬件模块之间的连接关系：总线（控制总线、地址总线、数据总线）

软件体系结构实例



(2) 体系结构设计中的关键决策

系统的基本构造单元(构件)是什么？ - 图像获取 、图像矫正、图像去噪……
这些构造单元连接方式？ - 函数调用 or 消息 or 通信中间件……
最终形成怎样的拓扑结构？
有哪些体系结构风格可供系统借鉴？
对于目标系统，如何基于一般性的体系结构进行适应性改造？
如何评估体系结构设计？

9.2 软件体系结构风格

不同风格体现在构件类型、连接机制、拓扑、约束等差异性上
不同体系结构风格，适于不同类型应用。

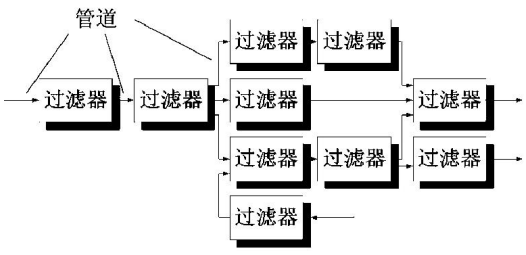
数据中心结构
管道-过滤器结构
调用返回结构
消息总线结构
面向对象结构
层次化结构

9.2.1 数据中心结构

将数据存储构件作为核心，与其他构件紧密结合，形成类似于星型结构的拓扑关系。

缺点：一旦中心数据存储出现问题会影响整个系统。

9.2.2 管道-过滤器结构



管道—过滤器体系结构

系统由若干构件(过滤器)按一定顺序组合而成，一个构件的输出是另一个构件的输入。

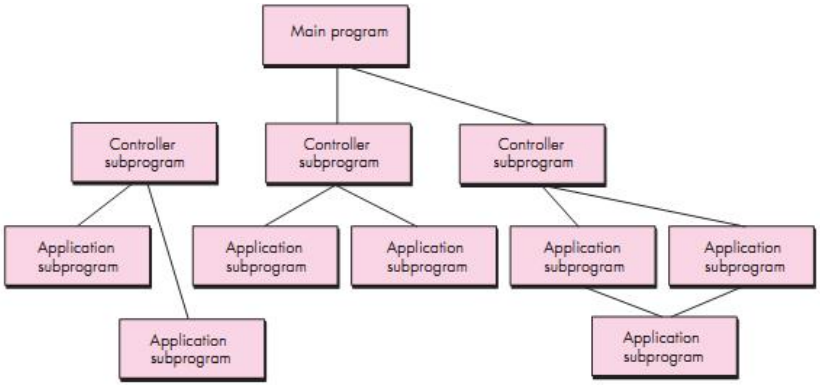
适合于批处理系统，不适用于交互式应用系统。

编译器就是一种管道-过滤器系统，包括词法分析、语法分析、中间码生成等构件。

图像处理系统等也大多采用该风格。

9.2.3 调用返回结构

(1)主程序/子程序结构



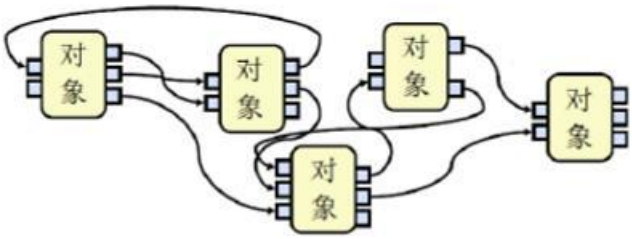
(2)远程过程调用结构

主程序/子程序中的构件分布在网络中的多台计算机。如：如分布式计算系统 Hadoop。

调用返回结构缺点

- 主程序需要预先知道被调子程序的接口标识。
- 程序接口标识的更改会影响所有调用它的其它程序，而且这种现象还会进行传递。

9.2.4 面向对象结构



构件：类和对象

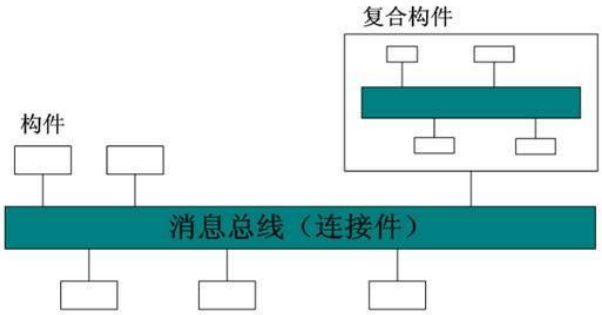
连接：基于消息机制。

拓扑结构：不同对象之间是平级的，没有主次之分。

构件不直接调用其它构件，而是广播一个或多个事件。

系统中的构件注册(监听)一个或多个事件源；

当一个事件被广播，系统自动调用该事件中注册的构件。

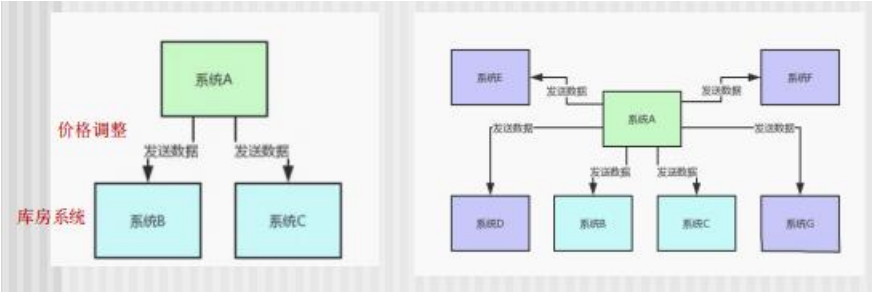


9.2.5 消息总线结构

如今，消息总线风格逐渐演变为消息中间件。

消息中间件应用场景：系统解耦、异步通讯、削峰。

下图为传统方式下系统 B、C 与 A 之间的通信连接，随着新的 D、E、F、G 系统的增加，必然导致巨大的维护成本（加入或退出均需要维护 A）。



9.2.6 层次化结构

系统被组织成若干个层次，每层都由一系列构件组成。

同层内构件可相互调用。

层之间存在调用接口，但只限于相邻层，下层可为上层服务，并作为再下一层客户。

9.2.7 客户-服务器结构

源于计算任务和计算资源的能力差异。

构件分布在两台或多台计算环境中。包括胖客户端和瘦客户端

如银行 ATM 系统，远程管理调度系统

演变：三层，多层 C/S，B/S 体系结构

9.2.8 MVC

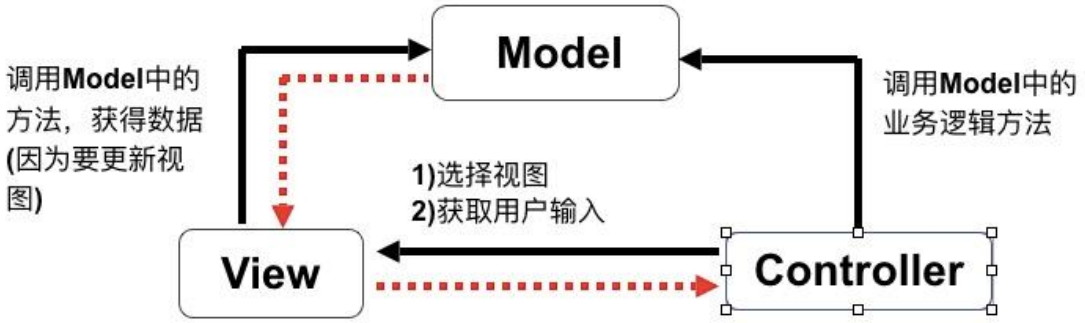
Model-View-Controller

常用于 Web 应用系统。

将系统分解为应用逻辑、用户界面、控制逻辑等不同构件类型。

优势： 任何一种构件的改变都不会对其它构件造成影响。

一个模型能为多个视图提供数据。



9.2.9 微服务架构

单体架构：应用打包(如 war)部署为一个独立的单元，通过一个进程方式运行。

微服务架构

应用(功能)被分割成更小的、独立的服务，各个微服务之间的关联通过 RPC(同步调用)或 MQ(异步调用)实现。

独立的微服务不需要部署在同一个虚拟机或应用服务器中。

9.2.10 混合风格

大规模系统往往包括多种体系结构风格

9.3 体系结构描述

系统体系结构多从逻辑架构、物理架构和开发架构、技术架构等视角描述。

逻辑架构关注软件系统中构件之间的关系，比如用户界面，数据库，外部系统接口，商业逻辑等。

物理架构关注网络、服务器等基础设施及其拓扑结构。

开发架构更关注程序包，除自己写的程序外，还包括应用程序依赖的 SDK、第三方类库等。

物理架构实例

如何保障网站能支持同时 10W 人在线、7*24 小时提供服务?

Nginx 实现负载均衡，将请求路由到对应的 Tomcat
Keepalived 用于监控系统中服务节点（Nginx）的状态。

9.4 体系结构设计方法

基本设计思路



包括：

面向对象设计方法

传统结构化设计方法

9.4.1 面向对象设计方法

(1) 定义系统交互的外部环境和交互特性

- 目的是识别系统的外部实体与接口,定义接口构件

(2) 识别系统内的结构原型(archetypes)

- 原型是系统中最核心的抽象类或模式，目标系统的体系结构可由这些原型组成。

- 原型(archetypes)是系统中最稳定的元素，可以基于系统行为以不同方式对这些元素实例化。

- 多数情况下，原型可以通过分析类来获得。

(3) 对每个结构原型进一步细化，以完善系统结构。

完整的 safehome 系统包括安全功能、监视功能、住宅管理等多个子系统。

案例 1：“安全功能”子系统体系结构设计。

“安全功能”子系统的需求：

允许户主通过控制面板 解除或开启 安全功能。开启后，传感器开始工作。

系统对传感器信号分析，检测是否达到报警条件。

一旦检测到报警信号，即通过电话、告警器等方式报警。

思考：该子系统涉及哪些用例？每个用例涉及哪些分析类

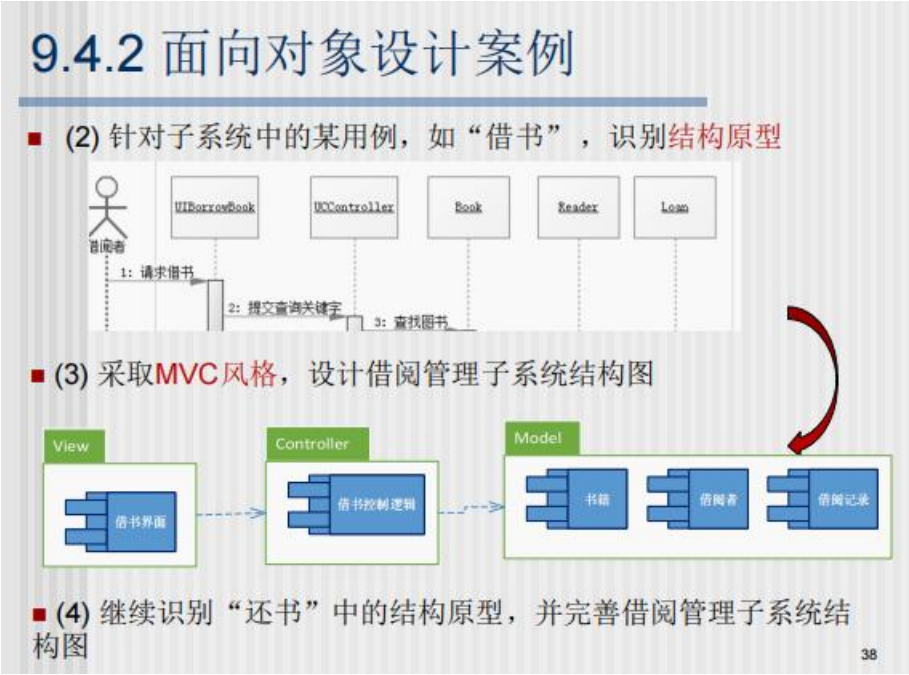
(1)识别系统外部环境

(2)定义原型

(3)设计安全功能子系统顶层结构 包括控制面板处理、探测器管理、报警处理

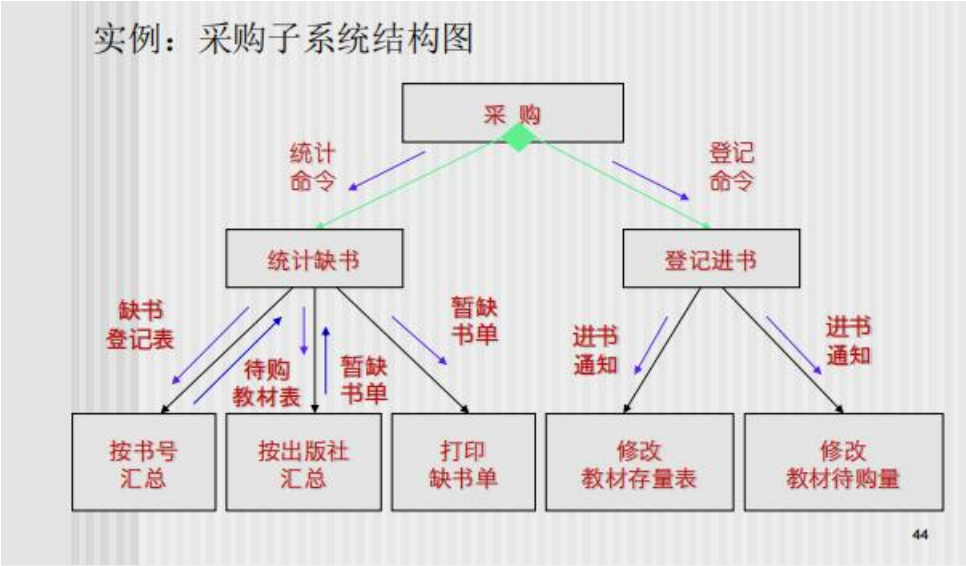
(4)安全管理子系统细化

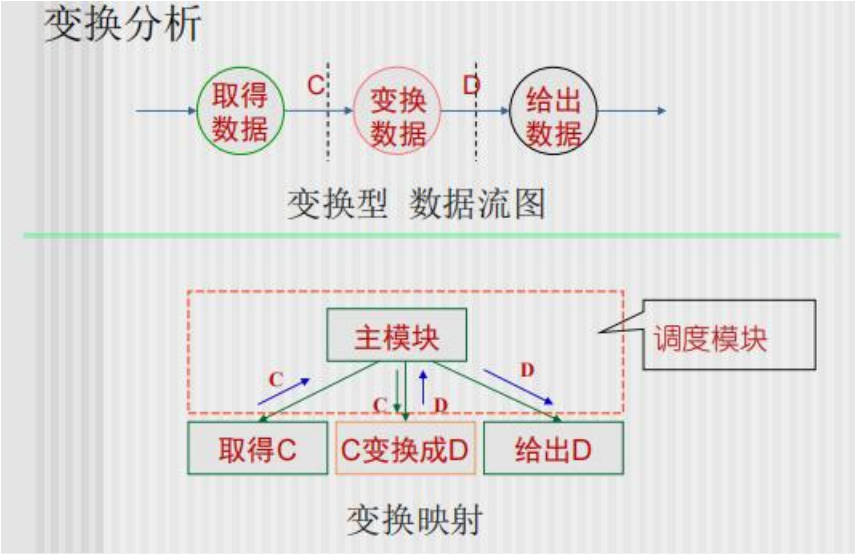
（5）采用分层体系结构，依据构件职责将其置于适当的层次。

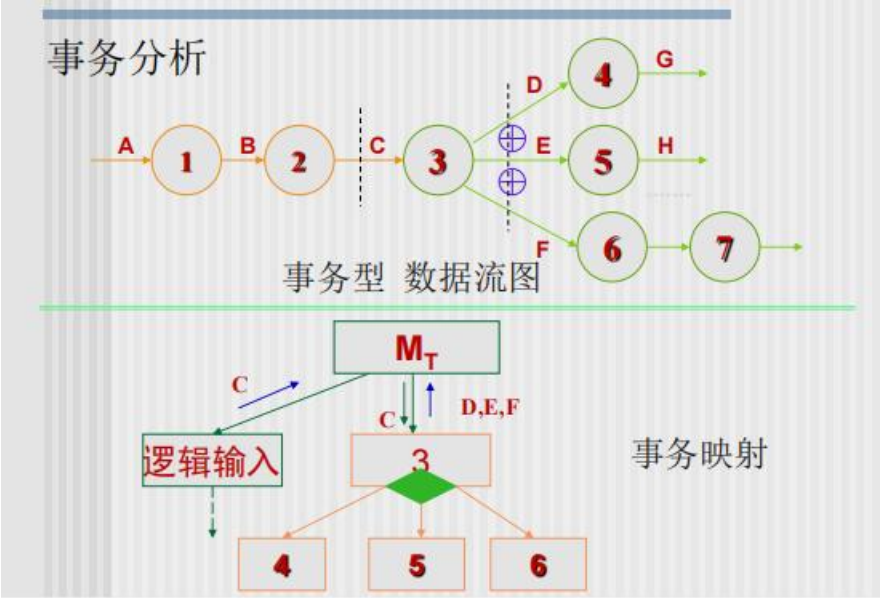


9.4.3 结构化设计方法

体系结构风格：主程序/子程序风格
将系统分割为多个功能模块
确定各个模块之间的调用关系
用系统结构图表示设计模型





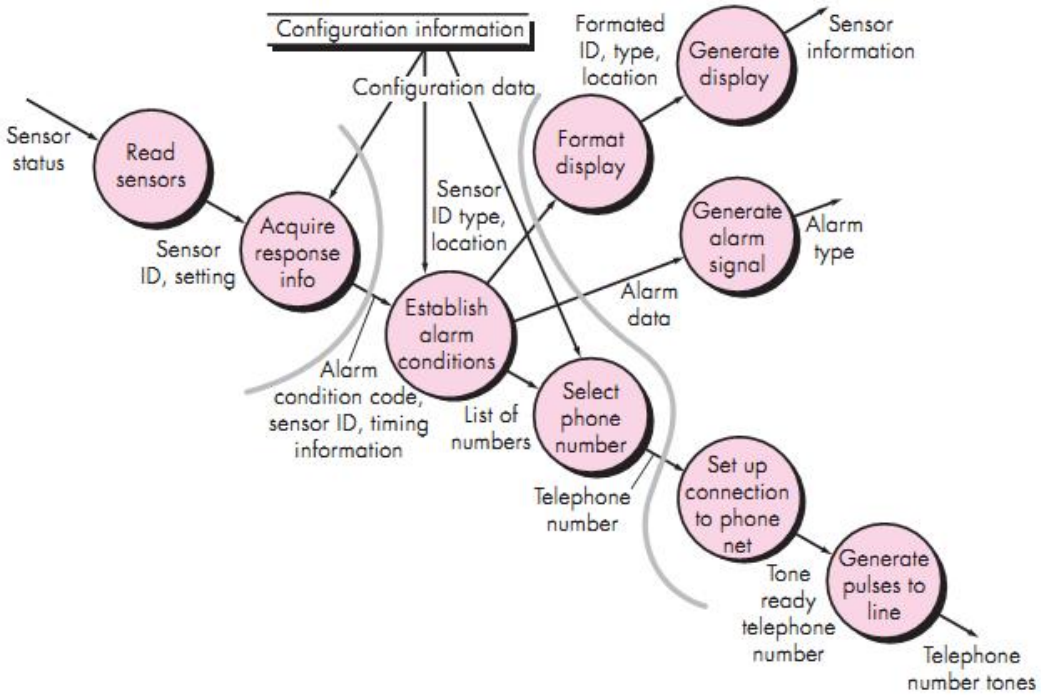


结构化设计基本步骤

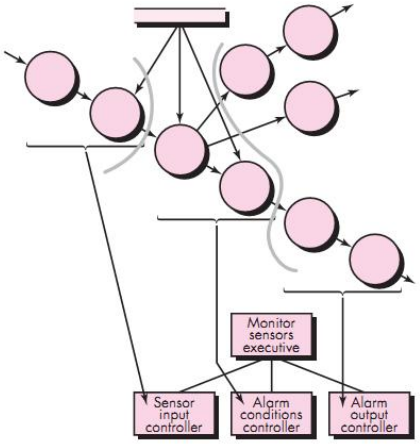
(1) 分析确定数据流图类型。
(2) 标识输入、输出流边界。
(3) 依据变化或事务分析规则，将 DFD 图映射为顶层和第一层模块结构。
(4) 对第一层中的模块继续细化和分解。
(5) 利用启发式规则优化初始系统结构图。

9.4.3.1 变换分析实例

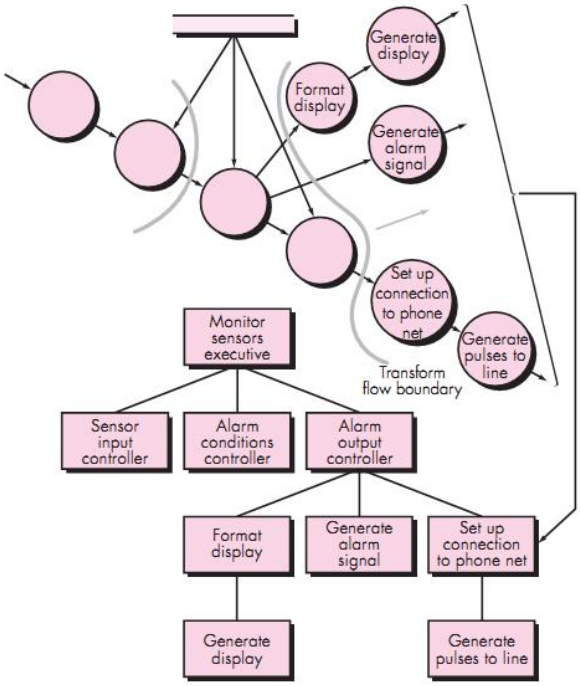
Step 1: 标识流边界



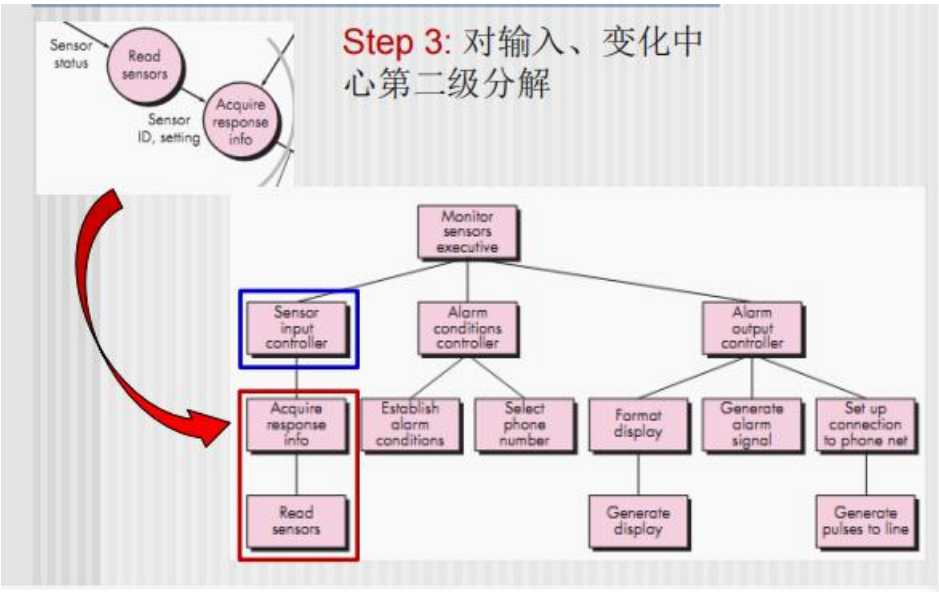
Step 2: 第一级分解



Step 3: 对输出模块第二级分解

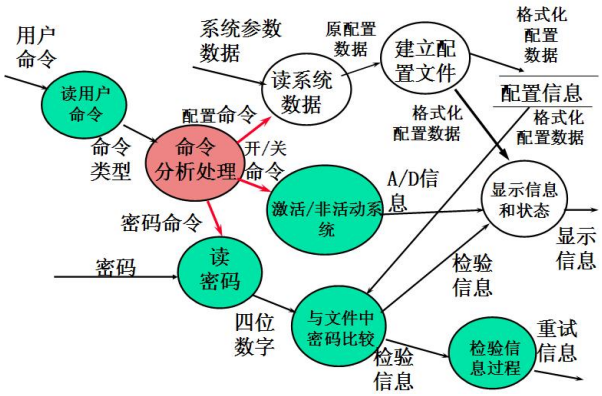


Step 3: 对输入、变化中心第二级分解



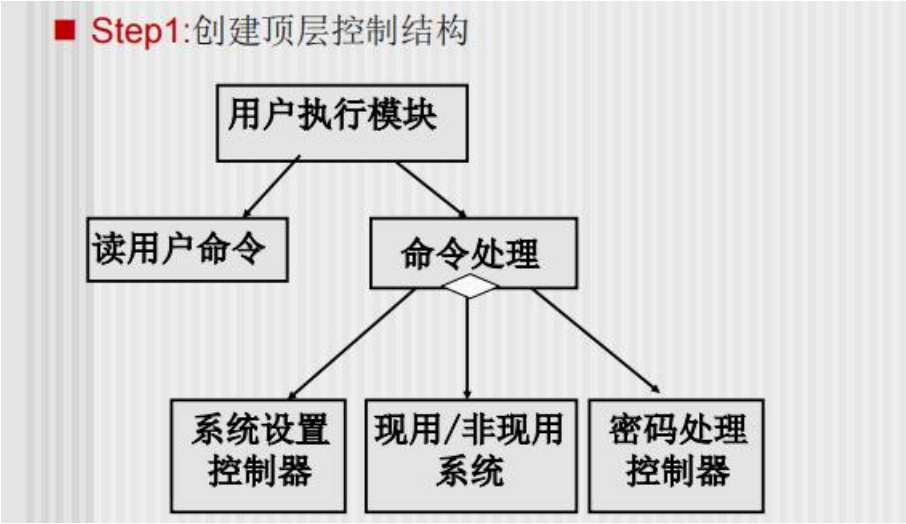
9.4.3.2 事务分析实例

假定某工控系统 DFD 图如下

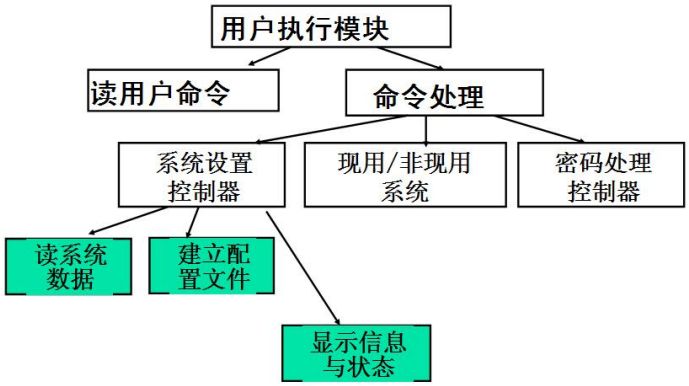


9.4.3.2 事务分析实例

Step1:创建顶层控制结构



Step 2:对输出分支进行细化



Chapter 10 构件设计

10.1 引入构件设计

构件设计是在体系结构设计的基础上，对其中每个构件内部构成的进一步设计。

体系结构设计相当于确定计算机的部件组成，连接关系等。

构件设计则着重于设计每个部件的内部细节，如运算器的内部电路和元件等。

10.1.1 什么是构件

构件是计算机软件中的一个模块化的构造块。它部件封装了实现并暴露一系列接口。

构件是一个抽象的概念，在程序中可以指程序函数、模块、类等。

面向对象视角 - 构件封装了一组相互协作的类（也可以是一个类）。

结构化视角

- 构件也称模块（如图像读入模块、图像压缩模块）封装了数据结构、算法和接口

10.1.2 结构化视角下构件设计

构件设计目标：细化构件(模块)内部的数据结构、算法和接口

10.1.3 面向对象视角下构件设计

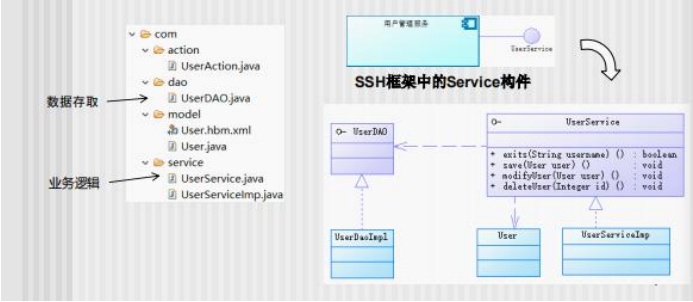


设计元素包括子系统、构件、设计类

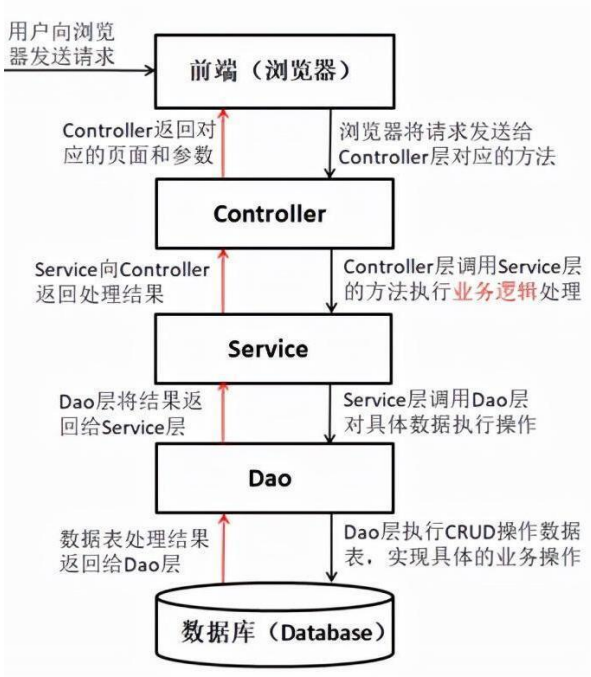
构件是设计类的集合

构件接口与实现需要严格分离。

一个构件中的职责需要多个设计类协同完成。



SSH 框架中的 Service 构件



10.1.4 面向对象构件设计任务

通过设计优化，可以提高构件复用性，灵活适应系统变更，并减少副作用的传播。

10.2 构件设计指导原则

(1)开闭原则（OCP）

对扩展开放：构件的行为是可扩展或改变的

对修改关闭：当构件改变行为时，不要修改其源码

实现途径: 用抽象构建框架，用继承/实现扩展细节。

- 需求变化时，不是通过修改类本身来完成，而是通过扩展一个子类来改变构件的行为。

优点：不会对旧逻辑造成影响避免产生 bug，同时达到扩展功能的效果。

(2)LSP:Liskov Substitution Principle

LSP 原则用来指导继承关系中子类该如何设计，子类的设计要保证在替换父类的时候，不改变原有程序的逻辑以及不破坏原有程序的正确性。

子类可以扩展父类的功能，但不能改变父类原有的功能。

LSP 原则能减少代码冗余，避免运行期的类型判别（实现动态加载）。

(3) 依赖倒置原则：Dependency Inversion Principle

核心思想：依赖于抽象，而非具体实现。

具体方法：

首先，分离接口和实现。

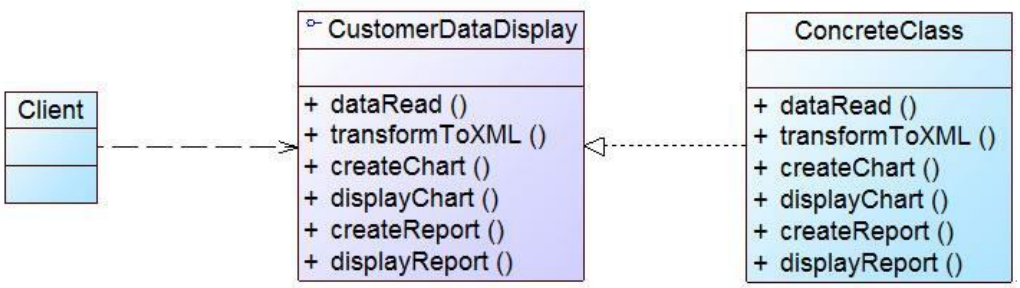
将类之间的依赖关系建立在高层抽象（如抽象类、接口）基础上，从而避免直接耦合。

依赖倒置原则实现

(4) 接口分离原则 Interface Segregation Principle

核心思想：接口尽量细化，解开耦合，每个对象只关心自身关注的接口。

优势：避免客户端依赖于不需要的接口。



(5) 单一职责原则

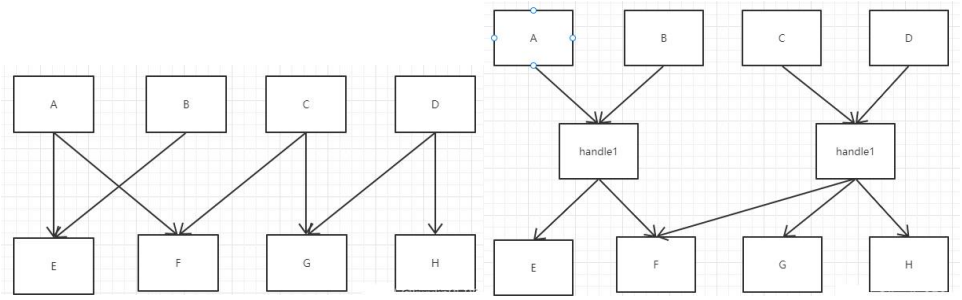
核心思想： 一个类只应承担一类职责。

如果某类有两个职责：p1，p2。一旦职责 p1 的需求发生改变需要修改该类时，可能导致原本运行正常的职责 P2 发生故障。

(6) 迪米特法则

核心思想：一个对象应该对其他对象保持最少的了解，从而减小类之间的耦合。

通过增加“中间类”将直接耦合的类隔开，后期只要修改中间类即可。



上述指导原则的主要目的就是为了保证类结构满足高内聚、低耦合。

内聚性和耦合性

内聚性意味着构件只封装那些密切相关的类，或者类只封装自身密切相关的属性和操作。

根据程度可细分为：功能内聚、分层内聚、通信内聚等。

耦合性是类之间彼此联系程度的一种定性度量。根据程度可细分为：内容耦合、共用耦合、控制耦合、标记耦合等。

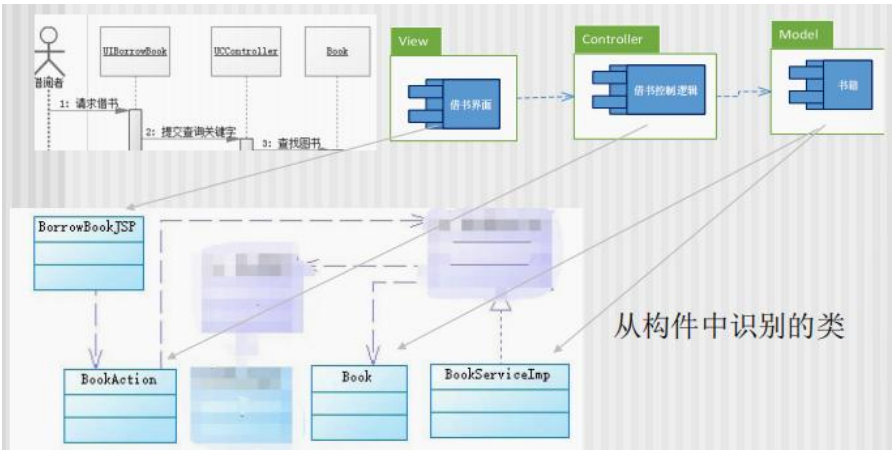
Step 1. 识别构件中与问题域相对应的设计类。 即将分析类图中的界面类、控制类、实体类均视为构件，并细化为对应的设计类。

一般每个用例考虑一个控制类，但如果多个用例功能相似（如增删改），可以共用一个控制类。

如果实体类的职责过于复杂，应该将其分解为多个设计类（依据单一职责原则）,否则无须分解。

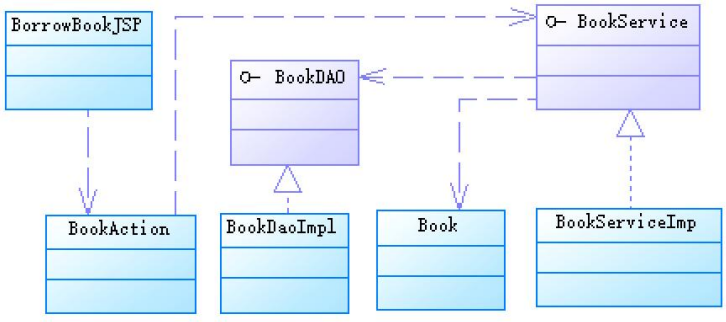
10.3 面向对象构件设计过程

Step 1. 识别构件中与问题域相对应的设计类。



Step 2. 识别构件中与基础设施对应的设计类。

基础设施类在分析模型中并不涉及，因此设计时需要补充。



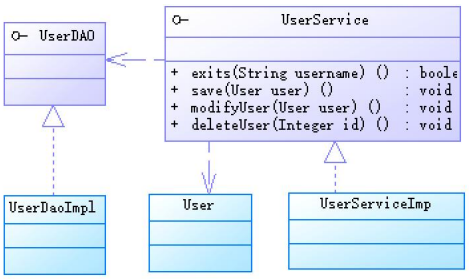
Step 3. 找到所有设计类后，转入对设计类的细化步骤，即定义设计类中接口、属性和操作。

Step 3a. 通过设计时序图或协作图说明协作细节。

将分析阶段时序图中职责进一步分解为具体操作，并分配给相关类；

定义操作对应的方法名，方法参数，返回值、可见性等。

Step 3b. 为每个构件确定合适的接口。



简单构件可直接定义 public 方法作为接口；

复杂构件单独定义接口类，构件负责实现该接口；

确定接口的参数；

Step 3c. 细化类的属性，并定义相应的数据类型和数据结构。

Step 3d. 描述类成员方法中的处理逻辑。

绘制程序流程图或活动图

如定义类 PrintJob 中的方法 computePaperCost()

Step 4. 说明持久数据源 (数据库或文件)，并确定管理数据源所需要的类。

对象可暂存于内存中，但要持久保存，需要通过数据存储实现。

可以通过对象关系映射（ORM）将类映射为表，以简化数据库使用。

Step 5. 细化部署图以提供额外的实现细节。

为系统选择硬件配置和运行平台；

将构件（如类、包）分配到各个硬件结点上

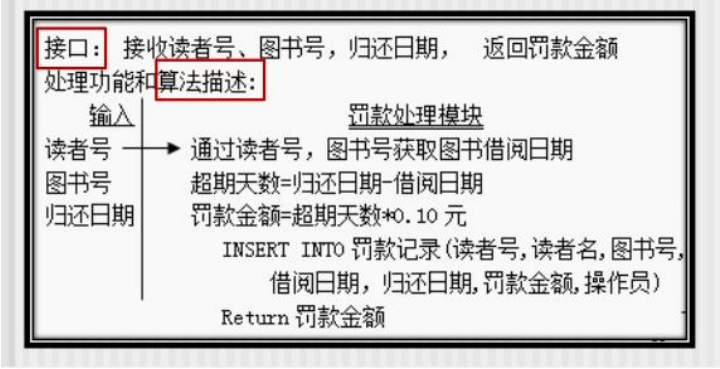
10.4 传统构件设计过程

设计目标

确定每个模块的算法，用工具描述算法逻辑。

确定每一模块的数据结构。

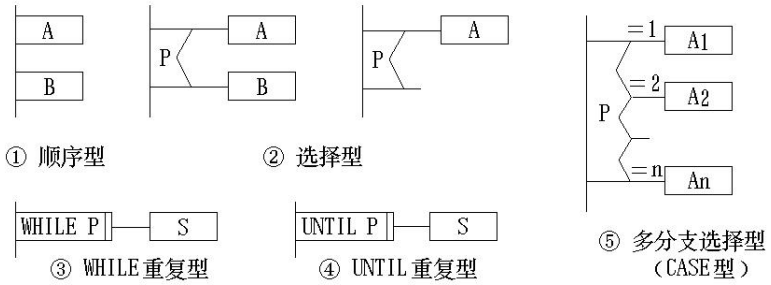
确定模块接口。



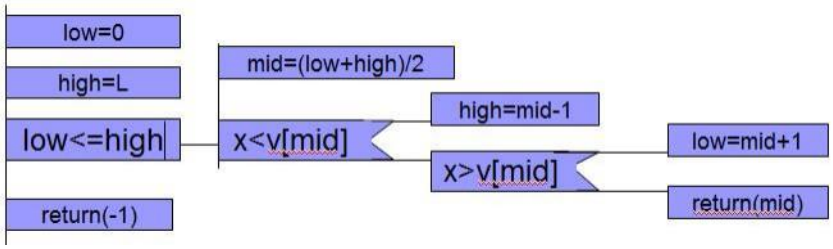
10.5 传统构件设计过程

常用算法描述工具

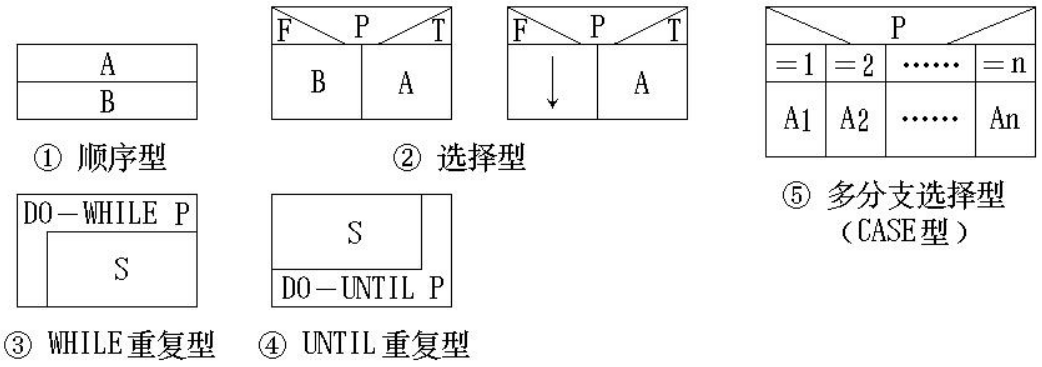
程序流程图



问题分析图(PAD)



盒图(N-S 图)



伪代码(PDL)

Chapter 11 用户接口设计

11.1 UI 设计

软件强大的计算能力和功能，最终要通过界面来表现。
用户界面是软件的直观形象，直接影响用户对软件的体验。
用户通常从用户界面（而非功能）的好坏来评价软件的质量。

11.2 常见的 UI 设计错误

lack of consistency
too much memorization

no guidance / help
no context sensitivity
poor response

11.3 黄金规则

Place the user in control　让用户驾驭软件，而非软件驾驭用户
Reduce the user's memory load
Make the interface consistent

11.3.1 让用户掌控一切

Provide for flexible interaction. 允许用户选择使用键盘、语音、鼠标等多种交互方式

Allow interaction to be interruptible and undoable. 如在手机编写短信时，允许接听电话，并能找到之前正在编写的短信

Streamline interaction as skill levels advance and allow the interaction to be customized. 允许定制交互.

Hide technical internals from the casual user.

Design for direct interaction with objects that appear on the screen. 允许用户直接与屏幕对象交互

11.3.2 减少用户记忆

Reduce demand on short-term memory.
Establish meaningful defaults.
Define shortcuts that are intuitive. 如 ctrl+S
尽量使用真实世界的比喻. 如图标
以一种渐进的方式揭示信息.

11.3.3 保持一致性

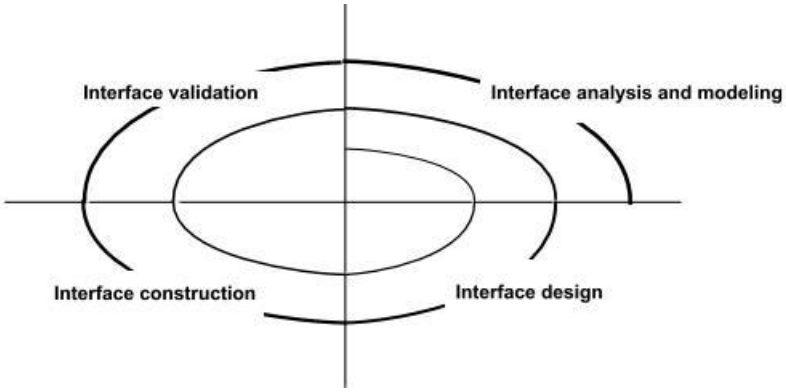
在完整的产品线上保持设计规则的一致性.

如果过去的交互模式已经建立起用户期望，除非有不得已的理由，否则不要改变它.

11.3.4 补充规则

反馈 对用户的操作，应该给予及时、明确的反馈，并提供上下文帮助。
宽容性 允许用户犯错，并能帮助用户从错误中及时恢复出来
个性化 允许用户根据自己的喜好定制界面元素，布局等
美观性
简洁法则

11.4 UI 设计过程



11.4.1 界面分析

Interface analysis means understanding
(1) the people (end-users) who will interact with the system through the interface;
(2) the tasks that end-users must perform to do their work,
(3) the content that is presented as part of the interface
(4) the environment in which tasks will be conducted.

(1) 用户分析

What level of formal education does the average user have?
Are users expert typists(专业录入员) or keyboard phobic(键盘恐惧者)?
What is the age range of the user community?
Is the software to be an integral part of the work users do or will it be used only occasionally?
What are the consequences if a user makes a mistake using the system?

(2) 任务分析

根据用例的时序图，将其映射为多个人机界面上的一组子任务。

(3) 显示内容分析

分析每项子任务对应界面中的呈现内容。

11.4.2 界面设计

界面设计主要以用例模型、交互模型（如时序图）等为主要依据。
对每项任务对应的界面进行交互设计和视觉设计。
交互设计的重点包括： 交互流程设计 交互方式设计

(1) 交互设计

交互流程设计
定义界面对象和动作。
定义可能导致界面变化的事件,对应时序图中用户向界面类发送的消息。
界面应该如何变化，对应时序图中界面类向用户返回的消息。
想向用户传达什么信息？
交互方式设计
好的交互方式可以让用户更简单、方便的使用产品。
如手机滑屏退出方式 优于 按钮点击方式。
“借书”界面中 扫描枪读取方式 优于 键盘录入方式

(2) 视觉设计

视觉设计是指：针对眼睛的主观形式的表现手段和结果。
视觉设计主要关注界面的布局、数据的呈现方式、色彩搭配等

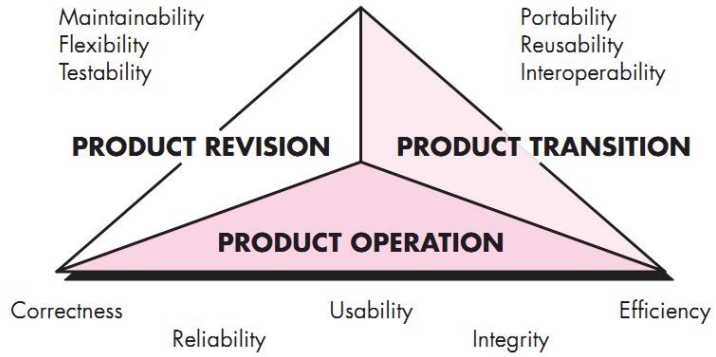
11.5 设计评审

所有设计工作完成后，为保证设计质量，需要对所有设计成果，包括体系结构、构件、接口、数据模型等进行综合设计评审。

Chapter 14 Quality Concepts

GB/T 11457-2006 中定义软件质量为：
（1）软件产品能满足给定需要的性质和特性的总体。
（2）软件具有所期望的各种属性的组合。
（3）顾客和用户觉得软件满足其综合期望的程度。
ANSI/IEEE std 729-1983 对软件质量定义为：
与软件产品满足规定的和隐含的需求能力有关的特征或特征的全体。

14.1 Quality Dimensions-McCall model



软件质量是很多种质量因素的综合体现，各种质量因素反应了软件质量的不同方面。

人们通过改善软件的各种质量因素，从而提高软件的整体质量。

14.2 Conflicts among Quality Factors

软件质量因素之间可能存在相互冲突或影响。

由于效率的要求，应该尽可能采用汇编语言。但是用汇编语言实现的程序可靠性、可移植性、可维护性等较差。

因此，实际软件开发过程中应该根据具体情况对各种质量因素进行折中或取舍，以获得用户和开发人员满足的总体质量。

14.3 Quality Model- 国标

GB/T 16260.1~3-2006/ 质量模型

质量内部标准 质量外部标准

14.4 Cost of Quality

Prevention costs include

- quality planning
- formal technical reviews
- test equipment
- Training

Internal failure costs include

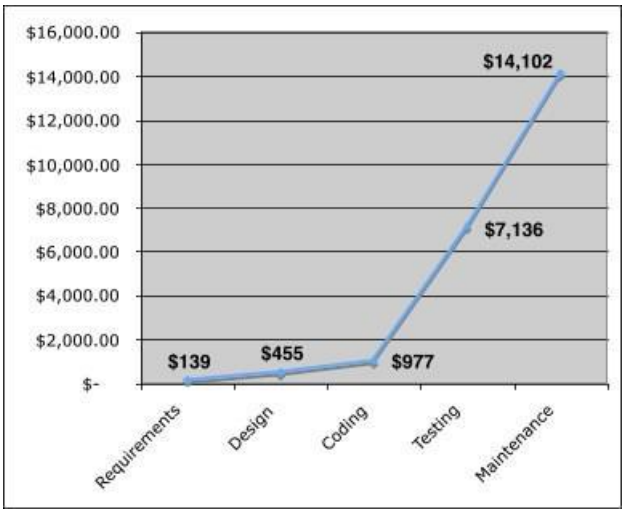
- rework
- repair
- failure mode analysis

External failure costs are

complaint resolution 投诉解决
product return and replacement 产品退换
help line support

Cost

The relative costs to find and repair an error or defect increase dramatically as we go from prevention to detection to internal failure to external failure costs.



14.5 Achieving Software Quality

Critical success factors:

Software Engineering Methods
Project Management Techniques
Quality Control
Quality Assurance

Chapter 15 Review Techniques

15.1 Quality Control

QC：包括一套软件工程活动，以帮助确保每个工作产品都符合其质量目标。
可以通过一系列指标，如缺陷率、测试覆盖率等反应产品的质量，因此 QC 是产品质量检查者。
QC 主要通过两种方式实现： 软件测试 软件评审

15.2 What Are Reviews?

Software reviews are a “filter” for the software process.

Reviews are applied at various points(如需求评审、设计评审、代码评审) during software engineering and serve to uncover errors and defects that can then be removed .

评审技术是软件早期查错最有效的一种技术。

15.3 评审的价值

早期发现错误，意味着修改的成本更低。

错误在软件过程中扩散的可能性更小。

业界统计表明：设计活动中引入的错误占软件过程中所有错误的 50~65%。

引入设计评审在发现设计缺陷方面有效率达 75%。

如：评审系统安全设计情况。

是否提供对使用系统资格进行检查的功能；

是否提供对系统内重要数据加密的功能等。

15.6 Informal Reviews

评审包括：非正式评审 和 正式评审

非正式评审适用于对较简单、还未形成正式工作成果的内容进行检查，没有正式的文档输出。

非正式评审方式包括:

desk check：由自己或同事对作者的工作产品进行评审（如对代码进行分析，依据编码规范检查源代码）

pair programming： encourages continuous review as a work product (design or code) is created.

walk through（走查）：由产品的设计者，引导其它成员浏览软件工作产品，并希望他们提给出问题反馈。

分发：又称轮查，产品的设计者将要评审的工作产品共享或分发，评审人员以修订标记或批注的方式将意见直接添加到工作产品复件上。

缺点：

由于缺乏事先规划、筹备，没有规范的会议议程，缺乏对错误进行后续的跟踪处理，非正式评审的有效性低于正式评审。

15.7 Formal Technical Reviews

正式评审在软件开发生存周期中每个阶段结束时实施，有明确的评审规程和文档化评审结果。

评审参与者

Producer—the individual who has developed the work product

Review leader—evaluates the product for readiness, generates copies of product materials, and distributes them to two or three *reviewers* for advance preparation.

Reviewer(s)—expected to spend between one and two hours reviewing the product, making notes, and otherwise becoming familiar with the work.

Recorder—reviewer who records (in writing) all important issues raised during the review.

15.8 Review Process

（1）预备：评审小组先召开一个预备会议，作者会针对工作产品向大家做一个总体的介绍，例如本产品的目标，相关的实现细节、开发标准等。
（2）审查：在预备会和正式评审会之间，评审小组成员会对工作产品进行彻底审查，并依据相关标准和准则评审工作产品。
（3）评审：在预定时间，评审小组成员以会议形式聚在一起，依次对产品进行检查，主持人负责对整个会议的进展进行控制，而记录员则负责记录下整个过程。
（4）评审记录：在工作产品中发现的每一个缺陷都会被认真记录下来，并被适当分类。
（5）返工：作者根据评审报告的决议，负责解决所有缺陷和问题。
（6）跟踪：评审组长确保所提出的每个问题都得到了圆满解决。

15.9 Review Reporting

At the end of the review meeting， a review issues list is produced. In addition, a formal technical review summary report is completed.

A review summary report answers three questions:

1. What was reviewed?
2. Who reviewed it?
3. What were the findings and conclusions?

The review issues list serves two purposes:

(1) to identify problem areas within the product and

(2) to serve as an action item checklist that guides the producer as corrections are made.

Chapter 16 Software Quality Assurance

16.1 Software Quality Assurance

好的标准和过程告诉我们应该“在什么时间、采取什么方法、做什么事情”，因此才能导致好的软件产品。

软件质量保证（SQA）：就是建立一套有计划、系统的方法，确保所拟定出的标准、步骤、实践和方法正在正确地被所有项目所采用。

SQA 类似于过程警察，主要职责是：检查开发和管理活动是否与已定的过程策略、标准和流程一致，检查工作产品是否遵循模板规定的内容和格式。

QA 与 QC 的区别：

QC：检验产品的质量，保证产品符合客户的需求；是产品质量检查者；

QA：审计过程的质量，保证过程被正确执行；是过程质量审计者；

16.2 软件质量保证活动

(1)Prepares an SQA plan for a project. 为项目制定 SQA 计划

(2)Participates in the development of the project's software process description. 参与开发项目的软件过程制定和描述

(3)Reviews software engineering activities to verify compliance with the defined software process. 评审各项软件工程活动，是否符合定义的软件过程

(4)Audits designated software work products to verify compliance with those defined as part of the software process. 审核指定的软件产品，是否符合软件过程中定义的相应部分

(5)Ensures that deviations in software work and work products are documented and handled according to a documented procedure. 确保软件工作和产品中的偏差已经被文档记录，并按规程处理

(6)Records any noncompliance and reports to senior management. 记录不符合的部分并上报

Chapter 17 软件测试策略

17.2 测试的目的

Grenford J. Myers 就软件测试提出以下观点：

测试是选择适当的测试用例并执行被测程序的过程。

目的在于发现错误，而不是为了验证程序不存在错误。

好的测试用例可以发现今未发现的错误。

软件测试的目标在于以最少的时间和人力找出软件中潜在的各种错误和缺陷。

17.3 软件测试策略

软件测试前，应先进行有效的、正式的技术评审。

测试应该应从“小规模”开始，逐步转向“大规模”。

在不同的时间点，应该采用不同的测试方法和技术。

测试应该由开发人员和独立的测试部门执行。

测试和调试是不同的活动，但任何测试策略都必须包含调试。

策略 1：测试不能取代评审

通过评审，很多错误在测试开始前就可以排除。
据说，检验项目质量的标准之一：
质量 = 项目评审时被骂的次数 / 项目评审时间

策略 2：谁来做测试

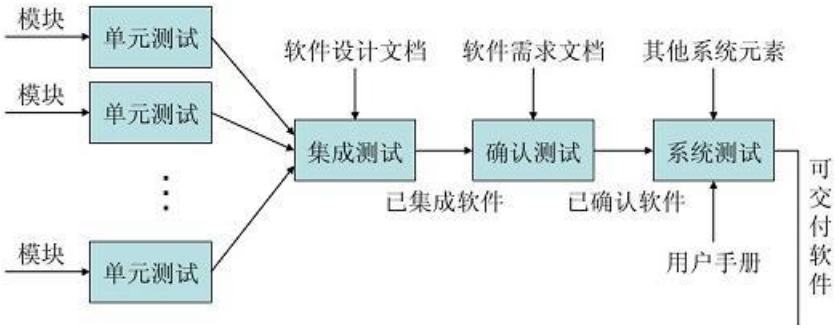
developer
independent tester

策略 3：测试什么时候开始 ？

软件测试从项目确立时就开始了，主要经过以下环节：
需求分析→测试计划→测试设计→测试环境搭建→测试执行→测试记录。

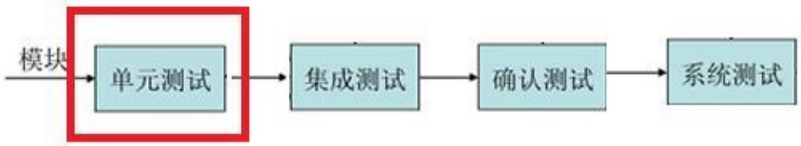
策略 4：测试过程

We begin by ‘testing-in-the-small’ and move toward ‘testing-in-the-large’



For conventional software
The module (component) is our initial focus
Integration of modules follows
For OO software
Our focus when “testing in the small” is OO class

17.4 传统软件:单元测试



测试用例=输入+预期的输出

单元测试目的

验证开发人员编写的模块或单元是否符合预期。

单元测试重点

局部数据结构

- 说明不正确或不一致；
- 初始化或缺省值错误；
- 数据类型不相容；
- 上溢、下溢或地址错误等。
- 全局数据与模块的相互影响。

单元测试环境

单元测试环境=被测模块 + 驱动模块 + 桩模块

驱动模块：用来模拟被测模块的上级调用模块。
桩模块：模拟被测试模块所调用的模块，返回被测模块所需的信息
可以通过测试工具自动生成驱动和桩模块，如 JUnit

17.5 传统软件：集成测试

为什么进行集成测试

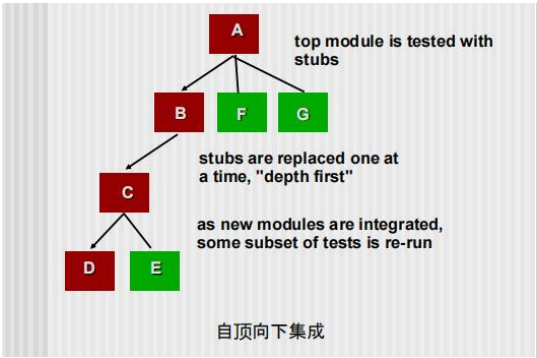
通过单元测试的模块集成在一起，仍然可能无法正常工作，因为模块相互调用时仍然可能出错。

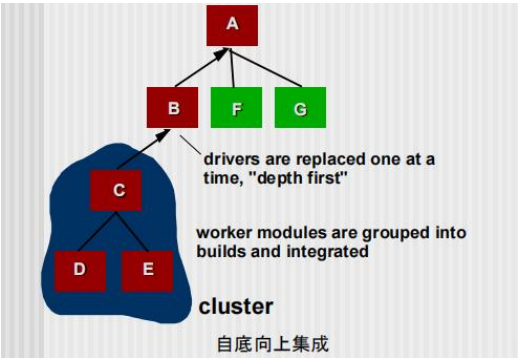
集成测试重点

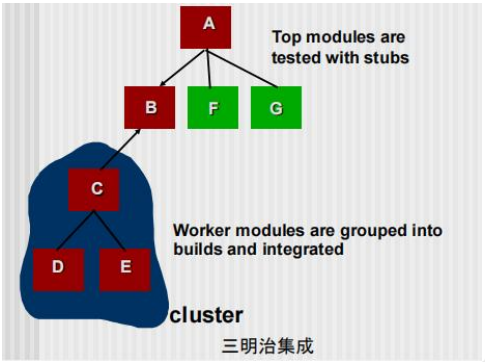
旨在发现与调用接口相关的错误。

集成策略

- the “big bang” approach
- an incremental construction strategy16







三明治集成

回归测试可视为一种特殊的集成测试。

集成测试中每加入新的模块，都会导致软件发生变更，这些变更可能使得原本正常工作的模块产生问题。

回归测试就是重新执行已经测试过的某些子集，以确保变更不会引入新的或额外的错误。

回归测试可以复用之前的测试用例和测试脚本。

可使用自动测试工具提高回归测试效率。如 WinRunner。

冒烟测试也是一种特殊的集成测试，旨在快速检查软件的核心功能是否正常运行，以确定软件是否可以进行更深入的测试(如功能、性能、安全测试等)。

通常在每个新版本或构建完成后执行。

如果冒烟测试通不过，则直接打回开发部门修复。

例如，冒烟测试内容：

检查被测系统或模块能否正常启动和退出；

数据库能否正常连接，控件能否正常加载；

是否存在严重错误或者数据严重丢失等。

17.6 面向对象软件:单元与集成测试

Class testing is the equivalent of unit testing

operations within the class are tested

Integration applied three different strategies

传统的自顶向下或自底向上的集成测试策略在面向对象软件的集成测试中无意义。

基于类之间的协作关系进行集成。

基于线程的集成—对系统一个输入或事件所需要的一组类进行集成。

基于使用关系的集成— 从独立类开始测试，再集成加入那些依赖于该独立类的其它类，直至整个系统。

17.7 确认测试

Validation testing 确认测试

Focus on software requirements

确认测试：检验所开发的软件是否满足需求规格说明中的各种功能和性能需求，以及软件配置是否完全和正确。

Alpha/Beta testing

可归为确认测试，产品正式发布前的测试，由用户主导。

α测试是由用户在开发环境下进行的测试；

β测试是由软件的多个用户在实际使用环境下进行的测试

17.8 系统测试

System testing

将软件与其它系统部分(如外部系统，硬件设计等）结合进行的整体测试。

关注非功能性需求。

系统测试包括多种类型

恢复测试　　安全测试　　压力测试　　性能测试

17.8.1 恢复测试

某些系统失效后必须在规定时间段内被恢复，否则将会导致严重的经济损失。

关注导致软件运行失败的各种条件，并验证其恢复过程能否正确执行。

实例：Word 在应用崩溃后的自我恢复功能。

如：系统供电故障后的恢复 日志处理恢复能力 程序运行崩溃后恢复能力

17.8.2 安全测试

安全测试主要用于发现软件应用程序中的漏洞、威胁和风险，并防止来自入侵者的恶意攻击。

IBM AppScan，包括众多的安全扫描产品，如针对源代码扫描 AppScan source edition，针对 WEB 应用扫描 AppScan standard edition，进行安全管理和汇总整合的 AppScan enterprise Edition。

17.8.3 压力测试

压力测试目的是为了发现系统能支持的最大负载等，前提是系统性能处在可以接受的范围(如规定 3 秒内)响应。

压力测试通过不断对系统增加压力，确定一个系统的瓶颈或者不能接受的性能点，从而获得系统最大负载能力。

最大吞吐量 tps: 不断增加并发数，加到 tps 达到一定值后开始出现下降，那个值就是最大 tps。

最大并发量：不断增加并发数，达到一定值后，服务器出现请求超时，则该值为最大的并发数。

压力测试工具：WebRunner，JMeter 等。

17.8.4 性能测试

通过测试确定系统(常态下)运行时的性能表现，如运行速度、响应时间、占有系统资源等方面的系统数据。

性能测试工具

LoadRunner：它能预测系统行为，优化性能。它通过模拟实际用户的操作行为和实行实时性能监测。

17.9 软件调试

调试目标：定位错误发生的原因和位置

17.9.1 调试技巧



（1）Brute force

一种比较简单，但效率较低的方法。包括：

① 主存信息输出法。 将计算机存储器和寄存器的全部内容打印出来。

② 关键部分设置打印语句。

③ 使用自动调试工具。

（2）Backtracking

反向跟踪是从可能的错误征兆处逐步向回追溯，直到找准错误，纠正为止。

（3）Induction(归纳法)

从线索（错误征兆出发），通过分析这些线索之间的关系找出故障。主要有4步：

① 收集数据。列出已知的测试用例和执行结果。

② 整理分析数据，以便发现规律，即“什么条件下出现。

③ 导出假设。分析线索之间的关系，提出关于错误的一个或多个假设。

④ 证明假设。证实提出的假设是导致错误发生的真正原因

（4）Deduction(演绎法）

① 设想出所有可能的出错原因；

② 试图用测试来排除每一个假设的原因，如果测试表明某个假设的原因可能是真的原因，则对数据进行细化以确定位错误。

Chapter 18 Testing Conventional Applications

18.1 关于测试的 2 个问题

（1）经过测试没有发现错误的软件就一定正确吗？

错误。测试只能证明软件是有错的,但不能证明软件是没有错误。

（2）可以通过穷举方式测试软件吗？

18.2 What is a “Good” Test

测试的有效性很大程度取决于选取的测试用例

A good A good test has a high probability of finding an error

test is not redundant.

A good test should be “best of breed” 一组相似测试用例中是最有效的

18.3 测试用例设计方法

Any engineered product can be tested in one of two ways:

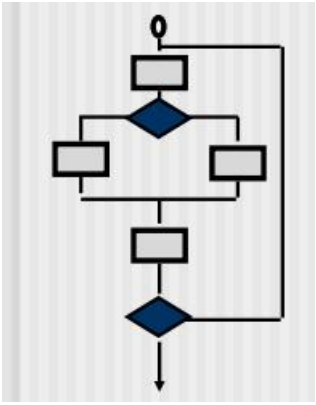
Knowing the specified function that a product has been designed

对应黑盒法：测试者把测试对象看作一黑盒子，仅依据程序的功能需求，设计测试用例并推断测试结果。

Knowing the internal workings of a product internal operations are performed according to specifications

对应白盒法：按照程序内部逻辑结构，设计测试数据并完成测试的一种测试方法。

18.3.1 白盒测试 White-Box Testing



该方法把测试对象看做一个透明的盒子，它允许测试人员利用程序内部的逻辑结构及有关信息，设计或选择测试用例。

目标是使得程序中各元素（如语句、判定）尽可能被覆盖到。

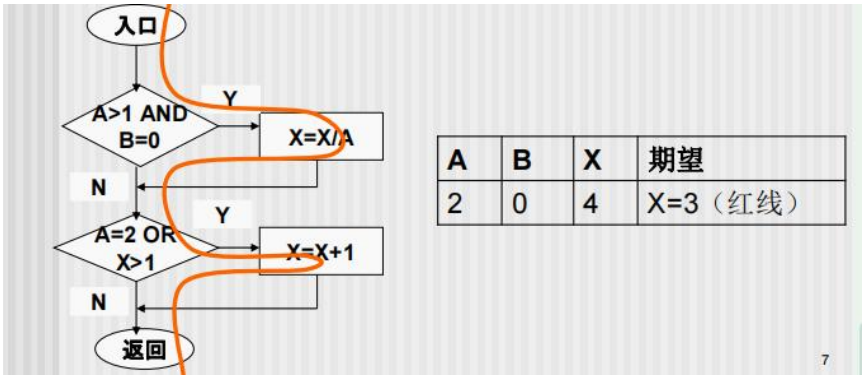
白盒测试一般在测试过程的早期执行。语句覆盖法

语句覆盖：指选择足够的测试用例，使得被测程序中每条语句至少执行一次。

是一种比较弱的测试标准。

语句覆盖法

语句覆盖：指选择足够的测试用例，使得被测程序中每条语句至少执行一次。是一种比较弱的测试标准。



A	B	X	期望
2	0	4	X=3（红线）

18.3.2 Basis Path Testing

如果程序中含有循环结构，完全的路径覆盖是不可能的。

基本路径法：在程序控制流图的基础上，通过导出基本可执行路径（注：不是所有可能路径）集合，从而设计测试用例。

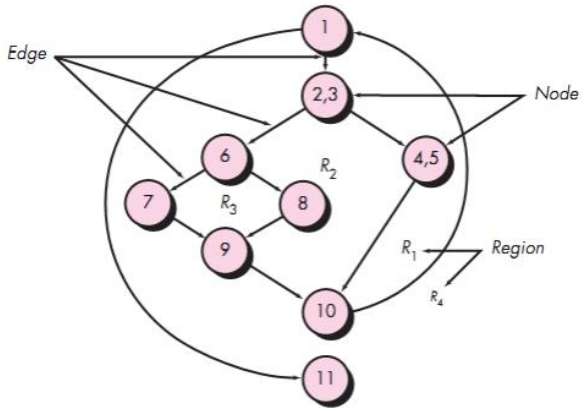
步骤如下：

Using the design or code as a foundation, draw a corresponding flow graph.

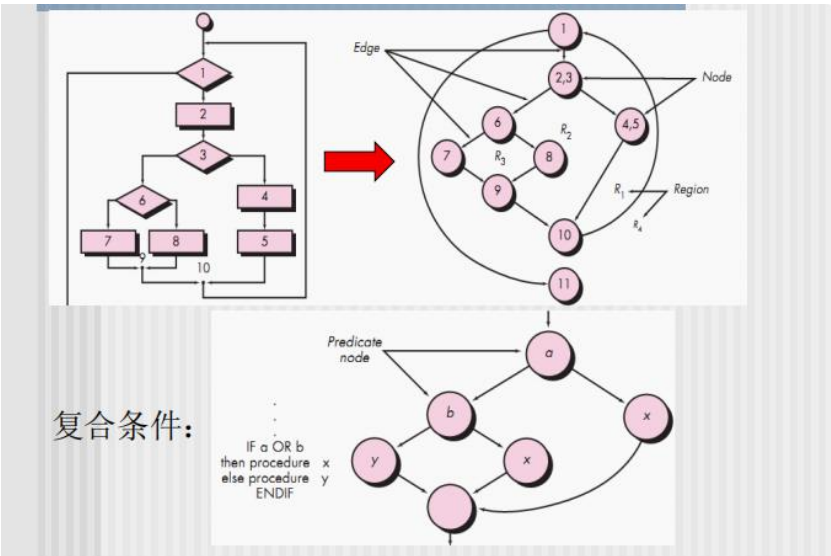
Determine the cyclomatic complexity of the resultant flow graph.

Determine a basis set of independent paths.

Prepare test cases that will force execution of each path in the basis set.



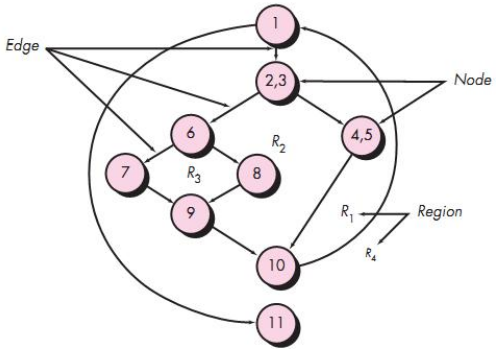
18.3.3 Flow Gragh



18.3.4 Basis Path Testing

First, we compute the cyclomatic complexity：number of simple decisions + 1 or number of enclosed areas + 1 In this case, V(G) = 4

18.3.5 Independent Path



Next, we derive the independent paths: Since V(G) = 4, there are four paths
Path 1: 1-11
Path 2: 1-2-3-4-5-10-1-11
Path 3: 1-2-3-6-8-9-10-1-11
Path 4: 1-2-3-6-7-9-10-1-11

独立路径：一条路径，至少包含一条定义 **Finally, we derive test cases to exercise these** 在该路径之前不曾用到的边（对应程序中 **paths.**至少一条新语句或条件）

18.3.6 Control Structure Testing

Condition testing — a test case design method that exercises the logical conditions contained in a program module

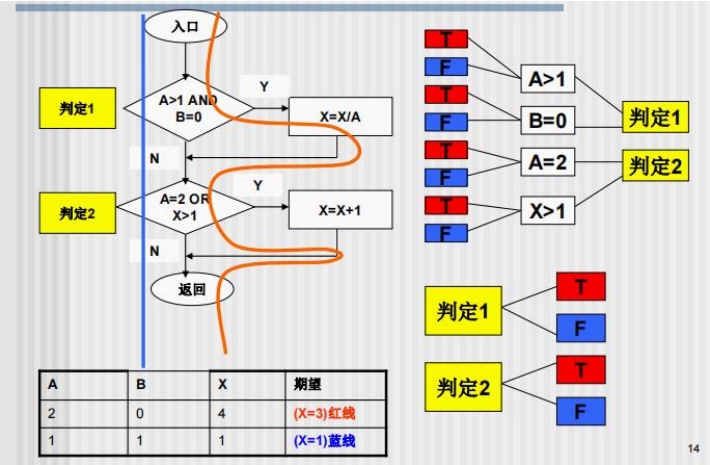
设计测试用例确保至少条件 C 为真为假执行各一次。

如果 C 是复合条件，则同时还要满足每个简单条件至少执行一次。

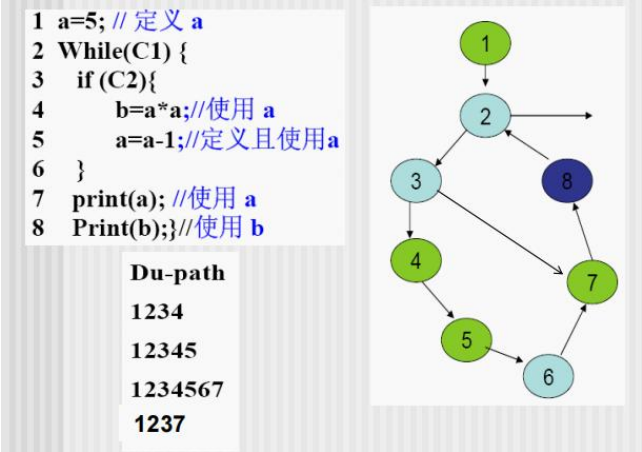
Data flow testing — selects test paths of a program according to the locations of definitions and uses of variables in the program

设计测试用例使得每个变量定义-使用链至少覆盖一次

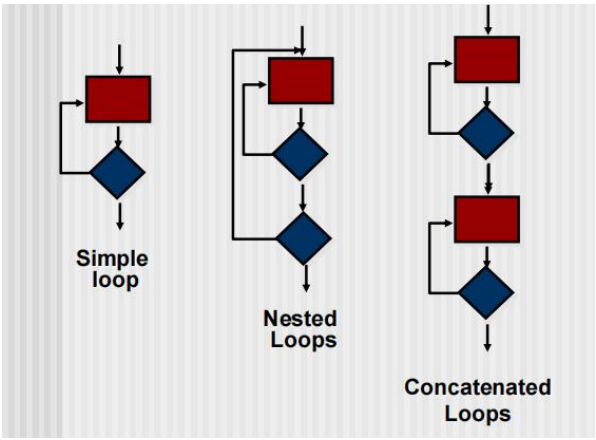
18.3.7 Condition Testing



18.3.8 Data Flow Testing



18.3.9 Loop Testing



Loop Testing: Simple Loops

Minimum conditions—Simple Loops

1. skip the loop entirely

2. only one pass through the loop

3. two passes through the loop

4. m passes through the loop m < n

5. (n-1), n, and (n+1) passes through the loop

where n is the maximum number of allowable passes

18.3.10 Black-Box Testing 黑盒测试

黑盒测试法是把测试对象看做一个黑盒子，测试人员完全不考虑程序内部的逻辑结构，只依据需求规格说明书，检查程序的功能是否符合它的功能说明。

难点：输入数据域太大？

黑盒测试一般在应用于测试的后期阶段。

1）Equivalence Partitioning

等价类划分法：将程序的输入域划分为若干个数据类，从中生成测试用例。

假定：对等价类中的代表值测试就等于对这一类值的测试。

2）等价类划分法设计实例

基本步骤:

(1)确定等价类;(2)为等价类选择一个测试用例

功能描述：某城市的电话号码由 3 部分组成。假定被测程序能接收一切符合下述规定的电话号码，拒绝所有不符合规定的电话号码。

地区码：空白或 3 位数字；

前缀：非‘0’或‘1’开头的 3 位数字；

后缀：4 位数字。

(1) 确定等价类

输入条件	有效等价类	无效等价类
地区码	空白(1) 3位数字(2)	有非数字字符(5) 少于3位数字(6) 多于3位数字(7)
前缀	从200到999的3位数字(3)	有非数字字符(8) 少于3位数字(9) 多于3位数字(10)起始位为‘0’(11) 起始位为 ‘1’ (12)
后缀	4位数字(4)	有非数字字符(13) 少于3位数字(14) 多于3位数字(15)

(2) 设计测试用例

设计测试用例，使其尽可能多的覆盖到尚未覆盖的有效等价类

设计测试用例，使其只覆盖一个尚未被覆盖的无效等价类

测试用例	测试范围	期望结果
276 2345	等价类(1) (3) (4)	有效
027 805 9321	等价类(2) (3) (4)	有效
20A 223 4356	等价类(5)	无效
剩下的10个用例	无效等价类(6)-(15)	无效

3）Boundary Value Analysis

A greater number of errors occurs at the boundaries of the input domain rather than in the "center."

Boundary value analysis leads to a selection of test cases that exercise bounding values

边界值应该尽可能选择等于边界，略小于边界，略大于边界的值

边界值法可与等价类划分法结合，作为对其的一种补充。

例：边界值分析

一个输入文件应该包含 1~255 条记录，则应用边界值分析法设计测试用例。

可以用 4 组数据进行测试：

含 1 条记录的文件

含 2 条记录的文件

含 254 条记录的文件

含 255 条记录的文件

4）Orthogonal Array Testing

Used when the number of input parameters is small and the values that each of the parameters may take are clearly bounded

正交数组测试通过挑选适量的、有代表性的点进行试验，能有效地减少测试用例数，节约测试成本。

该方法由日本口玄一博士提出并导出正交数组，从而使得测试数据具有"均匀分散、整齐可比"的特点。

Testing Example（1）

某大学刚考完某门课程，想通过“性别”、“班级”和“成绩”这 3 个查询条件对这门课程的成绩查询，查询条件包括：

“性别”=“男，女”

“班级”=“1 班，2 班”

“成绩”=“及格，不及格”

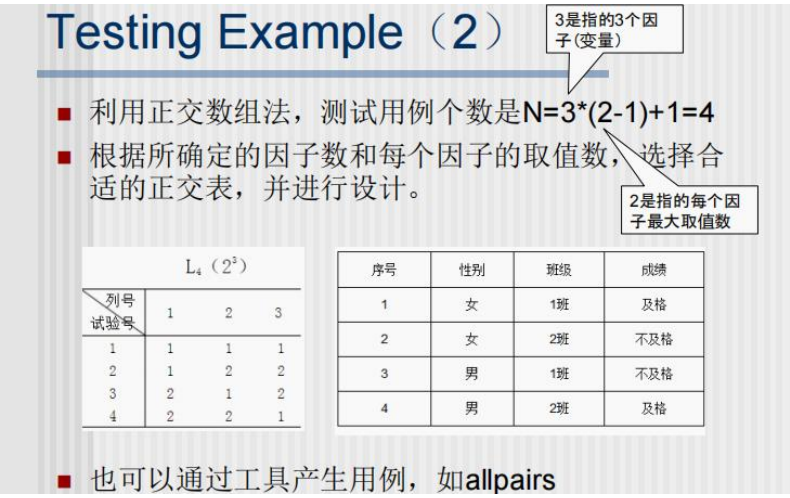
一般需要设计 2X2X2=8 个测试用例

Testing Example（2）

利用正交数组法，测试用例个数是 N=3*(2-1)+1=4

根据所确定的因子数和每个因子的取值数，选择合适的正交表，并进行设计。

也可以通过工具产生用例，如 allpairs



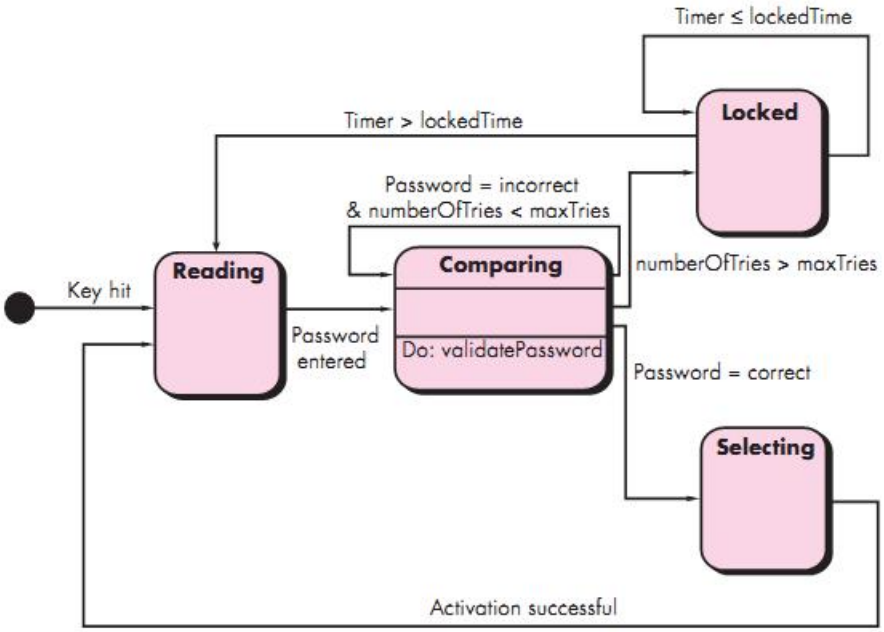
5）Model-Based Testing

Analyze an existing behavioral model for the software or create one.

Traverse the behavioral model and specify the inputs that will force the software to make the transition from state to state.

Review the behavioral model and note the expected outputs as the software makes the transition from state to state.

Execute the test cases.


Timer ≤ lockedTime
Locked
Timer > lockedTime
Password = incorrect
& numberOfTries < maxTries
Comparing
numberOfTries > maxTries
Key hit
Reading
Password
entered
Do: validatePassword
Password = correct
Selecting
Activation successful